Speech AI

From Signal Processing to Full-Duplex Dialogue

Dũng Vũ Phạm Trung

2026-05-09

Table of contents

1. Lời nói đầu

1.1. Tại sao cuốn sách này?

Trong thập kỷ qua, cộng đồng NLP đã chứng kiến sự bùng nổ của Large Language Models (LLMs) — từ BERT đến GPT-4, từ text generation đến multimodal reasoning. Nhưng có một lĩnh vực mà ranh giới giữa "ngôn ngữ" và "tín hiệu vật lý" trở nên mờ nhạt: **Speech AI**.

Speech AI không chỉ là "thêm một modality" vào LLM. Nó đòi hỏi sự hiểu biết sâu về:

- **Signal processing**: Biến đổi Fourier, mel spectrogram, MFCC
- **Sequence alignment**: CTC loss, attention-based alignment, transducers
- **Generative modeling**: VAE, normalizing flows, flow matching, GAN
- **Neural audio compression**: Vector quantization, residual VQ, codec tokens
- **Real-time systems**: Streaming inference, latency optimization, voice pipelines

Cuốn sách này được viết cho **NLP/LLM researchers** muốn nhanh chóng xây dựng kiến thức sâu về speech technologies.

1.2. Đối tượng Độc giả

Cuốn sách được thiết kế cho:

1. **NLP Data Scientists** muốn mở rộng sang speech modality
2. **LLM Researchers** quan tâm đến multimodal models (Qwen2-Audio, Moshi)
3. **ML Engineers** xây dựng voice AI pipelines cho production
4. **Graduate students** nghiên cứu speech processing

Prerequisites:

- Transformer architecture (self-attention, encoder-decoder)
- LLM training basics (tokenization, pre-training, fine-tuning)
- Python & PyTorch proficiency
- Linear algebra & probability fundamentals

1.3. Cấu trúc Cuốn sách

```
Speech AI: From Signal Processing to Full-Duplex Dialogue

  Part I: Foundations
     Ch 1: From NLP to Speech - The Conceptual Bridge
     Ch 2: Audio Signal Fundamentals

  Part II: ASR (Audio → Text)
     Ch 3: ASR Foundations (CTC, Seq2Seq, Transducers)
     Ch 4: Self-Supervised Speech (Wav2Vec 2.0, HuBERT)
     Ch 5: Whisper & Modern ASR

  Part III: TTS (Text → Audio) & Codecs
     Ch 6: TTS Foundations & Vocoders
     Ch 7: End-to-End TTS & Zero-Shot Cloning
     Ch 8: Audio Codecs & Neural Tokenization

  Part IV: Speech LLMs & Vietnamese
     Ch 9: Speech Language Models
     Ch 10: Vietnamese Speech Processing

  Part V: Production
     Ch 11: Production Speech Systems

  Appendices A-D
```

1.4. Hướng dẫn Đọc

1.4.1. Đến từ LLM/GPT research:

1. **Chương 1** (NLP-to-Speech Bridge) → bắt đầu từ đây
2. **Chương 9** (Speech LLMs) → lãnh thổ quen thuộc nhất
3. **Chương 2** (Audio Fundamentals) → bổ sung signal processing
4. Các chương còn lại theo thứ tự

1.4.2. Đến từ NLP/BERT research:

1. **Chương 1** → **Chương 2** → **Chương 4** (Wav2Vec 2.0 ≈ BERT for speech)
2. **Chương 3** → **Chương 5** → **Chương 6–8** → **Chương 9**

1.4.3. Đọc toàn bộ (comprehensive):

Chương 1 → 2 → 3 → ... → 11, theo thứ tự.

1.5. Conventions

Xuyên suốt cuốn sách, chúng tôi sử dụng các conventions sau:

> **Intuition**
>
> Giải thích trực giác về một khái niệm phức tạp.

> **Latency Warning**
>
> Cảnh báo về performance, latency, hoặc memory bottleneck trong production.

> **NLP Parallel**
>
> Mapping giữa khái niệm NLP quen thuộc và speech equivalent.

Code style:

```
# Mọi tensor operation đều có inline shape & dtype comments
x: Tensor  # [batch, time, features] - torch.float32
mel: Tensor  # [batch, n_mels, n_frames] - torch.float32
```

1.6. Nguồn Tham khảo

Cuốn sách được xây dựng dựa trên 35+ papers quan trọng trong lĩnh vực Speech AI, bao gồm Whisper [1], Wav2Vec 2.0 [2], VITS [3], VALL-E [4], EnCodec [5], và Moshi [6].

Part I.

Phần I: Nền tảng

2. From NLP to Speech — The Conceptual Bridge

2.1. 1.1 Tại sao NLP Researcher cần hiểu Speech?

Trong kỷ nguyên multimodal AI, ranh giới giữa text và speech đang dần biến mất. GPT-4o có thể nghe, nói, và suy luận đồng thời. Moshi [6] cho phép full-duplex dialogue — người và AI nói cùng lúc. Qwen2-Audio [7] hiểu audio ở mức semantic.

Tất cả đều xây dựng trên **cùng một nền tảng Transformer** mà NLP researchers đã quen thuộc. Sự khác biệt chính nằm ở **cách biểu diễn input/output** — và đó chính là cầu nối mà chương này xây dựng.

2.2. 1.2 Text vs Audio: Discrete vs Continuous

Table 2.1.: So sánh Text vs Audio

Chiều so sánh	Text (NLP)	Speech (Audio)
Input representation	Discrete tokens (BPE, ~50K vocab)	Continuous waveform (16,000 samples/sec)
Sequence length	~512 tokens cho 1 paragraph	~160,000 samples cho 10 giây
Tokenization	BPE / SentencePiece	Mel spectrogram hoặc neural codec tokens
Information density	Cao (mỗi token mang ý nghĩa)	Thấp (phần lớn là redundant acoustic info)
Alignment	1:1 (input token → output token)	Many:1 (hàng trăm frames → một từ)
Modality	1D symbolic sequence	1D continuous signal → 2D spectrogram
Key challenge	Long-range dependencies	Time-frequency analysis + alignment

> **Core Insight**
>
> Khi audio được chuyển thành discrete tokens (qua neural codecs như EnCodec), speech trở thành **bài toán language modeling** — và tất cả kỹ thuật LLM áp dụng trực tiếp.

2.3. 1.3 Ba Cách Tiếp cận Biểu diễn Audio

2.3.1. 1.3.1 Spectrogram-based

Pipeline quen thuộc nhất, tương tự cách xử lý ảnh:

$$\text{Waveform} \xrightarrow{\text{STFT}} \text{Power Spectrum} \xrightarrow{\text{Mel Filterbank}} \text{Mel Spectrogram} \tag{2.1}$$

- **Output**: Ma trận 2D kích thước $(n_{\text{mels}}, T_{\text{frames}})$
- **Ví dụ**: 10 giây audio $\rightarrow (80, 1000)$ ở 100 frames/sec
- **Sử dụng bởi**: Whisper, Tacotron 2, FastSpeech 2
- **NLP parallel**: Giống như OCR — xử lý text dưới dạng hình ảnh

2.3.2. 1.3.2 Self-Supervised Representations

Học continuous representations trực tiếp từ raw waveform:

$$\text{Waveform} \xrightarrow{\text{CNN Encoder}} \text{Latent} \xrightarrow{\text{Transformer}} \text{Contextualized Representations} \tag{2.2}$$

- **Output**: Sequence of vectors (T', d_{model}) ở ~50 fps
- **Sử dụng bởi**: Wav2Vec 2.0 [2], HuBERT [8]
- **NLP parallel**: Trực tiếp tương đương **BERT embeddings**

2.3.3. 1.3.3 Neural Codec Tokens

Nén audio thành discrete token sequences — **cầu nối trực tiếp đến LLM**:

$$\text{Waveform} \xrightarrow{\text{Encoder}} \text{Latent} \xrightarrow{\text{RVQ}} \text{Discrete Tokens} \in \{0, 1, \dots, C-1\}^{Q \times T''} \tag{2.3}$$

trong đó Q là số quantization layers (codebooks) và C là codebook size.

- **Output**: Ma trận integer (Q, T'') — ví dụ $(8, 750)$ cho 10 giây ở 75 fps
- **Sử dụng bởi**: VALL-E [4], AudioLM [9], Moshi

- **NLP parallel**: **Hoàn toàn tương đương BPE tokenization** — chỉ khác vocabulary

NLP Parallel: BPE vs Neural Codecs

	BPE (Text)	Neural Codec (Audio)
Input	Raw text	Raw waveform
Process	Statistical subword splitting	Learned compression + VQ
Output	Integer token IDs	Integer codebook indices
Vocab size	32K–128K	1024 per codebook × 8 layers
Reversible?	Lossless	Near-lossless (perceptual)
LM applicable?	Yes (GPT, BERT)	**Yes** (VALL-E, AudioLM)

2.4. 1.4 Architecture Taxonomy

```
Speech AI Models
   ASR (Audio → Text)
       CTC-based:          Conformer-CTC, DeepSpeech 2
       Encoder-Decoder:    Whisper (Transformer seq2seq)
       Self-Supervised:    Wav2Vec 2.0, HuBERT → CTC fine-tune
       Transducer:         RNN-T, Emformer (streaming)

   TTS (Text → Audio)
       Two-stage:          FastSpeech 2 + HiFi-GAN
       End-to-end:         VITS (VAE + Flow + GAN)
       LM-based:           VALL-E (codec token prediction)
       Flow/Diffusion:     F5-TTS, CosyVoice, Voicebox

   Speech LLM (Audio   Text   Audio)
       Cascaded:           ASR → LLM → TTS
       Speech-in/Text-out: Qwen2-Audio, SALMONN
       Text-in/Speech-out: VALL-E, MusicGen
       Full-duplex:        Moshi, GPT-4o voice

   Audio Codecs (Audio   Tokens)
        EnCodec:            8 RVQ layers, 75 Hz, 6 kbps
        DAC:                Improved EnCodec
        Mimi:               12.5 Hz, ultra-low latency
        SpeechTokenizer:    Semantic/acoustic disentangled
```

2.5. 1.5 NLP Speech Concept Mapping

Đây là bảng mapping quan trọng nhất trong chương này — mỗi khái niệm NLP quen thuộc đều có speech equivalent:

Table 2.3.: NLP Speech concept mapping

NLP Concept	Speech Equivalent	Ghi chú
Token	Audio frame / Codec token	1 token 1 frame (10–80ms)
BPE tokenizer	Mel spectrogram / EnCodec	Spectrogram = handcrafted; codec = learned
Embedding layer	Audio encoder (CNN)	Maps raw signal → latent space
BERT pre-training	Wav2Vec 2.0 / HuBERT	Masked prediction on speech
GPT (autoregressive LM)	VALL-E / AudioLM	Next-token prediction on codec tokens
Sequence classification	Audio classification	Speaker ID, emotion, language
Token classification	Frame classification + CTC	ASR = classify each frame → text
Seq2seq (translation)	Whisper (ASR) / TTS	Audio→Text or Text→Audio
Text generation	Audio generation	TTS, voice cloning, music
Vocabulary size	Codebook size × layers	1024 × 8 = 8192 effective
Context window	Audio duration × frame rate	30s × 100fps = 3000 frames
Positional encoding	Time encoding	Sinusoidal or learned
Attention mask	Causal / streaming mask	Streaming = limited look-ahead
Cross-attention	Encoder-decoder attention	Text conditions speech (TTS)
KV-cache	KV-cache (same!)	Autoregressive speech generation
Beam search	Beam search (same!)	ASR/TTS decoding

NLP Concept	Speech Equivalent	Ghi chú
Perplexity	Perplexity on codec tokens	Language model evaluation
BLEU/ROUGE	WER (ASR) / MOS (TTS)	Task-specific metrics
Fine-tuning	Fine-tuning (same!)	Adapter, LoRA work on speech models too
Distillation	Distil-Whisper	Same technique, applied to ASR

2.6. 1.6 Kích thước Dữ liệu: Text vs Audio

Hiểu được scale difference là rất quan trọng cho system design:

$$\text{Data rate}_{\text{text}} \approx 4 \text{ tokens/sec} \quad \text{vs} \quad \text{Data rate}_{\text{audio}} \approx 16{,}000 \text{ samples/sec} \tag{2.4}$$

Điều này có nghĩa:

Table 2.4.: So sánh kích thước dữ liệu Text vs Audio

Metric	Text	Audio (16kHz, 16-bit)
Raw data per second	~20 bytes	32,000 bytes
1 hour of data	~72 KB	115 MB
Typical dataset	~100 GB text	~680K hours (Whisper)
Tokens per second	~4 (words)	100 (mel frames) / 75 (codec)
Sequence length (10s)	~40 tokens	1,000 frames / 600 codec tokens

Latency Warning

Audio sequence dài hơn text ~25× ở mel frame level. Self-attention $O(L^2)$ trở thành bottleneck nghiêm trọng — đây là lý do Conformer [10] kết hợp convolution (local) với attention (global).

2.7. 1.7 Feature Comparison

```python
import torch
from torch import Tensor


def compare_representations(
    duration_sec: float = 1.0,
    sample_rate: int = 16000,
) -> dict[str, dict[str, int | float]]:
    """So sánh kích thước ba biểu diễn audio.

    Args:
        duration_sec: Độ dài audio (giây)
        sample_rate: Tần số lấy mẫu (Hz)

    Returns:
        Dictionary chứa thông tin từng biểu diễn
    """
    n_samples: int = int(duration_sec * sample_rate)

    # 1. Mel spectrogram
    n_mels: int = 80
    hop_length: int = 160  # 10ms frames
    n_frames: int = n_samples // hop_length  # [n_mels, n_frames] - float32
    mel_size_bytes: int = n_mels * n_frames * 4  # float32 = 4 bytes

    # 2. Self-supervised (Wav2Vec 2.0 / HuBERT)
    ssl_fps: int = 50  # 50 frames/sec (320x downsampling)
    ssl_dim: int = 768  # hidden dimension
    ssl_frames: int = int(duration_sec * ssl_fps)  # [ssl_frames, ssl_dim] - float3
    ssl_size_bytes: int = ssl_frames * ssl_dim * 4

    # 3. Neural codec tokens (EnCodec)
    codec_fps: int = 75  # 75 frames/sec
    n_codebooks: int = 8  # 8 RVQ layers
    codec_frames: int = int(duration_sec * codec_fps)  # [n_codebooks, codec_frames
    codec_size_bytes: int = n_codebooks * codec_frames * 8  # int64 = 8 bytes

    return {
        "mel_spectrogram": {
            "shape": f"({n_mels}, {n_frames})",
            "total_values": n_mels * n_frames,
            "size_bytes": mel_size_bytes,
            "fps": sample_rate // hop_length,
```

```
        },
        "self_supervised": {
            "shape": f"({ssl_frames}, {ssl_dim})",
            "total_values": ssl_frames * ssl_dim,
            "size_bytes": ssl_size_bytes,
            "fps": ssl_fps,
        },
        "codec_tokens": {
            "shape": f"({n_codebooks}, {codec_frames})",
            "total_values": n_codebooks * codec_frames,
            "size_bytes": codec_size_bytes,
            "fps": codec_fps,
        },
    }


# So sánh cho 1 giây audio
result: dict = compare_representations(duration_sec=1.0)
for name, info in result.items():
    print(f"{name:20s}: shape={info['shape']:15s}  "
          f"values={info['total_values']:>8,}  "
          f"size={info['size_bytes']:>8,} bytes  "
          f"fps={info['fps']}")
```

2.8. 1.8 Tóm tắt

Chương này đã thiết lập cầu nối khái niệm giữa NLP và Speech:

1. **Audio là continuous** — cần chuyển thành discrete representation trước khi áp dụng LM
2. **Ba cách tiếp cận**: spectrogram (handcrafted), self-supervised (learned), codec tokens (discrete)
3. **Neural codec tokens** là cầu nối trực tiếp — biến speech thành language modeling problem
4. **Mọi khái niệm NLP** đều có speech equivalent — Bảng 2.3 là tham chiếu chính

Chương tiếp theo sẽ đi sâu vào **Audio Signal Fundamentals** — nền tảng toán học cho tất cả speech processing.

3. Audio Signal Fundamentals

3.1. 2.1 Waveform & Sampling

3.1.1. 2.1.1 Tín hiệu Âm thanh

Âm thanh là sóng cơ học truyền qua không khí. Khi thu bằng microphone, tín hiệu analog liên tục được chuyển thành tín hiệu số (digital) thông qua quá trình **sampling** (lấy mẫu) và **quantization** (lượng tử hóa).

$$x[n] = x_a(n \cdot T_s), \quad n = 0, 1, 2, ... \tag{3.1}$$

trong đó $x_a(t)$ là tín hiệu analog, $T_s = 1/f_s$ là sampling period, và f_s là **sample rate** (tần số lấy mẫu).

3.1.2. 2.1.2 Nyquist Theorem

Định lý Nyquist-Shannon

Một tín hiệu có tần số tối đa $f_{\max}$ có thể được khôi phục hoàn hảo từ các mẫu rời rạc nếu và chỉ nếu:

$$f_s \geq 2f_{\max} \tag{3.2}$$

Tần số $f_N = f_s/2$ được gọi là **Nyquist frequency**.

Đối với speech processing:

Table 3.1.: Thông số chuẩn cho speech audio

Parameter	Giá trị	Lý do
Sample rate f_s	16,000 Hz	Standard cho speech models
Nyquist frequency	8,000 Hz	Đủ cho speech (F0: 85–300 Hz, formants: 5 kHz)

Parameter	Giá trị	Lý do
Bit depth	16-bit PCM	Dynamic range ~96 dB
1 giây audio	16,000 samples × 2 bytes = 32 KB	

3.1.3. 2.1.3 Pre-emphasis Filter

Bộ lọc high-pass đơn giản để bù spectral rolloff tự nhiên của giọng nói:

$$y[n] = x[n] - \alpha \cdot x[n-1], \quad \alpha \approx 0.97 \tag{3.3}$$

> NLP Parallel
>
> Pre-emphasis giống như **text normalization** — một bước tiền xử lý đơn giản nhưng cải thiện kết quả downstream. Tuy nhiên, các model hiện đại (Whisper, Wav2Vec 2.0) thường **bỏ qua** bước này.

3.2. 2.2 Discrete Fourier Transform (DFT)

3.2.1. 2.2.1 Công thức DFT

DFT chuyển tín hiệu từ miền thời gian sang miền tần số:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi kn}{N}}, \quad k = 0, 1, \ldots, N-1 \tag{3.4}$$

trong đó:

- $x[n]$: tín hiệu đầu vào (time domain)
- $X[k]$: hệ số Fourier tại frequency bin k (complex number)
- N: kích thước DFT (thường = `n_fft`)
- $j = \sqrt{-1}$: đơn vị ảo

Inverse DFT:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{j\frac{2\pi kn}{N}} \tag{3.5}$$

3.2.2. 2.2.2 Tính chất Quan trọng

- **Số frequency bins hữu ích**: $N/2 + 1$ (do conjugate symmetry cho real input)
- **Frequency resolution**: $\Delta f = f_s/N$
- **Tần số tại bin** k: $f_k = k \times f_s/N$

> **Ý nghĩa Vật lý**
>
> $|X[k]|$ cho biết **biên độ** (amplitude) của thành phần tần số f_k, còn $\angle X[k]$ cho biết **pha** (phase). Mel spectrogram chỉ giữ lại magnitude, loại bỏ phase — đó là lý do cần vocoder (như HiFi-GAN) để tái tạo waveform từ mel.

3.2.3. 2.2.3 Fast Fourier Transform (FFT)

FFT là thuật toán tính DFT hiệu quả với complexity $O(N \log N)$ thay vì $O(N^2)$:

$$\text{DFT} : O(N^2) \quad \xrightarrow{\text{FFT}} \quad O(N \log N) \tag{3.6}$$

Khi $N = 512$: DFT cần ~262K phép tính, FFT chỉ cần ~4.6K.

3.3. 2.3 Short-Time Fourier Transform (STFT)

3.3.1. 2.3.1 Tại sao cần STFT?

DFT phân tích **toàn bộ** tín hiệu → không có thông tin về **khi nào** một tần số xuất hiện. Speech là non-stationary (tần số thay đổi theo thời gian), nên cần phân tích **từng đoạn ngắn**:

$$\text{STFT}(m, k) = \sum_{n=0}^{N-1} x[n + m \cdot H] \cdot w[n] \cdot e^{-j\frac{2\pi kn}{N}} \tag{3.7}$$

trong đó:

- m: frame index (thời gian)
- k: frequency bin index (tần số)
- H: **hop length** (frame shift, tính bằng samples)
- $w[n]$: window function (cửa sổ)
- $x[n + m \cdot H]$: đoạn tín hiệu bắt đầu từ frame m

3.3.2. 2.3.2 Window Functions

Window function giảm **spectral leakage** (rò rỉ phổ):

Hann window (phổ biến nhất):

$$w[n] = 0.5 \times \left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right), \quad n = 0, 1, \ldots, N-1 \tag{3.8}$$

3.3.3. 2.3.3 Tham số Thực tế

$$\begin{aligned} f_s &= 16{,}000 \text{ Hz} \\ \text{frame\_length} &= 25 \text{ ms} \Rightarrow N = f_s \times 0.025 = 400 \\ \text{frame\_shift} &= 10 \text{ ms} \Rightarrow H = f_s \times 0.010 = 160 \\ \text{Frequency bins} &= N/2 + 1 = 201 \\ \text{Frames per second} &= f_s/H = 100 \end{aligned} \tag{3.9}$$

3.3.4. 2.3.4 Power Spectrogram

$$P(m,k) = |\text{STFT}(m,k)|^2 = \text{Re}(\text{STFT})^2 + \text{Im}(\text{STFT})^2 \tag{3.10}$$

Dimensions:

- Input: waveform x có T_{samples} samples
- Output: P có shape $(N/2+1, T_{\text{frames}})$
- $T_{\text{frames}} = \lfloor (T_{\text{samples}} - N)/H \rfloor + 1$

3.4. 2.4 Mel Scale & Mel Spectrogram

3.4.1. 2.4.1 Mel Scale

Tai người cảm nhận tần số theo thang **logarithmic**, không linear. Mel scale mô hình hóa điều này:

$$\text{mel}(f) = 2595 \times \log_{10}\left(1 + \frac{f}{700}\right) \tag{3.11}$$

Inverse:

$$f(\text{mel}) = 700 \times \left(10^{\text{mel}/2595} - 1\right) \tag{3.12}$$

> **Ý nghĩa Trực giác**
>
> Ở tần số thấp (< 1 kHz), mel scale gần linear — tai phân biệt tốt. Ở tần số cao (> 1 kHz), mel scale nén lại — tai kém nhạy hơn. Đây là lý do mel filterbank dùng narrow filters ở low freq và wide filters ở high freq.

3.4.2. 2.4.2 Mel Filterbank

Tập hợp M bộ lọc tam giác trên mel scale:

$$H_m(k) = \begin{cases} 0 & \text{if } f(k) < f(m-1) \\ \frac{f(k)-f(m-1)}{f(m)-f(m-1)} & \text{if } f(m-1) \leq f(k) \leq f(m) \\ \frac{f(m+1)-f(k)}{f(m+1)-f(m)} & \text{if } f(m) \leq f(k) \leq f(m+1) \\ 0 & \text{if } f(k) > f(m+1) \end{cases} \tag{3.13}$$

trong đó $f(m)$ là center frequency của filter m, phân bố đều trên mel scale.

3.4.3. 2.4.3 Mel Spectrogram

$$S_{\text{mel}}(m,t) = \log\left(\sum_{k=0}^{N/2} H_m(k) \cdot P(t,k) + \epsilon\right) \tag{3.14}$$

với $\epsilon = 10^{-10}$ để tránh $\log(0)$.

Dimensions tiêu biểu:

- $M = 80$ mel filters (Whisper, VITS) hoặc $M = 128$ (một số model)
- Input: Power spectrogram $(201, T_{\text{frames}})$
- Output: Mel spectrogram $(80, T_{\text{frames}})$ — torch.float32

```
import torch
import torchaudio
from torch import Tensor


def compute_mel_spectrogram(
    waveform: Tensor,          # [1, T_samples] - torch.float32
    sample_rate: int = 16000,
    n_fft: int = 400,          # 25ms window
    hop_length: int = 160,     # 10ms hop
```

```
    n_mels: int = 80,
) -> Tensor:
    """Tính log mel spectrogram từ waveform.

    Args:
        waveform: Audio tensor [1, T_samples] - float32
        sample_rate: Tần số lấy mẫu (Hz)
        n_fft: FFT size
        hop_length: Hop length (samples)
        n_mels: Số mel filters

    Returns:
        log_mel: Log mel spectrogram [1, n_mels, T_frames] - float32
    """
    mel_transform = torchaudio.transforms.MelSpectrogram(
        sample_rate=sample_rate,
        n_fft=n_fft,
        hop_length=hop_length,
        n_mels=n_mels,
        power=2.0,  # power spectrogram
    )

    mel_spec: Tensor = mel_transform(waveform)  # [1, n_mels, T_frames] - float32

    # Log compression
    log_mel: Tensor = torch.log(mel_spec + 1e-10)  # [1, n_mels, T_frames] - float3

    return log_mel


# Example: 1 giây audio
waveform: Tensor = torch.randn(1, 16000)  # [1, 16000] - float32
log_mel: Tensor = compute_mel_spectrogram(waveform)
print(f"Input shape:  {waveform.shape}")   # [1, 16000]
print(f"Output shape: {log_mel.shape}")    # [1, 80, 100]
```

3.5. 2.5 MFCC (Mel-Frequency Cepstral Coefficients)

3.5.1. 2.5.1 Từ Mel đến MFCC

MFCC áp dụng **Discrete Cosine Transform (DCT)** lên log mel spectrogram để decorrelate các mel bands:

$$\text{MFCC}(c,t) = \sum_{m=0}^{M-1} S_{\text{mel}}(m,t) \cdot \cos\left(\frac{\pi c(2m+1)}{2M}\right) \tag{3.15}$$

trong đó $c = 0, 1, \ldots, C-1$ là MFCC index (thường $C = 13$ hoặc $C = 40$).

3.5.2. 2.5.2 MFCC vs Mel Spectrogram

Table 3.2.: Mel Spectrogram vs MFCC

Feature	Mel Spectrogram	MFCC
Dimensions	$(80, T)$	$(13, T)$
Correlation	Correlated bands	Decorrelated
Information	Full spectral detail	Compressed envelope
Used by	Whisper, VITS, modern models	GMM-HMM (classic ASR)
Deep learning	**Preferred**	Legacy, rarely used now

> ⚠ Latency Warning
>
> MFCC bỏ mất thông tin spectral detail mà deep models có thể khai thác. Hầu hết models hiện đại (Whisper, Conformer, VITS) sử dụng **mel spectrogram trực tiếp** thay vì MFCC.

3.6. 2.6 SpecAugment

SpecAugment [11] là kỹ thuật data augmentation cực kỳ hiệu quả cho ASR — giảm WER 10–25% relative mà không cần thêm dữ liệu.

3.6.1. 2.6.1 Ba Phép Augmentation

1. **Time warping**: Biến dạng nhẹ theo trục thời gian
2. **Frequency masking**: Che F consecutive mel bands
3. **Time masking**: Che T consecutive time steps

$$\begin{aligned} \text{Freq mask}: \quad S'(m,t) &= \begin{cases} 0 & \text{if } f_0 \le m < f_0 + F \\ S(m,t) & \text{otherwise} \end{cases} \\ \text{Time mask}: \quad S'(m,t) &= \begin{cases} 0 & \text{if } t_0 \le t < t_0 + T \\ S(m,t) & \text{otherwise} \end{cases} \end{aligned} \tag{3.16}$$

trong đó f_0, t_0 được chọn ngẫu nhiên, F và T là mask widths.

> **NLP Parallel**
>
> SpecAugment tương đương **token dropout / span masking** trong NLP. Frequency masking masking specific features, time masking masking contiguous tokens.

```python
import torch
from torch import Tensor


def spec_augment(
    mel: Tensor,              # [batch, n_mels, T] - torch.float32
    freq_mask_param: int = 27,
    time_mask_param: int = 100,
    n_freq_masks: int = 2,
    n_time_masks: int = 2,
) -> Tensor:
    """Apply SpecAugment to mel spectrogram.

    Args:
        mel: Mel spectrogram [B, M, T] - float32
        freq_mask_param: Max frequency mask width F
        time_mask_param: Max time mask width T
        n_freq_masks: Number of frequency masks
        n_time_masks: Number of time masks

    Returns:
        augmented: Augmented mel spectrogram [B, M, T] - float32
```

```
    """
    augmented: Tensor = mel.clone()  # [B, M, T] - float32
    B, M, T = augmented.shape

    for _ in range(n_freq_masks):
        f: int = torch.randint(0, freq_mask_param, (1,)).item()
        f0: int = torch.randint(0, max(1, M - f), (1,)).item()
        augmented[:, f0:f0 + f, :] = 0.0  # mask freq bands

    for _ in range(n_time_masks):
        t: int = torch.randint(0, time_mask_param, (1,)).item()
        t0: int = torch.randint(0, max(1, T - t), (1,)).item()
        augmented[:, :, t0:t0 + t] = 0.0  # mask time steps

    return augmented  # [B, M, T] - float32
```

3.7. 2.7 Complete Feature Extraction Pipeline

```
import torch
import torch.nn as nn
from torch import Tensor


class AudioFeatureExtractor(nn.Module):
    """Complete audio feature extraction pipeline.

    Waveform → STFT → Mel Filterbank → Log → SpecAugment
    """

    def __init__(
        self,
        sample_rate: int = 16000,
        n_fft: int = 400,
        hop_length: int = 160,
        n_mels: int = 80,
        apply_augment: bool = True,
    ) -> None:
        super().__init__()
        self.sample_rate: int = sample_rate
        self.n_fft: int = n_fft
        self.hop_length: int = hop_length
```

```
        self.n_mels: int = n_mels
        self.apply_augment: bool = apply_augment

        # Hann window
        self.register_buffer(
            "window",
            torch.hann_window(n_fft),  # [n_fft] - float32
        )

        # Mel filterbank matrix
        mel_fb: Tensor = self._create_mel_filterbank()
        self.register_buffer("mel_fb", mel_fb)  # [n_mels, n_fft//2+1] - float32

    def _create_mel_filterbank(self) -> Tensor:
        """Create mel filterbank matrix.

        Returns:
            mel_fb: [n_mels, n_fft//2+1] - float32
        """
        n_freqs: int = self.n_fft // 2 + 1

        # Mel scale boundaries
        mel_low: float = 0.0
        mel_high: float = 2595.0 * torch.log10(
            torch.tensor(1.0 + self.sample_rate / 2.0 / 700.0)
        ).item()

        mel_points: Tensor = torch.linspace(
            mel_low, mel_high, self.n_mels + 2
        )  # [n_mels + 2]
        hz_points: Tensor = 700.0 * (10 ** (mel_points / 2595.0) - 1)  # [n_mels +

        freq_bins: Tensor = torch.linspace(
            0, self.sample_rate / 2, n_freqs
        )  # [n_freqs]

        fb: Tensor = torch.zeros(self.n_mels, n_freqs)  # [n_mels, n_freqs]

        for m in range(self.n_mels):
            f_left: float = hz_points[m].item()
            f_center: float = hz_points[m + 1].item()
            f_right: float = hz_points[m + 2].item()
```

```
            # Rising slope
            mask_up: Tensor = (freq_bins >= f_left) & (freq_bins <= f_center)
            fb[m, mask_up] = (freq_bins[mask_up] - f_left) / (f_center - f_left + 1e-10)

            # Falling slope
            mask_down: Tensor = (freq_bins > f_center) & (freq_bins <= f_right)
            fb[m, mask_down] = (f_right - freq_bins[mask_down]) / (f_right - f_center + 1e-10)

        return fb  # [n_mels, n_freqs] - float32

    def forward(self, waveform: Tensor) -> Tensor:
        """Extract log mel spectrogram from waveform.

        Args:
            waveform: [batch, T_samples] - float32

        Returns:
            log_mel: [batch, n_mels, T_frames] - float32
        """
        # STFT
        stft: Tensor = torch.stft(
            waveform,
            n_fft=self.n_fft,
            hop_length=self.hop_length,
            window=self.window,
            return_complex=True,
        )  # [batch, n_fft//2+1, T_frames] - complex64

        # Power spectrogram
        power: Tensor = stft.abs().pow(2)  # [batch, n_fft//2+1, T_frames] - float32

        # Mel filterbank: [n_mels, n_freqs] @ [batch, n_freqs, T]
        mel: Tensor = torch.matmul(
            self.mel_fb, power
        )  # [batch, n_mels, T_frames] - float32

        # Log compression
        log_mel: Tensor = torch.log(mel + 1e-10)  # [batch, n_mels, T_frames] - float32

        return log_mel
```

3.8. 2.8 Tóm tắt

Table 3.3.: Tóm tắt các phép biến đổi audio

Khái niệm	Công thức chính	Kết quả
Sampling	$x[n] = x_a(nT_s)$	Waveform [T_samples]
DFT	$X[k] = \sum x[n]e^{-j2\pi kn/N}$	Frequency spectrum [N/2+1]
STFT	DFT per frame with window	Spectrogram [N/2+1, T_frames]
Mel scale	$\text{mel}(f) = 2595 \log_{10}(1 + f/700)$	Perceptual frequency
Mel spectrogram	$\log(\mathbf{H} \cdot P + \epsilon)$	Features [n_mels, T_frames]
MFCC	DCT on log mel	Coefficients [C, T_frames]
SpecAugment	Random freq/time masking	Augmented features

Với nền tảng signal processing này, chương tiếp theo sẽ xây dựng **ASR Foundations** — cách chuyển mel spectrogram thành text.

Part II.

Phần II: Nhận dạng Giọng nói (ASR)

4. ASR Foundations

4.1. 3.1 Bài toán ASR

Automatic Speech Recognition (ASR) là bài toán ánh xạ từ tín hiệu âm thanh sang chuỗi ký tự/từ:

$$\hat{Y} = \arg\max_{Y} P(Y \mid X) \tag{4.1}$$

trong đó $X = (x_1, x_2, \dots, x_T)$ là chuỗi acoustic frames và $Y = (y_1, y_2, \dots, y_U)$ là chuỗi text tokens, với $T \gg U$ (audio dài hơn text rất nhiều).

Thách thức chính: Alignment giữa X và Y không biết trước — không biết frame nào tương ứng với ký tự nào.

> NLP Parallel
>
> ASR tương đương bài toán **sequence-to-sequence** trong NLP (machine translation), nhưng với input sequence dài hơn output 10–50×. Đây chính là lý do CTC và Transducers ra đời — giải quyết **monotonic alignment** mà standard seq2seq không xử lý tốt.

4.2. 3.2 CTC — Connectionist Temporal Classification

4.2.1. 3.2.1 Ý tưởng Cốt lõi

CTC [12] giải quyết alignment problem bằng cách:

1. **Thêm blank token** $\langle b \rangle$ vào vocabulary
2. Cho phép model output blank hoặc repeated characters tại mỗi frame
3. **Marginalize** trên tất cả valid alignments

4.2.2. 3.2.2 CTC Alignment

Cho output vocabulary $\mathcal{V} = \{a, b, c, \dots, z, \langle b \rangle\}$ và acoustic frames X có T frames:

- Model outputs: $P(c_t \mid X)$ cho mỗi frame t và mỗi character $c_t \in \mathcal{V} \cup \{\langle b \rangle\}$
- Một **CTC path** $\pi = (\pi_1, \pi_2, \dots, \pi_T)$ với $\pi_t \in \mathcal{V} \cup \{\langle b \rangle\}$

Collapsing function $\mathcal{B}$: loại bỏ repeated characters và blanks:

$$\mathcal{B}(\langle b \rangle h \langle b \rangle e \langle b \rangle ll \langle b \rangle lo \langle b \rangle) = \text{"hello"} \tag{4.2}$$

4.2.3. 3.2.3 CTC Loss

CTC loss marginalize trên tất cả valid paths:

$$P_{\text{CTC}}(Y \mid X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^{T} P(\pi_t \mid X) \tag{4.3}$$

$$\mathcal{L}_{\text{CTC}} = -\log P_{\text{CTC}}(Y \mid X) \tag{4.4}$$

> **Tại sao cần Blank Token?**
>
> Không có blank, model không thể phân biệt "hello" (1 chữ l) và "helllo" (2 chữ l liên tiếp). Blank cho phép "reset" — mỗi non-blank emission sau blank (hoặc khác character) là một ký tự mới.

4.2.4. 3.2.4 Forward-Backward Algorithm

Tính $P_{\text{CTC}}(Y \mid X)$ bằng dynamic programming. Định nghĩa modified label sequence Z bằng cách chèn blank giữa mỗi label:

$$Z = (\langle b \rangle, y_1, \langle b \rangle, y_2, \langle b \rangle, \dots, \langle b \rangle, y_U, \langle b \rangle) \tag{4.5}$$

có độ dài $|Z| = 2U + 1$.

Forward variable:

$$\alpha(t, s) = \sum_{\substack{\pi_{1:t}: \\ \mathcal{B}(\pi_{1:t}) = Z_{1:s}}} \prod_{t'=1}^{t} P(\pi_{t'} \mid X) \tag{4.6}$$

Recurrence:

$$\alpha(t,s) = P(z_s \mid x_t) \times \begin{cases} \alpha(t-1,s) + \alpha(t-1,s-1) & \text{if } z_s = \langle b \rangle \text{ or } z_s = z_{s-2} \\ \alpha(t-1,s) + \alpha(t-1,s-1) + \alpha(t-1,s-2) & \text{otherwise} \end{cases} \tag{4.7}$$

```python
import torch
import torch.nn as nn
from torch import Tensor


class CTCModel(nn.Module):
    """Minimal CTC-based ASR model.

    Architecture: Mel → Linear Encoder → CTC Output
    """

    def __init__(
        self,
        n_mels: int = 80,
        hidden_dim: int = 512,
        vocab_size: int = 29,  # 26 letters + space + apostrophe + blank
        n_layers: int = 4,
    ) -> None:
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(n_mels, hidden_dim),  # [B, T, n_mels] -> [B, T, hidden]
            nn.ReLU(),
            *[
                nn.Sequential(
                    nn.Linear(hidden_dim, hidden_dim),
                    nn.ReLU(),
                    nn.Dropout(0.1),
                )
                for _ in range(n_layers - 1)
            ],
        )
        self.output_proj = nn.Linear(
            hidden_dim, vocab_size
        )  # [B, T, hidden] -> [B, T, vocab]
        self.ctc_loss = nn.CTCLoss(blank=0, reduction="mean", zero_infinity=True)
```

```
    def forward(
        self,
        mel: Tensor,          # [batch, n_mels, T_frames] - float32
        targets: Tensor,      # [batch, U] - int64 (target text indices)
        input_lengths: Tensor, # [batch] - int64
        target_lengths: Tensor, # [batch] - int64
    ) -> tuple[Tensor, Tensor]:
        """Forward pass with CTC loss.

        Args:
            mel: Mel spectrogram [B, n_mels, T] - float32
            targets: Target indices [B, U] - int64
            input_lengths: Actual input lengths [B] - int64
            target_lengths: Actual target lengths [B] - int64

        Returns:
            loss: CTC loss scalar - float32
            log_probs: Log probabilities [T, B, vocab] - float32
        """
        # [B, n_mels, T] -> [B, T, n_mels]
        x: Tensor = mel.transpose(1, 2)  # [B, T, n_mels] - float32

        # Encode
        h: Tensor = self.encoder(x)  # [B, T, hidden] - float32

        # Project to vocab
        logits: Tensor = self.output_proj(h)  # [B, T, vocab] - float32
        log_probs: Tensor = logits.log_softmax(dim=-1)  # [B, T, vocab] - float32

        # CTC loss expects [T, B, vocab]
        log_probs_ctc: Tensor = log_probs.transpose(0, 1)  # [T, B, vocab] - float3

        loss: Tensor = self.ctc_loss(
            log_probs_ctc,    # [T, B, vocab]
            targets,          # [B, U]
            input_lengths,    # [B]
            target_lengths,   # [B]
        )  # scalar - float32

        return loss, log_probs

    @torch.no_grad()
    def greedy_decode(self, mel: Tensor) -> list[list[int]]:
```

```
        """Greedy CTC decoding.

        Args:
            mel: [batch, n_mels, T] - float32

        Returns:
            decoded: List of decoded token sequences (blanks removed)
        """
        x: Tensor = mel.transpose(1, 2)  # [B, T, n_mels]
        h: Tensor = self.encoder(x)       # [B, T, hidden]
        logits: Tensor = self.output_proj(h)  # [B, T, vocab]

        # Argmax per frame
        best_path: Tensor = logits.argmax(dim=-1)  # [B, T] - int64

        # Collapse: remove blanks and repeated
        decoded: list[list[int]] = []
        for b in range(best_path.size(0)):
            tokens: list[int] = []
            prev: int = -1
            for t in range(best_path.size(1)):
                tok: int = best_path[b, t].item()
                if tok != 0 and tok != prev:  # 0 = blank
                    tokens.append(tok)
                prev = tok
            decoded.append(tokens)

        return decoded
```

4.2.5. 3.2.5 CTC Decoding

Greedy decoding:

$$\hat{\pi}_t = \arg\max_c P(c \mid x_t), \quad \hat{Y} = \mathcal{B}(\hat{\pi}) \tag{4.8}$$

Beam search decoding kết hợp language model:

$$\hat{Y} = \arg\max_Y \left[\log P_{\text{CTC}}(Y \mid X) + \lambda \log P_{\text{LM}}(Y)\right] \tag{4.9}$$

4.2.6. 3.2.6 Hạn chế của CTC

1. **Conditional independence**: $P(\pi_t \mid X)$ được tính independent tại mỗi frame
2. **Monotonic alignment only**: Không thể xử lý reordering (nhưng speech alignment luôn monotonic)
3. **Output tokens input frames**: $U \leq T$ (mỗi non-blank phải có ít nhất 1 frame)
4. **No implicit LM**: CTC không học language model — cần external LM rescoring

4.3. 3.3 Encoder-Decoder with Attention

4.3.1. 3.3.1 Architecture

Kiến trúc seq2seq kinh điển cho ASR, tương tự machine translation:

$$\begin{aligned} \mathbf{h} &= \text{Encoder}(X) && // \ [\text{batch}, T, d_{\text{model}}] \\ P(y_u \mid y_{<u}, X) &= \text{Decoder}(\mathbf{h}, y_{<u}) && // \ \text{autoregressive} \end{aligned} \tag{4.10}$$

4.3.2. 3.3.2 Attention Mechanism cho ASR

Cross-attention giữa decoder query và encoder keys:

$$\text{Attention}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{enc}}^{\top}}{\sqrt{d_k}}\right) V_{\text{enc}} \tag{4.11}$$

> **Location-Sensitive Attention**
>
> Standard attention có thể "nhảy" — không bảo đảm monotonic alignment. **Location-sensitive attention** (LAS, Listen Attend and Spell) thêm convolution trên attention weights trước đó để khuyến khích monotonic progression.

4.3.3. 3.3.3 So sánh CTC vs Encoder-Decoder

Table 4.1.: CTC vs Encoder-Decoder comparison

Tính chất	CTC	Encoder-Decoder
Alignment	Learned (marginalized)	Learned (attention)
Decoding	Greedy / beam + LM	Autoregressive beam search
Dependencies	Conditional independence	Full output dependency

Tính chất	CTC	Encoder-Decoder
Implicit LM	No	Yes (decoder = LM)
Streaming	Easy (frame-by-frame)	Hard (needs full input)
Speed	Fast (parallel)	Slow (sequential decoding)

4.4. 3.4 RNN-Transducer (RNN-T)

4.4.1. 3.4.1 Motivation

RNN-T [13] kết hợp ưu điểm của cả CTC (streaming) và Encoder-Decoder (output dependency):

- **CTC**: Frame-level, không có output dependency → streaming nhưng kém chính xác
- **Encoder-Decoder**: Output dependency nhưng không streaming
- **RNN-T**: Cả hai — output dependency **và** streaming

4.4.2. 3.4.2 Components

RNN-T gồm 3 thành phần:

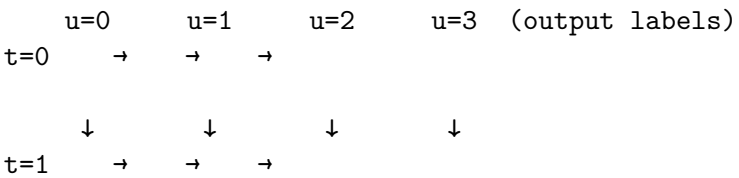
1. **Encoder** (transcription network): $\mathbf{h}_t = \text{Encoder}(x_{1:t})$
2. **Prediction network** (label history): $\mathbf{g}_u = \text{PredNet}(y_{1:u-1})$
3. **Joint network**: kết hợp encoder và prediction outputs

$$\mathbf{z}_{t,u} = \text{Joint}(\mathbf{h}_t, \mathbf{g}_u) = \text{Linear}\left(\tanh(\mathbf{W}_h\mathbf{h}_t + \mathbf{W}_g\mathbf{g}_u + \mathbf{b})\right) \tag{4.12}$$

$$P(k \mid t, u) = \text{softmax}(\mathbf{z}_{t,u})_k, \quad k \in \mathcal{V} \cup \{\langle b \rangle\} \tag{4.13}$$

4.4.3. 3.4.3 Lattice & Forward-Backward

RNN-T hoạt động trên **lattice** 2D ($T \times U$):



```
     ↓       ↓       ↓       ↓
t=2    →    →    →

     ↓       ↓       ↓       ↓
t=3    →    →    →

→ = emit label (advance u)
↓ = emit blank (advance t)
```

- Di chuyển **xuống** ($\downarrow$): emit blank, advance frame $t \to t+1$
- Di chuyển **phải** ($\rightarrow$): emit label y_u, advance label $u \to u+1$

Forward variable:

$$\alpha(t,u) = \alpha(t-1,u) \cdot P(\langle b \rangle \mid t-1,u) + \alpha(t,u-1) \cdot P(y_u \mid t,u-1) \tag{4.14}$$

Loss:

$$\mathcal{L}_{\text{RNN-T}} = -\log \alpha(T,U) \tag{4.15}$$

> **Latency Warning**
>
> RNN-T forward-backward trên lattice ($T \times U$) yêu cầu memory $O(T \times U \times V)$ cho full lattice. Với $T = 1000, U = 100, V = 5000$: cần ~2 GB per sample. Production systems sử dụng **pruned RNN-T** để giảm memory.

```python
import torch
import torch.nn as nn
from torch import Tensor


class RNNTransducer(nn.Module):
    """Simplified RNN-Transducer for ASR.

    Components: Encoder + Prediction Network + Joint Network
    """

    def __init__(
        self,
        n_mels: int = 80,
        encoder_dim: int = 512,
```

```
        predictor_dim: int = 256,
        joint_dim: int = 512,
        vocab_size: int = 29,  # includes blank at index 0
        encoder_layers: int = 6,
        predictor_layers: int = 2,
    ) -> None:
        super().__init__()

        # Encoder (transcription network)
        self.encoder_proj = nn.Linear(n_mels, encoder_dim)
        self.encoder = nn.LSTM(
            input_size=encoder_dim,
            hidden_size=encoder_dim,
            num_layers=encoder_layers,
            batch_first=True,
            dropout=0.1,
        )

        # Prediction network (label history)
        self.embedding = nn.Embedding(
            vocab_size, predictor_dim
        )  # [vocab] -> [predictor_dim]
        self.predictor = nn.LSTM(
            input_size=predictor_dim,
            hidden_size=predictor_dim,
            num_layers=predictor_layers,
            batch_first=True,
            dropout=0.1,
        )

        # Joint network
        self.joint_enc = nn.Linear(
            encoder_dim, joint_dim, bias=False
        )
        self.joint_pred = nn.Linear(
            predictor_dim, joint_dim, bias=True
        )
        self.joint_out = nn.Linear(
            joint_dim, vocab_size
        )

    def encode(self, mel: Tensor) -> Tensor:
        """Encode mel spectrogram.
```

```
        Args:
            mel: [batch, n_mels, T] - float32

        Returns:
            h: [batch, T, encoder_dim] - float32
        """
        x: Tensor = mel.transpose(1, 2)  # [B, T, n_mels] - float32
        x = self.encoder_proj(x)  # [B, T, encoder_dim] - float32
        h, _ = self.encoder(x)  # [B, T, encoder_dim] - float32
        return h

    def predict(self, targets: Tensor) -> Tensor:
        """Run prediction network on target labels.

        Args:
            targets: [batch, U] - int64

        Returns:
            g: [batch, U, predictor_dim] - float32
        """
        emb: Tensor = self.embedding(targets)  # [B, U, predictor_dim] - float32
        g, _ = self.predictor(emb)  # [B, U, predictor_dim] - float32
        return g

    def joint(self, h: Tensor, g: Tensor) -> Tensor:
        """Joint network: combine encoder and predictor.

        Args:
            h: [batch, T, 1, encoder_dim] - float32
            g: [batch, 1, U+1, predictor_dim] - float32

        Returns:
            logits: [batch, T, U+1, vocab_size] - float32
        """
        # Project and broadcast-add
        h_proj: Tensor = self.joint_enc(h)   # [B, T, 1, joint_dim] - float32
        g_proj: Tensor = self.joint_pred(g)  # [B, 1, U+1, joint_dim] - float32

        z: Tensor = torch.tanh(h_proj + g_proj)  # [B, T, U+1, joint_dim] - float32
        logits: Tensor = self.joint_out(z)  # [B, T, U+1, vocab] - float32
        return logits

    def forward(
```

```
        self,
        mel: Tensor,      # [B, n_mels, T] - float32
        targets: Tensor,  # [B, U] - int64
    ) -> Tensor:
        """Compute joint logits for RNN-T loss.

        Args:
            mel: [B, n_mels, T] - float32
            targets: [B, U] - int64

        Returns:
            logits: [B, T, U+1, vocab] - float32
        """
        h: Tensor = self.encode(mel)  # [B, T, enc_dim] - float32
        h = h.unsqueeze(2)  # [B, T, 1, enc_dim] - float32

        # Prepend blank (SOS) to targets
        B, U = targets.shape
        blank: Tensor = torch.zeros(
            B, 1, dtype=torch.long, device=targets.device
        )  # [B, 1]
        targets_sos: Tensor = torch.cat(
            [blank, targets], dim=1
        )  # [B, U+1] - int64

        g: Tensor = self.predict(targets_sos)  # [B, U+1, pred_dim] - float32
        g = g.unsqueeze(1)  # [B, 1, U+1, pred_dim] - float32

        logits: Tensor = self.joint(h, g)  # [B, T, U+1, vocab] - float32
        return logits
```

4.5. 3.5 So sánh Ba Paradigm

Table 4.2.: So sánh ba paradigm ASR

	CTC	Encoder-Decoder	RNN-T
Alignment	Monotonic (marginalized)	Soft (attention)	Monotonic (lattice)

	CTC	Encoder-Decoder	RNN-T
Output dependency	None (conditional indep.)	Full (autoregressive decoder)	Partial (prediction network)
Streaming	(frame-by-frame)	(needs full input)	(frame-by-frame)
Training	Forward-backward $O(TU)$	Cross-entropy $O(U)$ per step	Forward-backward $O(TU)$
Inference	Greedy $O(T)$ / Beam $O(TB)$	Beam $O(UB)$	Beam $O((T+U)B)$
External LM	Required for good results	Optional (implicit LM)	Helpful
Typical use	Offline ASR, simpler models	Whisper	Production streaming
Accuracy	Good + LM	Best (offline)	Best (streaming)

Industry Trend (2024–2026)

- **Google**: Conformer + RNN-T cho on-device streaming ASR
- **OpenAI**: Whisper = Encoder-Decoder (offline)
- **Meta**: Wav2Vec 2.0 + CTC fine-tuning
- **Hybrid**: CTC/Attention joint training (kết hợp CTC loss + attention loss)

4.6. 3.6 Metrics

4.6.1. 3.6.1 Word Error Rate (WER)

$$\text{WER} = \frac{S + D + I}{N} \times 100\% \tag{4.16}$$

trong đó S = substitutions, D = deletions, I = insertions, N = total words in reference.

4.6.2. 3.6.2 Character Error Rate (CER)

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \times 100\% \tag{4.17}$$

Tương tự WER nhưng ở character level — hữu ích cho ngôn ngữ không có word boundaries rõ ràng (tiếng Trung, tiếng Nhật).

4.7. 3.7 Tóm tắt

Table 4.3.: Tóm tắt ba paradigm ASR

Phương pháp	Equation	Ưu điểm chính
CTC	$-\log \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod P(\pi_t \mid X)$	Simple, parallel, streaming
Enc-Dec	$-\sum_u \log P(y_u \mid y_{<u}, X)$	Output dependency, high accuracy
RNN-T	$-\log \alpha(T, U)$ trên lattice	Streaming + output dependency

Chương tiếp theo sẽ khám phá **Self-Supervised Speech** — cách Wav2Vec 2.0 và HuBERT học representations mạnh mẽ từ unlabeled audio.

5. Self-Supervised Speech Representations

5.1. 4.1 Motivation: BERT for Speech

Trong NLP, BERT đã chứng minh rằng **pre-training trên unlabeled data + fine-tuning trên labeled data** cho kết quả vượt trội. Speech có lợi thế tương tự:

- **Unlabeled audio**: Rất nhiều (YouTube, podcasts, audiobooks) — hàng triệu giờ
- **Labeled transcriptions**: Đắt đỏ (cần người nghe và ghi lại) — chỉ vài nghìn giờ

$$\text{Unlabeled audio} \gg \text{Labeled transcriptions} \tag{5.1}$$

NLP Parallel

NLP	Speech
BERT (masked LM)	HuBERT (masked prediction)
Contrastive learning (SimCLR)	Wav2Vec 2.0 (contrastive + masking)
[MASK] token	Masked time steps
WordPiece vocabulary	Quantized speech units
Fine-tune on GLUE	Fine-tune on ASR (CTC)

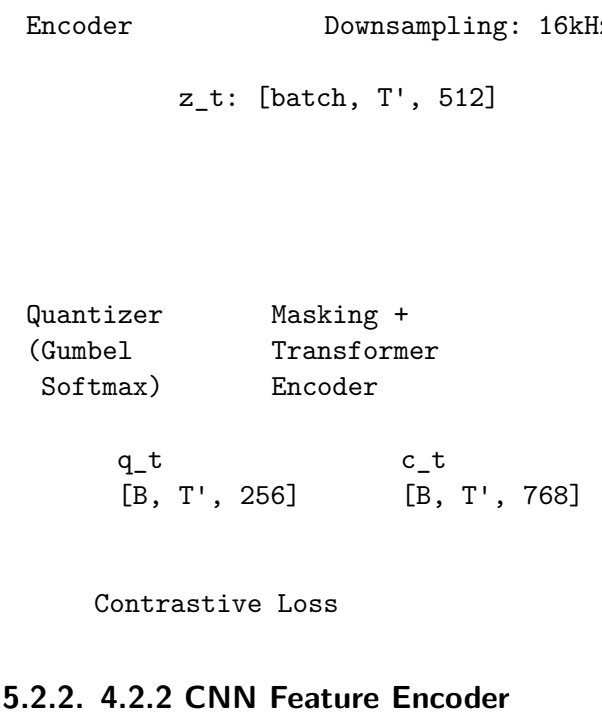
5.2. 4.2 Wav2Vec 2.0

5.2.1. 4.2.1 Architecture Overview

Wav2Vec 2.0 [2] gồm 3 components:

```
Raw Waveform [1, T_samples]


  CNN Feature          7 conv layers, stride 5×2×2×2×2×2×2 = 320
```



5.2.2. 4.2.2 CNN Feature Encoder

7 convolutional layers với temporal convolution:

$$\mathbf{z} = f_{\text{CNN}}(\mathbf{x}), \quad \mathbf{x} \in \mathbb{R}^{T_{\text{samples}}}, \quad \mathbf{z} \in \mathbb{R}^{T' \times d_z} \tag{5.2}$$

với total stride = 320, nên $T' = T_{\text{samples}}/320$. Ở 16 kHz: **50 frames/sec**.

5.2.3. 4.2.3 Quantization Module

Biến continuous features thành discrete units qua **Gumbel-Softmax**:

1. Có G codebook groups, mỗi group có V entries
2. Mỗi frame $\mathbf{z}_t$ được project thành logits cho mỗi group
3. Gumbel-Softmax chọn entry từ mỗi group (differentiable)
4. Concatenate entries từ tất cả groups → quantized vector $\mathbf{q}_t$

$$\mathbf{q}_t = \text{Concat}\left(\mathbf{e}_{g_1}^{(1)}, \mathbf{e}_{g_2}^{(2)}, \ldots, \mathbf{e}_{g_G}^{(G)}\right) \tag{5.3}$$

Gumbel-Softmax:

$$p_{g,v} = \frac{\exp\left((\ell_{g,v} + n_v)/\tau\right)}{\sum_{v'=1}^{V} \exp\left((\ell_{g,v'} + n_{v'})/\tau\right)} \tag{5.4}$$

trong đó $n_v = -\log(-\log(u_v))$, $u_v \sim \text{Uniform}(0, 1)$, và τ là temperature.

- $G = 2$ groups, $V = 320$ entries mỗi group
- Effective vocabulary: $320^2 = 102{,}400$ possible speech units

5.2.4. 4.2.4 Masking Strategy

Tương tự [MASK] trong BERT, nhưng cho contiguous spans:

1. Chọn ngẫu nhiên ~6.5% frames làm start positions
2. Mỗi mask span dài 10 frames (200ms ở 50 fps)
3. Tổng cộng mask ~49% frames
4. Replace masked frames bằng learned [MASK] embedding

5.2.5. 4.2.5 Contrastive Loss

Model phải chọn đúng quantized target $\mathbf{q}_t$ từ K distractors:

$$\mathcal{L}_{\text{contrastive}} = -\log \frac{\exp(\text{sim}(\mathbf{c}_t, \mathbf{q}_t)/\kappa)}{\sum_{\tilde{q} \in Q_t} \exp(\text{sim}(\mathbf{c}_t, \tilde{\mathbf{q}})/\kappa)} \tag{5.5}$$

trong đó:

- $\mathbf{c}_t$: contextualized representation từ Transformer (tại masked position)
- $\mathbf{q}_t$: true quantized target
- Q_t: set gồm $\mathbf{q}_t$ và K distractors (uniformly sampled từ cùng utterance)
- $\text{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b} / \|\mathbf{a}\| \|\mathbf{b}\|$: cosine similarity
- $\kappa = 0.1$: temperature

5.2.6. 4.2.6 Diversity Loss

Khuyến khích sử dụng đều tất cả codebook entries:

$$\mathcal{L}_{\text{diversity}} = \frac{1}{GV} \sum_{g=1}^{G} H(\bar{p}_g) = -\frac{1}{GV} \sum_{g=1}^{G} \sum_{v=1}^{V} \bar{p}_{g,v} \log \bar{p}_{g,v} \tag{5.6}$$

trong đó $\bar{p}_{g,v}$ là average probability của entry v trong group g qua batch.

Total loss:

$$\mathcal{L} = \mathcal{L}_{\text{contrastive}} + \alpha \cdot \mathcal{L}_{\text{diversity}} + \beta \cdot \|\mathbf{z} - \text{sg}(\mathbf{q})\|_2^2 \tag{5.7}$$

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch import Tensor


class GumbelVectorQuantizer(nn.Module):
    """Gumbel-Softmax vector quantizer for Wav2Vec 2.0."""

    def __init__(
        self,
        input_dim: int = 512,
        n_groups: int = 2,
        n_entries: int = 320,
        entry_dim: int = 128,
        temperature: float = 2.0,
    ) -> None:
        super().__init__()
        self.n_groups: int = n_groups
        self.n_entries: int = n_entries
        self.temperature: float = temperature

        # Project input to logits for each group
        self.proj = nn.Linear(
            input_dim, n_groups * n_entries, bias=False
        )  # [B, T, input_dim] -> [B, T, G*V]

        # Codebook entries
        self.entries = nn.Parameter(
            torch.randn(n_groups, n_entries, entry_dim)  # [G, V, entry_dim]
        )

    def forward(self, z: Tensor) -> tuple[Tensor, Tensor]:
        """Quantize input features.

        Args:
            z: [batch, T, input_dim] - float32

        Returns:
            q: Quantized vectors [batch, T, G * entry_dim] - float32
            diversity_loss: Diversity loss scalar - float32
        """
        B, T, _ = z.shape
```

```
        # Compute logits
        logits: Tensor = self.proj(z)  # [B, T, G*V] - float32
        logits = logits.view(B, T, self.n_groups, self.n_entries)
        # [B, T, G, V] - float32

        if self.training:
            # Gumbel-Softmax (differentiable)
            probs: Tensor = F.gumbel_softmax(
                logits, tau=self.temperature, hard=True, dim=-1
            )  # [B, T, G, V] - float32 (one-hot)
        else:
            # Hard argmax at inference
            indices: Tensor = logits.argmax(dim=-1)  # [B, T, G] - int64
            probs = F.one_hot(
                indices, self.n_entries
            ).float()  # [B, T, G, V] - float32

        # Lookup codebook entries: [B, T, G, V] @ [G, V, D] -> [B, T, G, D]
        selected: list[Tensor] = []
        for g in range(self.n_groups):
            entry: Tensor = torch.matmul(
                probs[:, :, g, :],  # [B, T, V]
                self.entries[g],     # [V, D]
            )  # [B, T, D] - float32
            selected.append(entry)

        q: Tensor = torch.cat(selected, dim=-1)  # [B, T, G*D] - float32

        # Diversity loss: maximize entropy of avg prob distribution
        avg_probs: Tensor = probs.mean(dim=(0, 1))  # [G, V] - float32
        diversity_loss: Tensor = -torch.sum(
            avg_probs * torch.log(avg_probs + 1e-10)
        ) / self.n_groups  # scalar

        return q, diversity_loss


class Wav2Vec2ConvEncoder(nn.Module):
    """CNN feature encoder for Wav2Vec 2.0.

    7 conv layers with total stride 320 (16kHz → 50 fps).
    """
```

```
    def __init__(self, output_dim: int = 512) -> None:
        super().__init__()
        # Strides: 5, 2, 2, 2, 2, 2, 2 → total = 320
        channels: list[int] = [512, 512, 512, 512, 512, 512, 512]
        kernels: list[int] = [10, 3, 3, 3, 3, 2, 2]
        strides: list[int] = [5, 2, 2, 2, 2, 2, 2]

        layers: list[nn.Module] = []
        in_ch: int = 1
        for i, (ch, k, s) in enumerate(zip(channels, kernels, strides)):
            layers.append(
                nn.Conv1d(in_ch, ch, kernel_size=k, stride=s, bias=False)
            )
            if i == 0:
                layers.append(nn.GroupNorm(1, ch))  # Layer norm for first layer
            layers.append(nn.GELU())
            in_ch = ch

        self.conv = nn.Sequential(*layers)
        self.proj = nn.Linear(channels[-1], output_dim)

    def forward(self, waveform: Tensor) -> Tensor:
        """Encode raw waveform to latent features.

        Args:
            waveform: [batch, T_samples] - float32

        Returns:
            z: [batch, T', output_dim] - float32
               where T' = T_samples // 320
        """
        x: Tensor = waveform.unsqueeze(1)  # [B, 1, T_samples] - float32
        x = self.conv(x)  # [B, 512, T'] - float32
        x = x.transpose(1, 2)  # [B, T', 512] - float32
        z: Tensor = self.proj(x)  # [B, T', output_dim] - float32
        return z
```

5.3. 4.3 HuBERT

5.3.1. 4.3.1 Key Insight

HuBERT [8] thay đổi cách nhìn: thay vì contrastive learning (phân biệt true vs distractors), HuBERT dùng **offline clustering + masked prediction**:

$$\text{Wav2Vec 2.0: contrastive (which target?)} \quad \longrightarrow \quad \text{HuBERT: predictive (which cluster?)} \tag{5.8}$$

5.3.2. 4.3.2 Training Procedure

Iterative process:

1. **Iteration 1**: Cluster MFCC features bằng k-means → pseudo-labels
2. Train HuBERT với masked prediction trên pseudo-labels
3. **Iteration 2**: Cluster HuBERT features (từ iteration 1) → better pseudo-labels
4. Train HuBERT lại → better representations

$$\text{MFCC} \xrightarrow{\text{k-means}} \text{Pseudo-labels}_1 \xrightarrow{\text{train}} \text{HuBERT}_1 \xrightarrow{\text{k-means}} \text{Pseudo-labels}_2 \xrightarrow{\text{train}} \text{HuBERT}_2 \tag{5.9}$$

5.3.3. 4.3.3 Masked Prediction Loss

Tại mỗi masked position t, predict cluster assignment $c_t \in \{1, \dots, K\}$:

$$\mathcal{L}_{\text{HuBERT}} = -\sum_{t \in \mathcal{M}} \log P(c_t \mid \tilde{X}) \tag{5.10}$$

trong đó:

- $\mathcal{M}$: set of masked positions
- c_t: cluster assignment (pseudo-label) cho frame t
- $\tilde{X}$: masked input (masked frames replaced bằng [MASK])
- $P(c_t \mid \tilde{X}) = \text{softmax}(\mathbf{W} \cdot \mathbf{h}_t)_c$

5.3.4. 4.3.4 So sánh Wav2Vec 2.0 vs HuBERT

Table 5.2.: Wav2Vec 2.0 vs HuBERT comparison

	Wav2Vec 2.0	HuBERT
Pre-training objective	Contrastive (InfoNCE)	Masked prediction (cross-entropy)
Target	Quantized CNN features (online)	K-means clusters (offline)
Quantizer	Gumbel-Softmax (learned)	K-means (fixed per iteration)
Negative samples	Required (from same utterance)	Not needed
Iterations	Single training	Iterative (2+ rounds)
Architecture	CNN encoder + Transformer	CNN encoder + Transformer
Performance	Excellent	Slightly better (especially on low-resource)

i Tại sao HuBERT Tốt hơn?

HuBERT có lợi thế: **consistency of targets**. Wav2Vec 2.0 targets (quantized features) thay đổi trong quá trình training (vì CNN encoder cũng đang được train). HuBERT targets (k-means clusters) **cố định** trong mỗi iteration → training ổn định hơn.

5.4. 4.4 Conformer

5.4.1. 4.4.1 Motivation

Transformer self-attention giỏi **global** dependencies nhưng kém **local** patterns (phoneme-level features). Convolution ngược lại. Conformer [10] kết hợp cả hai:

$$\text{Conformer} = \text{Self-Attention (global)} + \text{Depthwise Conv (local)} \tag{5.11}$$

5.4.2. 4.4.2 Conformer Block

Mỗi Conformer block gồm 4 modules theo thứ tự **macaron-style**:

$$\begin{aligned}\tilde{\mathbf{x}} &= \mathbf{x} + \frac{1}{2}\text{FFN}_1(\mathbf{x}) \\ \mathbf{x}' &= \tilde{\mathbf{x}} + \text{MHSA}(\tilde{\mathbf{x}}) \\ \mathbf{x}'' &= \mathbf{x}' + \text{Conv}(\mathbf{x}') \\ \mathbf{y} &= \text{LayerNorm}\left(\mathbf{x}'' + \frac{1}{2}\text{FFN}_2(\mathbf{x}'')\right)\end{aligned} \tag{5.12}$$

Macaron-Style FFN

Hai half-step FFN ($\frac{1}{2}$) bao quanh attention + conv. Thực nghiệm cho thấy configuration này tốt hơn single full FFN — liên quan đến Stochastic Depth và residual learning.

5.4.3. 4.4.3 Convolution Module

```
Input [B, T, d]

Pointwise Conv (expand: d → 2d)  ← GLU activation halves channels

Depthwise Conv1d (kernel=31)    ← Local pattern extraction

BatchNorm + Swish

Pointwise Conv (project: d → d)

Dropout

Output [B, T, d]
```

$$\text{Conv}(\mathbf{x}) = \text{PointwiseConv}(\text{Swish}(\text{BN}(\text{DepthwiseConv}(\text{GLU}(\text{PointwiseConv}(\mathbf{x}))))))) \tag{5.13}$$

```
import torch
import torch.nn as nn
from torch import Tensor


class ConformerConvModule(nn.Module):
    """Conformer convolution module."""

    def __init__(
        self,
        d_model: int = 256,
        kernel_size: int = 31,
        dropout: float = 0.1,
    ) -> None:
        super().__init__()
        # Pointwise conv (expand + GLU)
        self.pointwise1 = nn.Conv1d(
            d_model, 2 * d_model, kernel_size=1
        )
        self.glu = nn.GLU(dim=1)

        # Depthwise conv
        padding: int = (kernel_size - 1) // 2
        self.depthwise = nn.Conv1d(
            d_model, d_model, kernel_size=kernel_size,
            padding=padding, groups=d_model,
        )
        self.bn = nn.BatchNorm1d(d_model)
        self.swish = nn.SiLU()

        # Pointwise conv (project back)
        self.pointwise2 = nn.Conv1d(d_model, d_model, kernel_size=1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: Tensor) -> Tensor:
        """Forward pass.

        Args:
            x: [batch, T, d_model] - float32

        Returns:
            out: [batch, T, d_model] - float32
        """
```

```
        # [B, T, D] -> [B, D, T] for conv1d
        h: Tensor = x.transpose(1, 2)  # [B, D, T] - float32

        h = self.pointwise1(h)   # [B, 2D, T] - float32
        h = self.glu(h)          # [B, D, T] - float32
        h = self.depthwise(h)    # [B, D, T] - float32
        h = self.bn(h)           # [B, D, T] - float32
        h = self.swish(h)        # [B, D, T] - float32
        h = self.pointwise2(h)   # [B, D, T] - float32
        h = self.dropout(h)      # [B, D, T] - float32

        return h.transpose(1, 2)  # [B, T, D] - float32


class ConformerBlock(nn.Module):
    """Single Conformer block (macaron-style).

    Order: FFN(½) → MHSA → Conv → FFN(½) → LayerNorm
    """

    def __init__(
        self,
        d_model: int = 256,
        n_heads: int = 4,
        ff_dim: int = 1024,
        conv_kernel: int = 31,
        dropout: float = 0.1,
    ) -> None:
        super().__init__()
        # First half-step FFN
        self.ffn1 = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, ff_dim),
            nn.SiLU(),
            nn.Dropout(dropout),
            nn.Linear(ff_dim, d_model),
            nn.Dropout(dropout),
        )

        # Multi-head self-attention
        self.attn_norm = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(
            d_model, n_heads, dropout=dropout, batch_first=True,
```

```
        )
        self.attn_dropout = nn.Dropout(dropout)

        # Convolution module
        self.conv = ConformerConvModule(
            d_model=d_model, kernel_size=conv_kernel, dropout=dropout,
        )

        # Second half-step FFN
        self.ffn2 = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, ff_dim),
            nn.SiLU(),
            nn.Dropout(dropout),
            nn.Linear(ff_dim, d_model),
            nn.Dropout(dropout),
        )

        self.final_norm = nn.LayerNorm(d_model)

    def forward(
        self,
        x: Tensor,                          # [B, T, d_model] - float32
        mask: Tensor | None = None,         # [B, T] - bool
    ) -> Tensor:
        """Forward pass through Conformer block.

        Args:
            x: Input [batch, T, d_model] - float32
            mask: Optional padding mask [batch, T] - bool

        Returns:
            y: Output [batch, T, d_model] - float32
        """
        # Half-step FFN 1
        x = x + 0.5 * self.ffn1(x)  # [B, T, D] - float32

        # MHSA
        x_norm: Tensor = self.attn_norm(x)  # [B, T, D] - float32
        attn_out: Tensor
        attn_out, _ = self.attn(
            x_norm, x_norm, x_norm,
            key_padding_mask=mask,
```

```
        )  # [B, T, D] - float32
        x = x + self.attn_dropout(attn_out)  # [B, T, D] - float32

        # Convolution
        x = x + self.conv(x)  # [B, T, D] - float32

        # Half-step FFN 2
        x = x + 0.5 * self.ffn2(x)  # [B, T, D] - float32

        y: Tensor = self.final_norm(x)  # [B, T, D] - float32
        return y
```

5.4.4. 4.4.4 Conformer Performance

Table 5.3.: Conformer results on LibriSpeech

Model	Params	LibriSpeech test-clean WER	test-other WER
Transformer (baseline)	~118M	2.4%	5.6%
Conformer-S	10.3M	2.7%	6.3%
Conformer-M	30.7M	2.3%	5.0%
Conformer-L	118.8M	**2.1%**	**4.3%**

5.5. 4.5 Fine-tuning for ASR

Cả Wav2Vec 2.0 và HuBERT đều được fine-tune cho ASR bằng **CTC loss**:

$$\text{Pre-trained model} + \text{Linear head} \xrightarrow{\text{CTC fine-tuning}} \text{ASR model} \tag{5.14}$$

```
import torch
import torch.nn as nn
from torch import Tensor


class SSLForASR(nn.Module):
    """Fine-tune self-supervised speech model for ASR with CTC.

    Architecture: Frozen/Unfrozen SSL Encoder → Linear → CTC
    """
```

```
    def __init__(
        self,
        encoder: nn.Module,        # Pre-trained SSL encoder
        hidden_dim: int = 768,     # SSL encoder output dim
        vocab_size: int = 32,      # Character vocab + blank
        freeze_encoder: bool = False,
    ) -> None:
        super().__init__()
        self.encoder: nn.Module = encoder
        self.ctc_head = nn.Linear(hidden_dim, vocab_size)  # [B, T, D] -> [B, T, V]
        self.ctc_loss = nn.CTCLoss(blank=0, reduction="mean", zero_infinity=True)

        if freeze_encoder:
            for param in self.encoder.parameters():
                param.requires_grad = False

    def forward(
        self,
        waveform: Tensor,           # [batch, T_samples] - float32
        targets: Tensor,            # [batch, U] - int64
        input_lengths: Tensor,      # [batch] - int64
        target_lengths: Tensor,     # [batch] - int64
    ) -> Tensor:
        """Forward pass with CTC loss.

        Args:
            waveform: Raw audio [B, T_samples] - float32
            targets: Target indices [B, U] - int64
            input_lengths: Actual encoder output lengths [B] - int64
            target_lengths: Actual target lengths [B] - int64

        Returns:
            loss: CTC loss scalar - float32
        """
        # Encode: [B, T_samples] -> [B, T', hidden_dim]
        h: Tensor = self.encoder(waveform)  # [B, T', 768] - float32

        # Project to vocab
        logits: Tensor = self.ctc_head(h)  # [B, T', vocab] - float32
        log_probs: Tensor = logits.log_softmax(dim=-1)  # [B, T', vocab] - float32

        # CTC expects [T', B, vocab]
        log_probs_t: Tensor = log_probs.transpose(0, 1)  # [T', B, vocab] - float32
```

```
        loss: Tensor = self.ctc_loss(
            log_probs_t, targets, input_lengths, target_lengths,
        )  # scalar - float32
        return loss
```

5.5.1. 4.5.1 Kết quả Fine-tuning

Table 5.4.: Self-supervised ASR results on LibriSpeech

Model	Pre-train data	Fine-tune data	WER (test-clean)	WER (test-other)
Wav2Vec 2.0 Base	LS-960h	LS-960h	3.4%	8.0%
Wav2Vec 2.0 Large	LV-60K h	LS-960h	**1.8%**	**3.3%**
Wav2Vec 2.0 Large	LV-60K h	10 min (!)	4.8%	8.2%
HuBERT Base	LS-960h	LS-960h	3.4%	8.1%
HuBERT Large	LV-60K h	LS-960h	**1.9%**	**3.3%**
HuBERT X-Large	LV-60K h	LS-960h	**1.5%**	**2.6%**

⚠ Latency Warning

Wav2Vec 2.0 Large (317M params) cần ~2 GB VRAM chỉ cho inference. Với batch processing 10 giây audio: latency ~200ms trên A100, ~800ms trên T4. Production thường dùng **distilled** hoặc **Base** model (95M params).

5.6. 4.6 Tóm tắt

Table 5.5.: Tóm tắt self-supervised speech models

Model	Pre-training Objective	Key Innovation
Wav2Vec 2.0	Contrastive + Diversity	Online quantization (Gumbel-Softmax)
HuBERT	Masked prediction	Offline clustering (iterative k-means)
Conformer	N/A (architecture)	Conv + Attention (macaron-style)

Chương tiếp theo sẽ khám phá **Whisper** — model mà OpenAI đã train trên 680K giờ audio để tạo ra ASR model mạnh nhất cho nhiều ngôn ngữ.

6. Whisper & Modern ASR

6.1. 5.1 Whisper: Weak Supervision at Scale

Whisper [1] là ASR model của OpenAI, được train trên **680,000 giờ** audio với weak supervision — không cần human-annotated transcriptions.

6.1.1. 5.1.1 Key Insight

Thay vì tự tạo labels (self-supervised) hay thuê người gán nhãn (supervised), Whisper thu thập **audio + transcript pairs từ internet** — noisy nhưng massive scale:

$$\underbrace{680\text{K hours}}_{\text{Whisper (weak supervision)}} \gg \underbrace{60\text{K hours}}_{\text{Wav2Vec 2.0 (self-supervised)}} \gg \underbrace{960\text{ hours}}_{\text{LibriSpeech (supervised)}} \tag{6.1}$$

> **NLP Parallel**
>
> Whisper's approach tương tự **GPT pre-training**: thu thập text từ internet (noisy, uncurated) nhưng massive scale → model tự học filtering noise. Trong speech: thu thập audio + subtitles từ YouTube/podcasts.

6.2. 5.2 Architecture

6.2.1. 5.2.1 Overview

Whisper là **encoder-decoder Transformer** kinh điển:

```
Audio Waveform (16kHz, 30s)


Log Mel Spectrogram (80 mels, 100 fps)
[1, 80, 3000]  ← 30s × 100 fps
```

```
Audio Encoder
(Transformer)
- 2× Conv1d (stride 2)   ← Downsampling: 3000 → 1500 frames
- N transformer blocks
- Sinusoidal pos enc

         [1, 1500, d_model]


Text Decoder
(Transformer)
- Learned pos enc
- Causal self-attn
- Cross-attention
- Token prediction



Output Tokens
<|startoftranscript|> <|en|> <|transcribe|> ...text... <|endoftext|>
```

6.2.2. 5.2.2 Audio Encoder

$$\begin{aligned}
\mathbf{x}_{\text{mel}} &\in \mathbb{R}^{80\times 3000} && \text{// Log mel spectrogram} \\
\mathbf{x}_1 &= \text{GELU}(\text{Conv1d}(\mathbf{x}_{\text{mel}}, k{=}3, s{=}1)) && \text{// [d, 3000]} \\
\mathbf{x}_2 &= \text{GELU}(\text{Conv1d}(\mathbf{x}_1, k{=}3, s{=}2)) && \text{// [d, 1500] — 2× downsampling} \\
\mathbf{h} &= \text{TransformerEncoder}(\mathbf{x}_2 + \mathbf{p}_{\text{sin}}) && \text{// [1500, d]}
\end{aligned} \tag{6.2}$$

6.2.3. 5.2.3 Text Decoder

Autoregressive decoder với **special tokens** cho multitask:

$$P(y_u \mid y_{<u}, \mathbf{h}) = \text{softmax}(\mathbf{W}_o \cdot \text{TransformerDecoder}(y_{<u}, \mathbf{h})) \tag{6.3}$$

6.2.4. 5.2.4 Model Sizes

Table 6.1.: Whisper model sizes

Model	Layers	Width	Heads	Params	English WER
Tiny	4+4	384	6	39M	7.6%
Base	6+6	512	8	74M	5.0%
Small	12+12	768	12	244M	3.4%
Medium	24+24	1024	16	769M	2.9%
Large-v3	32+32	1280	20	1550M	**2.2%**

6.3. 5.3 Multitask Training Format

6.3.1. 5.3.1 Special Token Protocol

Whisper sử dụng sequence of special tokens để chỉ định task:

```
<|startoftranscript|> <|lang|> <|task|> [<|notimestamps|>] ...text... <|endoftext|>
```

- `<|lang|>`: Language token (e.g., `<|en|>`, `<|vi|>`, `<|zh|>`)
- `<|task|>`: `<|transcribe|>` hoặc `<|translate|>` (→ English)
- `<|notimestamps|>`: Optional, nếu không cần word-level timestamps

6.3.2. 5.3.2 Tasks

Table 6.2.: Whisper multitask format

Task	Input	Output	Token
Transcription	Audio (any lang)	Text (same lang)	`<|transcribe|>`
Translation	Audio (any lang)	Text (English)	`<|translate|>`
Language ID	Audio	Language token	Predict `<|lang|>`
VAD	Audio	Timestamps	`<|0.00|>` … `<|30.00|>`

Multitask = Prompt Engineering for ASR

Whisper's special token system tương đương **prompt engineering** cho LLM. Thay vì train riêng models cho mỗi task, dùng prompt tokens để điều khiển behavior — giống GPT-3 instruction following.

6.4. 5.4 Training Details

6.4.1. 5.4.1 Data Distribution

- **680K hours** total audio
- **96 languages** (nhưng phân bố rất không đều)
- English: ~438K hours (~65%)
- Top 10 languages: ~90% tổng data
- Vietnamese: ước tính ~2,000–5,000 hours

6.4.2. 5.4.2 Training Setup

$$\begin{aligned} \text{Optimizer} &: \text{AdamW}, \quad \beta = (0.9, 0.98), \quad \epsilon = 10^{-6} \\ \text{LR schedule} &: \text{Linear warmup (2048 steps)} \rightarrow \text{cosine decay} \\ \text{Batch size} &: 256 \text{ segments} \times 30\text{s} = 2.13 \text{ hours/batch} \\ \text{Total steps} &: \sim 2^{20} \approx 1{,}048{,}576 \\ \text{Total compute} &: \sim 5{,}500 \text{ GPU-days (A100)} \end{aligned} \tag{6.4}$$

6.4.3. 5.4.3 Loss Function

Standard cross-entropy trên text tokens:

$$\mathcal{L} = -\frac{1}{U} \sum_{u=1}^{U} \log P(y_u \mid y_{<u}, \mathbf{h}_{\text{encoder}}) \tag{6.5}$$

6.5. 5.5 Inference & Decoding

```
import torch
import torch.nn as nn
import math
from torch import Tensor


class WhisperEncoder(nn.Module):
    """Simplified Whisper audio encoder.

    Conv1d downsampling + Transformer encoder.
    """
```

```
    def __init__(
        self,
        n_mels: int = 80,
        d_model: int = 512,
        n_heads: int = 8,
        n_layers: int = 6,
        max_len: int = 1500,
    ) -> None:
        super().__init__()
        # Two conv layers for downsampling
        self.conv1 = nn.Conv1d(
            n_mels, d_model, kernel_size=3, padding=1,
        )  # [B, 80, T] -> [B, d, T]
        self.conv2 = nn.Conv1d(
            d_model, d_model, kernel_size=3, stride=2, padding=1,
        )  # [B, d, T] -> [B, d, T//2]
        self.gelu = nn.GELU()

        # Sinusoidal positional encoding
        pe: Tensor = torch.zeros(max_len, d_model)  # [max_len, d]
        position: Tensor = torch.arange(0, max_len).unsqueeze(1).float()
        div_term: Tensor = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
        )
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)  # [max_len, d]

        # Transformer encoder layers
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=d_model * 4,
            dropout=0.1,
            activation="gelu",
            batch_first=True,
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=n_layers,
        )
        self.ln = nn.LayerNorm(d_model)

    def forward(self, mel: Tensor) -> Tensor:
```

```
        """Encode mel spectrogram.

        Args:
            mel: [batch, n_mels, T_frames] - float32
                 T_frames = 3000 for 30s audio at 100fps

        Returns:
            h: [batch, T_frames//2, d_model] - float32
               T_frames//2 = 1500 for 30s audio
        """
        x: Tensor = self.gelu(self.conv1(mel))  # [B, d, T] - float32
        x = self.gelu(self.conv2(x))  # [B, d, T//2] - float32
        x = x.transpose(1, 2)  # [B, T//2, d] - float32

        T: int = x.size(1)
        x = x + self.pe[:T]  # [B, T//2, d] - float32

        h: Tensor = self.transformer(x)  # [B, T//2, d] - float32
        h = self.ln(h)  # [B, T//2, d] - float32
        return h


class WhisperDecoder(nn.Module):
    """Simplified Whisper text decoder."""

    def __init__(
        self,
        vocab_size: int = 51865,
        d_model: int = 512,
        n_heads: int = 8,
        n_layers: int = 6,
        max_len: int = 448,
    ) -> None:
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_len, d_model)

        decoder_layer = nn.TransformerDecoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=d_model * 4,
            dropout=0.1,
            activation="gelu",
```

```
            batch_first=True,
        )
        self.transformer = nn.TransformerDecoder(
            decoder_layer, num_layers=n_layers,
        )
        self.ln = nn.LayerNorm(d_model)
        self.proj = nn.Linear(d_model, vocab_size, bias=False)

    def forward(
        self,
        tokens: Tensor,    # [batch, U] - int64
        encoder_out: Tensor,  # [batch, T', d_model] - float32
    ) -> Tensor:
        """Decode tokens with encoder context.

        Args:
            tokens: Input token IDs [B, U] - int64
            encoder_out: Encoder output [B, T', d] - float32

        Returns:
            logits: [B, U, vocab_size] - float32
        """
        U: int = tokens.size(1)
        positions: Tensor = torch.arange(
            U, device=tokens.device
        )  # [U]

        x: Tensor = self.token_emb(tokens) + self.pos_emb(positions)
        # [B, U, d] - float32

        # Causal mask
        causal_mask: Tensor = nn.Transformer.generate_square_subsequent_mask(
            U, device=tokens.device,
        )  # [U, U] - float32

        h: Tensor = self.transformer(
            x, encoder_out, tgt_mask=causal_mask,
        )  # [B, U, d] - float32

        h = self.ln(h)  # [B, U, d] - float32
        logits: Tensor = self.proj(h)  # [B, U, vocab] - float32
        return logits
```

6.6. 5.6 Faster Whisper & Optimizations

6.6.1. 5.6.1 Distil-Whisper

Distil-Whisper [14] giảm size bằng knowledge distillation:

$$\mathcal{L}_{\text{distil}} = \alpha \cdot \mathcal{L}_{\text{CE}}(y, \hat{y}_{\text{student}}) + (1-\alpha) \cdot \text{KL}(\hat{p}_{\text{teacher}} \| \hat{p}_{\text{student}}) \tag{6.6}$$

Table 6.3.: Distil-Whisper performance

Model	Params	Speed (vs Large-v3)	WER (test-clean)
Whisper Large-v3	1550M	1×	2.2%
Distil-Whisper Large-v3	756M	6.3×	2.5%

6.6.2. 5.6.2 faster-whisper (CTranslate2)

Tối ưu inference bằng:

- **INT8 quantization**: Giảm memory 2–4×
- **KV-cache optimization**: Reuse computed attention keys/values
- **Batched beam search**: Parallel decoding
- **VAD-based chunking**: Chỉ process segments có speech

⚠ Latency Warning

Configuration	RTF (Real-Time Factor)	Latency (30s audio)	VRAM
Whisper Large-v3 (FP16, A100)	0.03	~1s	6 GB
Whisper Large-v3 (INT8, T4)	0.15	~4.5s	3 GB
faster-whisper Large-v3 (INT8, T4)	0.05	~1.5s	2 GB
Distil-Whisper (INT8, T4)	0.02	~0.6s	1.5 GB

6.6.3. 5.6.3 Whisper cho Tiếng Việt

Table 6.5.: Whisper Vietnamese performance

Model	Vietnamese WER (FLEURS)	Ghi chú
Whisper Tiny	~35%	Quá nhỏ cho tonal language

Model	Vietnamese WER (FLEURS)	Ghi chú
Whisper Small	~18%	Acceptable cho demo
Whisper Medium	~12%	Good cho production
Whisper Large-v3	~8%	Best, nhưng heavy

Vietnamese Challenges

Tiếng Việt là tonal language (6 tones) — Whisper thường gặp lỗi với:

- **Hỏi vs Ngã** tone confusion (Southern dialect merger)
- **Regional accents**: Bắc/Trung/Nam rất khác biệt
- **Code-switching**: Vietnamese + English trong business contexts
- **Proper nouns**: Tên riêng Việt Nam thường bị sai

Fine-tuning trên Vietnamese data cải thiện đáng kể — xem Chương 10.

6.7. 5.7 Tóm tắt

Table 6.6.: Whisper summary

Aspect	Whisper
Architecture	Encoder-decoder Transformer
Pre-training	680K hours weak supervision
Input	30s mel spectrogram (80, 3000)
Tasks	Transcription, translation, VAD, language ID
Multilingual	96 languages
Best WER	2.2% (Large-v3, LibriSpeech test-clean)
Vietnamese	~8% WER (FLEURS)
Optimization	Distil-Whisper, faster-whisper, INT8

Chương tiếp theo chuyển sang **Text-to-Speech (TTS)** — chiều ngược lại: từ text sang audio.

Part III.

Phần III: Tổng hợp Giọng nói (TTS) & Audio Codecs

7. TTS Foundations & Vocoders

7.1. 6.1 Bài toán Text-to-Speech

Text-to-Speech (TTS) là bài toán ngược của ASR — chuyển text thành waveform:

$$\hat{\mathbf{x}} = \arg\max_{\mathbf{x}} P(\mathbf{x} \mid Y) \tag{7.1}$$

trong đó $Y = (y_1, \dots, y_U)$ là chuỗi text tokens và $\mathbf{x}$ là waveform.

7.1.1. 6.1.1 Two-Stage Pipeline

Hầu hết TTS systems chia thành 2 giai đoạn:

```
Text → [Acoustic Model] → Mel Spectrogram → [Vocoder] → Waveform

      Stage 1: Text-to-Mel                  Stage 2: Mel-to-Wave
      (Tacotron 2, FastSpeech 2)            (HiFi-GAN, WaveNet)
```

Tại sao 2 stages?

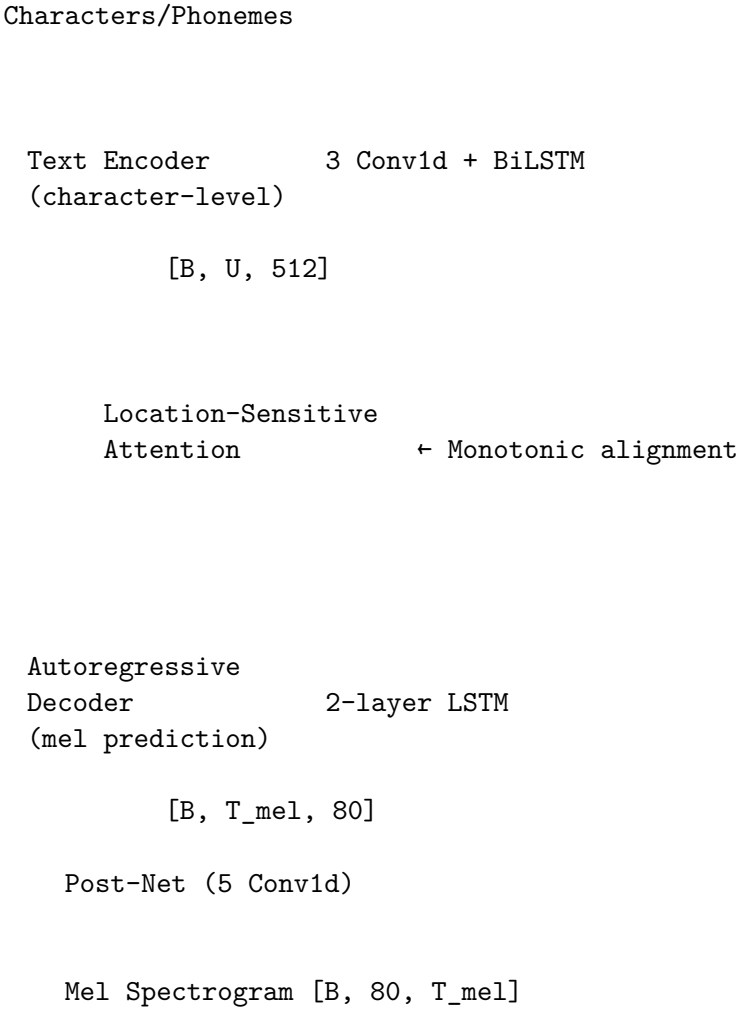
- Mel spectrogram là intermediate representation **compact** (80 dims × 100 fps)
- Waveform là **high-dimensional** (16,000 samples/sec)
- Tách 2 stages cho phép **optimize từng phần** independently

> NLP Parallel
>
> Two-stage TTS giống **retrieval-augmented generation (RAG)**: Stage 1 tạo "plan" (mel), Stage 2 "execute" (vocoder). End-to-end TTS (VITS) giống standard autoregressive LM — xem Chương 7.

7.2. 6.2 Tacotron 2

7.2.1. 6.2.1 Architecture

Tacotron 2 [15] là attention-based seq2seq model cho Text-to-Mel:



7.2.2. 6.2.2 Location-Sensitive Attention

Attention cho TTS phải **monotonic** — không được nhảy lùi. Location-sensitive attention thêm convolution trên previous attention weights:

$$
\begin{aligned}
f_i &= F * \alpha_{i-1} && \text{// Conv1d on previous alignment} \\
e_{i,j} &= \mathbf{v}^\top \tanh(\mathbf{W}_s \mathbf{s}_i + \mathbf{W}_h \mathbf{h}_j + \mathbf{W}_f f_{i,j} + \mathbf{b}) \\
\alpha_{i,j} &= \text{softmax}(e_{i,:})_j && \text{// Attention weights}
\end{aligned}
\tag{7.2}
$$

trong đó α_{i-1} là attention weights từ decoder step trước.

7.2.3. 6.2.3 Decoder

Autoregressive prediction — mỗi step predict 1 mel frame (hoặc r frames với reduction factor):

$$
\begin{aligned}
\mathbf{c}_i &= \sum_j \alpha_{i,j} \mathbf{h}_j && \text{// Context vector} \\
(\mathbf{s}_i, \mathbf{o}_i) &= \text{LSTM}([\text{PreNet}(\hat{\mathbf{m}}_{i-1}); \mathbf{c}_i], \mathbf{s}_{i-1}) \\
\hat{\mathbf{m}}_i &= \text{Linear}([\mathbf{o}_i; \mathbf{c}_i]) && \text{// [80] mel prediction}
\end{aligned}
\tag{7.3}
$$

7.2.4. 6.2.4 Stop Token

Binary classifier dự đoán khi nào dừng generation:

$$
p_{\text{stop}}(i) = \sigma(\mathbf{w}_{\text{stop}}^\top [\mathbf{o}_i; \mathbf{c}_i] + b_{\text{stop}}) \tag{7.4}
$$

7.2.5. 6.2.5 Hạn chế của Tacotron 2

Table 7.1.: Tacotron 2 limitations

Vấn đề	Nguyên nhân
Robustness	Attention alignment có thể fail (repeats, skips)
Speed	Autoregressive → sequential (không parallel)
Controllability	Không control được duration, pitch trực tiếp
Quality	Tốt nhưng cần vocoder riêng

7.3. 6.3 FastSpeech 2

7.3.1. 6.3.1 Key Innovation: Non-Autoregressive

FastSpeech 2 [16] giải quyết tất cả hạn chế của Tacotron 2 bằng **parallel generation + explicit duration prediction**:

```
Phonemes


  Encoder          Transformer encoder
  (phoneme-level)

         [B, U, 256]

         → Duration Predictor → predicted duration d_u
         → Pitch Predictor → predicted pitch f0
         → Energy Predictor → predicted energy


  Length           Repeat each phoneme d_u times
  Regulator       U phonemes → T mel frames

         [B, T_mel, 256]


  Mel Decoder      Transformer decoder (parallel!)



  Mel Spectrogram [B, 80, T_mel]
```

7.3.2. 6.3.2 Variance Adaptors

Duration predictor:

$$\hat{d}_u = \text{ReLU}(\text{DurationPredictor}(\mathbf{h}_u)), \quad d_u \in \mathbb{Z}^+ \tag{7.5}$$

Trained với L2 loss trên log-duration (ground truth từ forced alignment):

$$\mathcal{L}_{\text{dur}} = \frac{1}{U}\sum_{u=1}^{U}\Big(\log(\hat{d}_u + 1) - \log(d_u + 1)\Big)^2 \tag{7.6}$$

Pitch predictor (F0):

$$\hat{f}_0(t) = \text{PitchPredictor}(\mathbf{h}_{u(t)}) \tag{7.7}$$

Energy predictor:

$$\hat{e}(t) = \text{EnergyPredictor}(\mathbf{h}_{u(t)}) \tag{7.8}$$

7.3.3. 6.3.3 Length Regulator

Biến phoneme-level sequence thành mel-level sequence bằng cách repeat:

$$\text{LR}(\mathbf{H}, \mathbf{d}) = [\underbrace{\mathbf{h}_1, \dots, \mathbf{h}_1}_{d_1 \text{ times}}, \underbrace{\mathbf{h}_2, \dots, \mathbf{h}_2}_{d_2 \text{ times}}, \dots] \tag{7.9}$$

Output length: $T_{\text{mel}} = \sum_{u=1}^{U} d_u$

7.3.4. 6.3.4 Total Loss

$$\mathcal{L} = \mathcal{L}_{\text{mel}} + \lambda_d \mathcal{L}_{\text{dur}} + \lambda_p \mathcal{L}_{\text{pitch}} + \lambda_e \mathcal{L}_{\text{energy}} \tag{7.10}$$

7.3.5. 6.3.5 Speed Comparison

Table 7.2.: FastSpeech 2 vs Tacotron 2 speed

Model	Inference Speed (vs real-time)	Parallelizable	Robustness
Tacotron 2	~0.5× (slower than RT)	No	Attention failures
FastSpeech 2	**50×** faster	**Yes**	**No attention failures**

> **Latency Warning**
>
> FastSpeech 2 mel generation rất nhanh (~50× real-time) nhưng **vocoder vẫn là bottleneck**. Pipeline latency = mel generation + vocoder. Cần HiFi-GAN (realtime) thay vì WaveNet (100× slower than RT).

```python
import torch
import torch.nn as nn
from torch import Tensor


class VariancePredictor(nn.Module):
    """Variance predictor for duration/pitch/energy.

    2 Conv1d + Linear → scalar per frame.
    """

    def __init__(
        self,
        d_model: int = 256,
        kernel_size: int = 3,
        dropout: float = 0.1,
    ) -> None:
        super().__init__()
        padding: int = (kernel_size - 1) // 2
        self.conv1 = nn.Conv1d(
            d_model, d_model, kernel_size=kernel_size, padding=padding,
        )
        self.conv2 = nn.Conv1d(
            d_model, d_model, kernel_size=kernel_size, padding=padding,
        )
        self.ln1 = nn.LayerNorm(d_model)
        self.ln2 = nn.LayerNorm(d_model)
        self.proj = nn.Linear(d_model, 1)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: Tensor) -> Tensor:
        """Predict variance (duration/pitch/energy).

        Args:
            x: [batch, seq_len, d_model] - float32

        Returns:
            pred: [batch, seq_len] - float32
        """
        h: Tensor = x.transpose(1, 2)  # [B, D, L] - float32
        h = self.dropout(self.relu(self.ln1(self.conv1(h).transpose(1, 2))))
        # [B, L, D] - float32
```

```
        h = h.transpose(1, 2)  # [B, D, L]
        h = self.dropout(self.relu(self.ln2(self.conv2(h).transpose(1, 2))))
        # [B, L, D] - float32
        pred: Tensor = self.proj(h).squeeze(-1)  # [B, L] - float32
        return pred


class LengthRegulator(nn.Module):
    """Expand phoneme-level features to mel-level using durations."""

    def forward(
        self,
        x: Tensor,          # [batch, U, d_model] - float32
        durations: Tensor,  # [batch, U] - int64 (predicted durations)
    ) -> Tensor:
        """Regulate length by repeating each phoneme d_u times.

        Args:
            x: Phoneme features [B, U, D] - float32
            durations: Duration per phoneme [B, U] - int64

        Returns:
            expanded: Mel-level features [B, T_mel, D] - float32
        """
        expanded_list: list[Tensor] = []
        for b in range(x.size(0)):
            repeated: list[Tensor] = []
            for u in range(x.size(1)):
                d: int = max(1, durations[b, u].item())
                repeated.append(
                    x[b, u].unsqueeze(0).expand(d, -1)
                )  # [d, D] - float32
            expanded_list.append(torch.cat(repeated, dim=0))  # [T_mel_b, D]

        # Pad to max length in batch
        max_len: int = max(t.size(0) for t in expanded_list)
        batch_size: int = x.size(0)
        d_model: int = x.size(2)

        expanded: Tensor = torch.zeros(
            batch_size, max_len, d_model,
            device=x.device, dtype=x.dtype,
        )  # [B, T_mel, D] - float32
```

```
        for b, t in enumerate(expanded_list):
            expanded[b, :t.size(0)] = t

        return expanded
```

7.4. 6.4 Vocoders

7.4.1. 6.4.1 Bài toán Vocoder

Vocoder chuyển mel spectrogram thành waveform — bài toán **super-resolution** vì mel (80 dims, 100 fps) chứa ít thông tin hơn waveform (1 dim, 16000 fps):

$$\hat{\mathbf{x}} = \text{Vocoder}(\mathbf{S}_{\text{mel}}), \quad \mathbf{S}_{\text{mel}} \in \mathbb{R}^{80 \times T}, \quad \hat{\mathbf{x}} \in \mathbb{R}^{160 \cdot T} \tag{7.11}$$

7.4.2. 6.4.2 WaveNet (2016)

WaveNet [17] — vocoder đầu tiên cho chất lượng human-level:

$$P(\mathbf{x}) = \prod_{n=1}^{N} P(x_n \mid x_1, \ldots, x_{n-1}) \tag{7.12}$$

Autoregressive: Generate 1 sample at a time → 16,000 steps/sec → **cực kỳ chậm**.

> ⚠ Latency Warning
>
> WaveNet inference: ~0.01× real-time trên GPU (100× chậm hơn real-time). 1 giây audio cần ~100 giây compute. **Không khả thi cho production.**

7.4.3. 6.4.3 HiFi-GAN (2020)

HiFi-GAN [18] — vocoder hiện đại, **nhanh** và **chất lượng cao**:

Generator: Upsample mel → waveform qua transposed convolutions:

$$\text{Mel} \xrightarrow{\text{Upsample}_{8\times}} \xrightarrow{\text{Upsample}_{8\times}} \xrightarrow{\text{Upsample}_{2\times}} \xrightarrow{\text{Upsample}_{2\times}} \text{Waveform} \tag{7.13}$$

Total upsampling factor: $8 \times 8 \times 2 \times 2 = 256$ (matches hop_length=256 ở 22.05kHz, hoặc 160 ở 16kHz).

Multi-Period Discriminator (MPD) + Multi-Scale Discriminator (MSD):

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}}(G) + \lambda_{\text{fm}}\mathcal{L}_{\text{fm}}(G) + \lambda_{\text{mel}}\mathcal{L}_{\text{mel}}(G) \tag{7.14}$$

trong đó:

- $\mathcal{L}_{\text{adv}}$: Adversarial loss (fool discriminators)
- $\mathcal{L}_{\text{fm}}$: Feature matching loss (intermediate discriminator features)
- $\mathcal{L}_{\text{mel}}$: Mel reconstruction loss (stability)

```python
import torch
import torch.nn as nn
from torch import Tensor


class ResBlock(nn.Module):
    """Residual block with dilated convolutions."""

    def __init__(
        self,
        channels: int,
        kernel_size: int = 3,
        dilations: tuple[int, ...] = (1, 3, 5),
    ) -> None:
        super().__init__()
        self.convs = nn.ModuleList()
        for d in dilations:
            self.convs.append(
                nn.Sequential(
                    nn.LeakyReLU(0.1),
                    nn.Conv1d(
                        channels, channels,
                        kernel_size=kernel_size,
                        dilation=d,
                        padding=(kernel_size * d - d) // 2,
                    ),
                )
            )

    def forward(self, x: Tensor) -> Tensor:
        """Forward with residual connections.

        Args:
```

```
            x: [batch, channels, T] - float32

        Returns:
            out: [batch, channels, T] - float32
        """
        for conv in self.convs:
            x = x + conv(x)  # [B, C, T] - float32
        return x


class HiFiGANGenerator(nn.Module):
    """HiFi-GAN vocoder generator.

    Mel spectrogram → Waveform via transposed convolutions.
    """

    def __init__(
        self,
        n_mels: int = 80,
        upsample_initial: int = 512,
        upsample_rates: tuple[int, ...] = (8, 8, 2, 2, 2),
        upsample_kernels: tuple[int, ...] = (16, 16, 4, 4, 4),
        resblock_kernels: tuple[int, ...] = (3, 7, 11),
        resblock_dilations: tuple[tuple[int, ...], ...] = (
            (1, 3, 5), (1, 3, 5), (1, 3, 5),
        ),
    ) -> None:
        super().__init__()
        # Input projection
        self.conv_pre = nn.Conv1d(
            n_mels, upsample_initial, kernel_size=7, padding=3,
        )  # [B, 80, T] -> [B, 512, T]

        # Upsampling blocks
        self.ups = nn.ModuleList()
        self.resblocks = nn.ModuleList()

        ch: int = upsample_initial
        for i, (rate, kernel) in enumerate(
            zip(upsample_rates, upsample_kernels)
        ):
            ch_out: int = ch // 2
            self.ups.append(
```

```
                nn.ConvTranspose1d(
                    ch, ch_out,
                    kernel_size=kernel,
                    stride=rate,
                    padding=(kernel - rate) // 2,
                )
            )
            # Multi-receptive-field fusion (MRF)
            for k in resblock_kernels:
                self.resblocks.append(
                    ResBlock(ch_out, kernel_size=k, dilations=resblock_dilations[0])
                )
            ch = ch_out

        self.conv_post = nn.Conv1d(
            ch, 1, kernel_size=7, padding=3,
        )  # [B, ch, T_wav] -> [B, 1, T_wav]
        self.tanh = nn.Tanh()

        self.n_resblocks_per_up: int = len(resblock_kernels)

    def forward(self, mel: Tensor) -> Tensor:
        """Generate waveform from mel spectrogram.

        Args:
            mel: [batch, n_mels, T_frames] - float32

        Returns:
            waveform: [batch, 1, T_samples] - float32
        """
        x: Tensor = self.conv_pre(mel)  # [B, 512, T] - float32

        for i, up in enumerate(self.ups):
            x = nn.functional.leaky_relu(x, 0.1)
            x = up(x)  # [B, ch//2, T*rate] - float32

            # Sum multi-receptive-field outputs
            xs: Tensor = torch.zeros_like(x)
            for j in range(self.n_resblocks_per_up):
                idx: int = i * self.n_resblocks_per_up + j
                xs = xs + self.resblocks[idx](x)
            x = xs / self.n_resblocks_per_up
```

```
        x = nn.functional.leaky_relu(x, 0.1)
        x = self.conv_post(x)  # [B, 1, T_samples] - float32
        waveform: Tensor = self.tanh(x)  # [B, 1, T_samples] - float32

        return waveform
```

7.4.4. 6.4.4 Vocoder Comparison

Table 7.3.: Vocoder comparison

Vocoder	Year	Type	Speed (RTF)	Quality (MOS)
WaveNet	2016	Autoregressive	0.01×	4.21
WaveRNN	2018	Autoregressive	1×	4.15
WaveGlow	2019	Flow-based	20×	4.09
HiFi-GAN	2020	GAN	**80×**	**4.23**
BigVGAN	2022	GAN	60×	**4.30**

7.5. 6.5 Tóm tắt

Table 7.4.: TTS pipeline summary

Component	Model	Key Idea	Speed
Acoustic Model	Tacotron 2	Attention-based autoregressive	Slow
Acoustic Model	FastSpeech 2	Non-autoregressive + variance adaptors	**Fast**
Vocoder	WaveNet	Autoregressive sample-by-sample	Very slow
Vocoder	HiFi-GAN	GAN + multi-scale discriminators	**Very fast**
Best Pipeline	FastSpeech 2 + HiFi-GAN	Parallel mel + fast vocoder	Production-ready

Chương tiếp theo sẽ khám phá **End-to-End TTS** — VITS, VALL-E, và F5-TTS: các model bỏ qua two-stage pipeline.

8. End-to-End TTS & Zero-Shot Voice Cloning

8.1. 7.1 Từ Two-Stage đến End-to-End

Chương trước đã trình bày pipeline **Text → Mel → Waveform** (FastSpeech 2 + HiFi-GAN). Chương này khám phá các model **end-to-end** — trực tiếp từ text sang waveform, và đặc biệt là **zero-shot voice cloning**.

```
Evolution of TTS:
2018: Tacotron 2 + WaveNet      (autoregressive + autoregressive)
2020: FastSpeech 2 + HiFi-GAN   (non-AR + GAN vocoder)
2021: VITS                       (end-to-end VAE + flow + GAN)
2023: VALL-E                     (codec language model)
2024: F5-TTS                     (flow matching + DiT)
```

8.2. 7.2 VITS — Variational Inference with Adversarial Learning

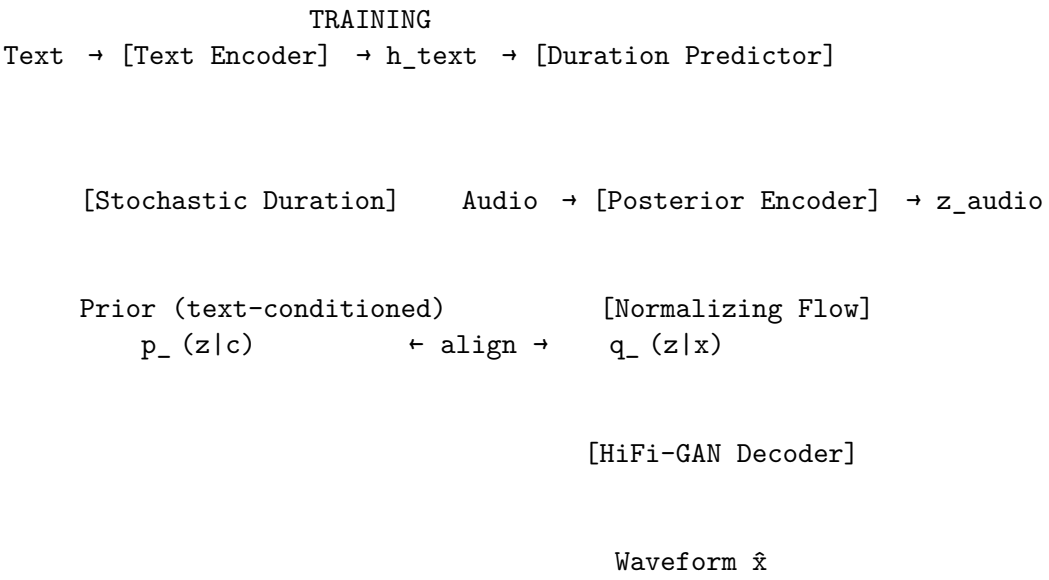
8.2.1. 7.2.1 Key Innovation

VITS [3] là model **end-to-end** đầu tiên đạt chất lượng ngang two-stage systems. Kết hợp 3 frameworks:

1. **VAE** (Variational Autoencoder): Latent representation learning
2. **Normalizing Flows**: Flexible posterior distribution
3. **GAN**: High-fidelity waveform generation

$$\text{VITS} = \text{VAE} + \text{Normalizing Flow} + \text{Adversarial Training} \tag{8.1}$$

8.2.2. 7.2.2 Architecture Overview



8.2.3. 7.2.3 ELBO Objective

$$\log p_\theta(\mathbf{x} \mid c) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x} \mid \mathbf{z}) - \log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid c)}\right] \tag{8.2}$$

trong đó:

- $p_\theta(\mathbf{x} \mid \mathbf{z})$: Decoder (HiFi-GAN) — reconstruct waveform từ latent
- $q_\phi(\mathbf{z} \mid \mathbf{x})$: Posterior encoder — encode audio sang latent
- $p_\theta(\mathbf{z} \mid c)$: Prior — text-conditioned prior distribution

8.2.4. 7.2.4 Normalizing Flow

Flow biến simple distribution thành complex distribution qua invertible transformations:

$$\mathbf{z}_K = f_K \circ f_{K-1} \circ \cdots \circ f_1(\mathbf{z}_0), \quad \mathbf{z}_0 \sim \mathcal{N}(\mu_\theta(c), \sigma_\theta(c)) \tag{8.3}$$

$$\log q_\phi(\mathbf{z}_K \mid \mathbf{x}) = \log q(\mathbf{z}_0) - \sum_{k=1}^{K} \log \left| \det \frac{\partial f_k}{\partial \mathbf{z}_{k-1}} \right| \tag{8.4}$$

VITS sử dụng **affine coupling layers** (similar to WaveGlow/Glow):

$$\begin{aligned} \mathbf{z}_a, \mathbf{z}_b &= \text{split}(\mathbf{z}) \\ \mathbf{z}'_b &= \mathbf{z}_b \odot \exp(s(\mathbf{z}_a)) + t(\mathbf{z}_a) \\ f(\mathbf{z}) &= \text{concat}(\mathbf{z}_a, \mathbf{z}'_b) \end{aligned} \tag{8.5}$$

8.2.5. 7.2.5 Monotonic Alignment Search (MAS)

VITS tìm hard alignment giữa text và latent frames bằng dynamic programming:

$$A^* = \arg\max_{A \in \{0,1\}^{T \times U}} \sum_{t,u} A_{t,u} \log p_\theta(z_t \mid c_u) \tag{8.6}$$

subject to monotonicity constraints. Giải bằng DP $O(TU)$.

8.2.6. 7.2.6 Total Loss

$$\mathcal{L}_{\text{VITS}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}} + \mathcal{L}_{\text{dur}} + \mathcal{L}_{\text{adv}} + \mathcal{L}_{\text{fm}} \tag{8.7}$$

```python
import torch
import torch.nn as nn
from torch import Tensor


class PosteriorEncoder(nn.Module):
    """VITS posterior encoder: Audio → latent z.

    Uses WaveNet-style dilated convolutions.
    """

    def __init__(
        self,
        in_channels: int = 513,      # linear spectrogram bins (n_fft//2+1)
        hidden_channels: int = 192,
        latent_channels: int = 192,
        kernel_size: int = 5,
        n_layers: int = 16,
        dilation_rate: int = 1,
    ) -> None:
        super().__init__()
        self.pre = nn.Conv1d(in_channels, hidden_channels, 1)
        self.enc = nn.ModuleList()
        for i in range(n_layers):
```

```
            dilation: int = dilation_rate ** (i % 4)
            padding: int = (kernel_size - 1) * dilation // 2
            self.enc.append(
                nn.Sequential(
                    nn.Conv1d(
                        hidden_channels, 2 * hidden_channels,
                        kernel_size, dilation=dilation, padding=padding,
                    ),
                    nn.GroupNorm(1, 2 * hidden_channels),
                )
            )
        self.proj = nn.Conv1d(hidden_channels, 2 * latent_channels, 1)

    def forward(
        self, x: Tensor,              # [B, in_channels, T] - float32
        x_mask: Tensor | None = None,  # [B, 1, T] - float32
    ) -> tuple[Tensor, Tensor, Tensor]:
        """Encode audio spectrogram to latent distribution.

        Args:
            x: Linear spectrogram [B, 513, T] - float32
            x_mask: Optional mask [B, 1, T] - float32

        Returns:
            z: Sampled latent [B, latent_ch, T] - float32
            mu: Mean [B, latent_ch, T] - float32
            log_sigma: Log std [B, latent_ch, T] - float32
        """
        h: Tensor = self.pre(x)  # [B, hidden, T] - float32
        if x_mask is not None:
            h = h * x_mask

        for layer in self.enc:
            h_gated: Tensor = layer(h)  # [B, 2*hidden, T] - float32
            h_a, h_b = h_gated.chunk(2, dim=1)  # each [B, hidden, T]
            h = h + torch.tanh(h_a) * torch.sigmoid(h_b)  # [B, hidden, T]
            if x_mask is not None:
                h = h * x_mask

        stats: Tensor = self.proj(h)  # [B, 2*latent, T] - float32
        mu, log_sigma = stats.chunk(2, dim=1)  # each [B, latent, T]

        # Reparameterization trick
```

```
        z: Tensor = mu + torch.randn_like(mu) * torch.exp(log_sigma)
        # [B, latent, T] - float32

        return z, mu, log_sigma
```

8.3. 7.3 VALL-E — Neural Codec Language Model for TTS

8.3.1. 7.3.1 Paradigm Shift

VALL-E [4] biến TTS thành **language modeling problem** trên neural codec tokens:

$$\text{Traditional TTS: Text} \rightarrow \text{Mel} \rightarrow \text{Waveform}$$

$$\text{VALL-E: Text} + \text{Audio Prompt} \rightarrow \text{Codec Tokens} \rightarrow \text{Waveform} \tag{8.8}$$

> NLP Parallel: GPT for Speech
>
> VALL-E là **GPT applied to speech**. Thay vì predict next BPE token, nó predict next audio codec token. 3-second audio prompt = **in-context learning** — giống few-shot prompting cho LLM.

8.3.2. 7.3.2 Architecture

VALL-E sử dụng EnCodec (8 RVQ codebooks) và chia thành 2 stages:

```
Input: Text tokens + 3s audio prompt (encoded as codec tokens)

Stage 1: Autoregressive (AR) Model
    Predict first codebook c_1 token-by-token
    Input:  [text; prompt_c1] → predict c1_t autoregressively
    Output: c^(1) = [c^(1)_1, c^(1)_2, ..., c^(1)_T]

Stage 2: Non-Autoregressive (NAR) Model
    Predict codebooks 2-8 in parallel, conditioned on previous
    Input:  [text; c^(1); c^(2); ...; c^(q-1)] → predict c^(q) all at once
    Output: c^(2), c^(3), ..., c^(8)

Finally: EnCodec decoder converts all 8 codebooks → waveform
```

8.3.3. 7.3.3 AR Model (Codebook 1)

$$P_{\text{AR}}(\mathbf{c}^{(1)} \mid \mathbf{y}, \tilde{\mathbf{c}}^{(1)}) = \prod_{t=1}^{T} P(c_t^{(1)} \mid c_{<t}^{(1)}, \mathbf{y}, \tilde{\mathbf{c}}^{(1)}) \tag{8.9}$$

trong đó:

- $\mathbf{y}$: text (phoneme) tokens
- $\tilde{\mathbf{c}}^{(1)}$: first codebook tokens từ 3s audio prompt
- $c_t^{(1)}$: predicted token at time t for codebook 1

8.3.4. 7.3.4 NAR Model (Codebooks 2-8)

$$P_{\text{NAR}}(\mathbf{c}^{(q)} \mid \mathbf{c}^{(1)}, \ldots, \mathbf{c}^{(q-1)}, \mathbf{y}) = \prod_{t=1}^{T} P(c_t^{(q)} \mid \mathbf{c}^{(<q)}, \mathbf{y}) \tag{8.10}$$

8.3.5. 7.3.5 Zero-Shot Voice Cloning

VALL-E đạt zero-shot voice cloning chỉ với **3 giây** audio prompt:

$$\text{Voice Cloning} = \text{In-Context Learning trên Codec Tokens} \tag{8.11}$$

Table 8.1.: VALL-E comparison

Feature	VALL-E	Tacotron 2	VITS
Voice cloning	**3s prompt** (zero-shot)	Requires fine-tuning	Requires fine-tuning
Training data	60K hours	24 hours (single speaker)	24+ hours
Naturalness (MOS)	3.8	4.1	4.2
Speaker similarity	**0.58** (zero-shot!)	N/A (same speaker)	N/A

8.4. 7.4 F5-TTS — Flow Matching + DiT

8.4.1. 7.4.1 Flow Matching

F5-TTS [19] sử dụng **flow matching** [20] — phương pháp mới hơn diffusion:

$$\frac{d\mathbf{x}_t}{dt} = v_\theta(\mathbf{x}_t, t, c), \quad t \in [0, 1] \tag{8.12}$$

trong đó:

- $\mathbf{x}_0 \sim \mathcal{N}(0, I)$: noise
- $\mathbf{x}_1$: target mel spectrogram
- v_θ: learned velocity field (neural network)
- c: conditioning (text + speaker)

Training objective (Conditional Flow Matching):

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{x}_1}\left[\|v_\theta(\mathbf{x}_t, t, c) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2\right] \tag{8.13}$$

với interpolation:

$$\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1 \tag{8.14}$$

Flow Matching vs Diffusion

	Diffusion	Flow Matching
Path	Stochastic (SDE)	Deterministic (ODE)
Training	Score matching	Velocity matching
Sampling	Many steps (50–1000)	Fewer steps (10–50)
Implementation	Complex noise schedules	**Simple**: linear interpolation
Speed	Slower	**Faster**

8.4.2. 7.4.2 DiT Architecture (Diffusion Transformer)

F5-TTS sử dụng **DiT** (Diffusion Transformer) thay vì U-Net:

```
Input: noisy mel x_t + text embedding + time embedding


  DiT Block (× N)

    AdaLN (time-conditioned)
    Self-Attention
    Cross-Attention (text)
    AdaLN + FFN
```

Predicted velocity v_

Adaptive Layer Normalization (AdaLN):

$$\text{AdaLN}(\mathbf{h}, t) = \gamma(t) \odot \frac{\mathbf{h} - \mu}{\sigma} + \beta(t) \tag{8.15}$$

trong đó $\gamma(t), \beta(t)$ được predict từ time embedding.

8.4.3. 7.4.3 F5-TTS Inference

```
import torch
from torch import Tensor


def flow_matching_sample(
    model: torch.nn.Module,
    text_cond: Tensor,       # [B, U, D] - float32, text conditioning
    speaker_cond: Tensor,    # [B, D_spk] - float32, speaker embedding
    n_frames: int = 200,     # number of mel frames to generate
    n_steps: int = 32,       # ODE solver steps (Euler)
    n_mels: int = 80,
) -> Tensor:
    """Generate mel spectrogram via flow matching ODE.

    Args:
        model: Trained velocity network v_
        text_cond: Text conditioning [B, U, D] - float32
        speaker_cond: Speaker embedding [B, D_spk] - float32
        n_frames: Number of mel frames to generate
        n_steps: Number of Euler steps
        n_mels: Mel channels

    Returns:
        x: Generated mel spectrogram [B, n_mels, n_frames] - float32
    """
    B: int = text_cond.size(0)
    device: torch.device = text_cond.device

    # Start from noise
```

```
    x: Tensor = torch.randn(
        B, n_mels, n_frames, device=device,
    )  # [B, 80, T] - float32

    dt: float = 1.0 / n_steps

    for step in range(n_steps):
        t: float = step / n_steps
        t_tensor: Tensor = torch.full(
            (B,), t, device=device,
        )  # [B] - float32

        # Predict velocity
        v: Tensor = model(
            x, t_tensor, text_cond, speaker_cond,
        )  # [B, 80, T] - float32

        # Euler step
        x = x + v * dt  # [B, 80, T] - float32

    return x  # [B, 80, T] - float32, generated mel
```

8.4.4. 7.4.4 F5-TTS Results

Table 8.3.: End-to-end TTS comparison

Model	MOS ↑	Speaker Sim ↑	RTF ↓	Zero-shot?
VITS	4.2	N/A	0.01	No
VALL-E	3.8	0.58	0.8	Yes (3s)
NaturalSpeech 2	4.2	0.62	0.3	Yes (3s)
F5-TTS	**4.3**	**0.68**	**0.15**	**Yes (3s)**
CosyVoice	4.1	0.65	0.2	Yes (3s)

8.5. 7.5 TTS Evolution Timeline

Table 8.4.: TTS evolution timeline

Year	Model	Type	Key Innovation
2016	WaveNet	AR vocoder	Sample-level generation

Year	Model	Type	Key Innovation
2018	Tacotron 2	Attention seq2seq	Location-sensitive attention
2020	FastSpeech 2	Non-AR + vocoder	Duration/pitch/energy predictors
2020	HiFi-GAN	GAN vocoder	Multi-period/scale discriminators
2021	VITS	End-to-end	VAE + Flow + GAN
2023	VALL-E	Codec LM	Zero-shot cloning via in-context learning
2024	F5-TTS	Flow matching	DiT + CFM, fastest zero-shot

Latency Warning

Zero-shot TTS models (VALL-E, F5-TTS) có latency cao hơn traditional TTS:

- FastSpeech 2 + HiFi-GAN: ~**50ms** cho 5s audio
- VITS: ~**100ms** cho 5s audio
- VALL-E: ~**2s** cho 5s audio (AR decoding bottleneck)
- F5-TTS: ~**500ms** cho 5s audio (32 ODE steps)

Production voice cloning cần cân bằng quality vs latency.

8.6. 7.6 Tóm tắt

Table 8.5.: End-to-end TTS summary

Model	Approach	Pros	Cons
VITS	VAE + Flow + GAN	End-to-end, high quality	Single speaker, no cloning
VALL-E	Codec LM (AR + NAR)	Zero-shot cloning	Slow AR decoding
F5-TTS	Flow matching + DiT	Fast, high quality, zero-shot	Requires good codec

Chương tiếp theo sẽ đi sâu vào **Audio Codecs** — EnCodec, DAC, và Mimi — nền tảng cho VALL-E và các Speech LLMs.

9. Audio Codecs & Neural Tokenization

9.1. 8.1 Tại sao cần Neural Audio Codecs?

Neural audio codecs giải quyết bài toán **nén audio thành discrete tokens** — cầu nối trực tiếp giữa speech và language modeling:

$$\text{Waveform} \xrightarrow{\text{Encoder}} \text{Latent} \xrightarrow{\text{RVQ}} \text{Discrete Tokens} \xrightarrow{\text{Decoder}} \text{Reconstructed Waveform} \tag{9.1}$$

NLP Parallel: Tokenizer cho Audio

Text Processing	Audio Processing
BPE tokenizer	Neural codec (EnCodec)
Vocabulary (32K–128K)	Codebook (1024 per layer $\times$ 8 layers)
Lossless (exact reconstruction)	Near-lossless (perceptual)
Deterministic	Learned (neural network)
Input to GPT/BERT	Input to VALL-E/AudioLM

9.2. 8.2 Vector Quantization (VQ)

9.2.1. 8.2.1 Basic VQ

Vector Quantization [21] ánh xạ continuous vector sang nearest codebook entry:

$$q(\mathbf{z}) = \arg\min_{\mathbf{e}_k \in \mathcal{C}} \|\mathbf{z} - \mathbf{e}_k\|_2 \tag{9.2}$$

trong đó $\mathcal{C} = \{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_K\}$ là codebook với K entries.

Vấn đề: argmin không differentiable $\rightarrow$ cần **Straight-Through Estimator (STE)**:

$$\hat{\mathbf{z}} = \mathbf{z} + \text{sg}(\mathbf{e}_{k^*} - \mathbf{z}) \tag{9.3}$$

trong đó $\text{sg}(\cdot)$ là stop-gradient operator. Forward pass: $\hat{\mathbf{z}} = \mathbf{e}_{k^*}$. Backward pass: gradient flows straight through to $\mathbf{z}$.

9.2.2. 8.2.2 VQ-VAE Loss

$$\mathcal{L}_{\text{VQ-VAE}} = \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2}_{\text{reconstruction}} + \underbrace{\|\text{sg}(\mathbf{z}) - \mathbf{e}\|_2^2}_{\text{codebook loss}} + \beta \underbrace{\|\mathbf{z} - \text{sg}(\mathbf{e})\|_2^2}_{\text{commitment loss}} \tag{9.4}$$

9.2.3. 8.2.3 Hạn chế của Single VQ

Với codebook size $K = 1024$ và dimension $d = 128$:

- Mỗi frame chỉ chọn 1 trong 1024 entries → **10 bits** per frame
- Ở 50 fps: bitrate $= 50 \times 10 = 500$ bps — **quá thấp** cho audio quality

→ Cần **Residual Vector Quantization** để tăng capacity.

9.3. 8.3 Residual Vector Quantization (RVQ)

9.3.1. 8.3.1 Ý tưởng

Thay vì 1 codebook lớn, dùng **nhiều codebooks nhỏ** quantize **residual** (phần dư):

$$\begin{aligned}
&\mathbf{r}_0 = \mathbf{z} && \text{// Original latent} \\
&\hat{\mathbf{z}}_1 = q_1(\mathbf{r}_0), \quad \mathbf{r}_1 = \mathbf{r}_0 - \hat{\mathbf{z}}_1 && \text{// Quantize + compute residual} \\
&\hat{\mathbf{z}}_2 = q_2(\mathbf{r}_1), \quad \mathbf{r}_2 = \mathbf{r}_1 - \hat{\mathbf{z}}_2 && \text{// Quantize residual} \\
&\quad \vdots \\
&\hat{\mathbf{z}}_Q = q_Q(\mathbf{r}_{Q-1}) && \text{// Final quantization} \\
&\hat{\mathbf{z}} = \sum_{q=1}^{Q} \hat{\mathbf{z}}_q && \text{// Reconstructed latent}
\end{aligned} \tag{9.5}$$

9.3.2. 8.3.2 Bitrate Calculation

Với Q codebooks, mỗi codebook size K, ở frame rate f:

$$\text{Bitrate} = Q \times \log_2(K) \times f \text{ bps} \tag{9.6}$$

EnCodec example: $Q = 8, K = 1024, f = 75$:

$$\text{Bitrate} = 8 \times 10 \times 75 = 6{,}000 \text{ bps} = 6 \text{ kbps} \tag{9.7}$$

> **RVQ Hierarchy**
>
> Các codebook layers mang thông tin khác nhau:
>
> - **Layer 1**: Semantic/linguistic content (phonemes, words)
> - **Layers 2–4**: Prosody, speaker identity
> - **Layers 5–8**: Fine acoustic details, timbre
>
> Đây là lý do AudioLM và VALL-E xử lý codebook 1 (AR) và codebooks 2–8 (NAR) khác nhau.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch import Tensor


class VectorQuantize(nn.Module):
    """Single-codebook vector quantization with EMA updates."""

    def __init__(
        self,
        dim: int = 128,
        codebook_size: int = 1024,
        decay: float = 0.99,
    ) -> None:
        super().__init__()
        self.codebook_size: int = codebook_size
        self.decay: float = decay

        self.codebook = nn.Embedding(codebook_size, dim)
        nn.init.uniform_(self.codebook.weight, -1.0 / codebook_size, 1.0 / codebook_size)
```

```
    def forward(self, z: Tensor) -> tuple[Tensor, Tensor, Tensor]:
        """Quantize input vectors.

        Args:
            z: [batch, T, dim] - float32

        Returns:
            z_q: Quantized vectors [batch, T, dim] - float32
            indices: Codebook indices [batch, T] - int64
            commit_loss: Commitment loss scalar - float32
        """
        # Find nearest codebook entry
        # z: [B, T, D], codebook: [K, D]
        dist: Tensor = (
            z.pow(2).sum(-1, keepdim=True)          # [B, T, 1]
            - 2 * z @ self.codebook.weight.T         # [B, T, K]
            + self.codebook.weight.pow(2).sum(-1)     # [K]
        )  # [B, T, K] - float32 (squared distances)

        indices: Tensor = dist.argmin(dim=-1)  # [B, T] - int64
        z_q: Tensor = self.codebook(indices)   # [B, T, D] - float32

        # Commitment loss
        commit_loss: Tensor = F.mse_loss(z, z_q.detach())  # scalar

        # Straight-Through Estimator
        z_q = z + (z_q - z).detach()  # [B, T, D] - float32

        return z_q, indices, commit_loss


class ResidualVQ(nn.Module):
    """Residual Vector Quantization with Q codebooks."""

    def __init__(
        self,
        dim: int = 128,
        codebook_size: int = 1024,
        n_quantizers: int = 8,
    ) -> None:
        super().__init__()
        self.n_quantizers: int = n_quantizers
        self.quantizers = nn.ModuleList([
```

```
            VectorQuantize(dim=dim, codebook_size=codebook_size)
            for _ in range(n_quantizers)
        ])

    def forward(
        self, z: Tensor,  # [batch, T, dim] - float32
    ) -> tuple[Tensor, Tensor, Tensor]:
        """Apply residual vector quantization.

        Args:
            z: Input latent [B, T, dim] - float32

        Returns:
            z_q: Quantized sum [B, T, dim] - float32
            all_indices: Codebook indices [B, Q, T] - int64
            total_loss: Sum of commitment losses - float32
        """
        residual: Tensor = z.clone()  # [B, T, D] - float32
        z_q: Tensor = torch.zeros_like(z)  # [B, T, D] - float32
        all_indices: list[Tensor] = []
        total_loss: Tensor = torch.tensor(0.0, device=z.device)

        for q, quantizer in enumerate(self.quantizers):
            zq_i, indices_i, loss_i = quantizer(residual)
            # zq_i: [B, T, D], indices_i: [B, T], loss_i: scalar

            residual = residual - zq_i.detach()  # [B, T, D] - float32
            z_q = z_q + zq_i  # [B, T, D] - float32
            all_indices.append(indices_i)
            total_loss = total_loss + loss_i

        indices_stacked: Tensor = torch.stack(
            all_indices, dim=1
        )  # [B, Q, T] - int64

        return z_q, indices_stacked, total_loss
```

9.4. 8.4 EnCodec

9.4.1. 8.4.1 Architecture

EnCodec [5] là neural audio codec của Meta:

```
Waveform (24kHz)


  Encoder                 4 conv blocks (stride 2,4,5,8)
  (1D CNN)                Total stride: 320
                          24kHz / 320 = 75 fps

            [B, 128, T/320]


  RVQ                     8 codebooks × 1024 entries
  (8 layers)              Bitrate: 6 kbps

            [B, 8, T/320] (integer tokens)


  Decoder                 4 transposed conv blocks
  (1D CNN)                Mirror of encoder


Reconstructed Waveform (24kHz)
```

9.4.2. 8.4.2 Training Losses

$$\mathcal{L}_{\text{EnCodec}} = \lambda_t \mathcal{L}_{\text{time}} + \lambda_f \mathcal{L}_{\text{freq}} + \lambda_g \mathcal{L}_{\text{gan}} + \lambda_{\text{fm}} \mathcal{L}_{\text{feat}} + \lambda_w \mathcal{L}_{\text{vq}} \tag{9.8}$$

Table 9.2.: EnCodec loss components

Loss	Formula	Purpose
Time domain	$\|\mathbf{x} - \hat{\mathbf{x}}\|_1$	Waveform reconstruction
Frequency domain	$\sum_s \|\text{STFT}_s(\mathbf{x}) - \text{STFT}_s(\hat{\mathbf{x}})\|_1 + \|\log \text{STFT}_s\|_1$	Multi-scale spectral

Loss	Formula	Purpose
GAN	$\sum_k \max(0, 1 - D_k(\mathbf{x})) + \max(0, 1 + D_k(\hat{\mathbf{x}}))$	Perceptual quality
Feature matching	$\sum_k \sum_l \|D_k^l(\mathbf{x}) - D_k^l(\hat{\mathbf{x}})\|_1$	Discriminator features
VQ commitment	$\|\mathbf{z} - \text{sg}(\hat{\mathbf{z}})\|_2^2$	Encoder-codebook alignment

9.4.3. 8.4.3 EnCodec Specifications

Table 9.3.: EnCodec specifications

Parameter	Value
Sample rate	24 kHz
Encoder stride	320 ($\rightarrow$ 75 fps)
Codebook size	1024 per layer
RVQ layers	1–32 (adjustable)
Bitrate	1.5 / 3 / 6 / 12 / 24 kbps
Latent dim	128
Model params	~15M
Latency	13.3ms (1 frame at 75 Hz)

9.5. 8.5 DAC (Descript Audio Codec)

DAC [22] cải tiến EnCodec với:

1. **Improved codebook utilization**: Factorized codes + L2 normalization
2. **Snake activation**: $\text{Snake}(x) = x + \frac{1}{\alpha}\sin^2(\alpha x)$ — tốt hơn cho periodic signals
3. **Higher quality**: Đặc biệt ở bitrate thấp

Table 9.4.: DAC vs EnCodec quality

Codec	ViSQOL (↑) @ 6kbps	Codebook Usage
EnCodec	3.69	~60%
DAC	**4.01**	**~95%**

9.6. 8.6 SpeechTokenizer

9.6.1. 8.6.1 Disentangled Semantic/Acoustic Tokens

SpeechTokenizer tách biệt **semantic** và **acoustic** information:

```
Layer 1:  Semantic tokens   ← Aligned with HuBERT features
Layers 2-8: Acoustic tokens ← Speaker, prosody, timbre details
```

Training: Thêm **semantic distillation loss** cho layer 1:

$$\mathcal{L}_{\text{semantic}} = \|\hat{\mathbf{z}}^{(1)} - \text{HuBERT}(\mathbf{x})\|_2^2 \tag{9.9}$$

Lợi ích: Speech LLMs có thể xử lý semantic tokens (layer 1) riêng → tốt hơn cho language understanding tasks.

9.7. 8.7 Mimi — Ultra-Low Latency Codec

9.7.1. 8.7.1 Key Innovation

Mimi (dùng trong Moshi [6]) đạt **ultra-low frame rate**:

$$\text{EnCodec: 75 Hz} \xrightarrow{\text{Mimi}} \text{12.5 Hz} \tag{9.10}$$

9.7.2. 8.7.2 Cách đạt 12.5 Hz

1. **Larger encoder stride**: 1920 (vs 320 cho EnCodec)
2. **Transformer layers** trong encoder/decoder: Capture long-range dependencies
3. **Semantic distillation**: Layer 1 aligned với WavLM features
4. **Split RVQ**: 1 semantic codebook + 7 acoustic codebooks

9.7.3. 8.7.3 Tại sao 12.5 Hz quan trọng?

$$\text{Tokens per second} = 12.5 \times 8 = 100 \text{ tokens/s (total)} \tag{9.11}$$

So sánh: EnCodec = $75 \times 8 = 600$ tokens/s. Mimi giảm **6×** số tokens → Transformer self-attention nhanh hơn **36×** ($O(L^2)$).

Latency Warning

Codec	Frame Rate	Tokens/sec	10s Audio Tokens	Self-Attn Cost
EnCodec	75 Hz	600	6,000	$O(36M)$
Mimi	12.5 Hz	100	1,000	$O(1M)$
Reduction	6×	6×	6×	**36×**

Mimi's low frame rate là yếu tố quyết định cho full-duplex dialogue trong Moshi.

9.8. 8.8 Codec Comparison

Table 9.6.: Audio codec comparison

Codec	Frame Rate	RVQ Layers	Bitrate	Quality	Semantic?
EnCodec	75 Hz	8	6 kbps	Good	No
DAC	86 Hz	9	8 kbps	**Better**	No
SpeechTokenizer	50 Hz	8	4 kbps	Good	**Layer 1**
Mimi	**12.5 Hz**	8	1.1 kbps	Good	**Layer 1**

```
import torch
import torch.nn as nn
from torch import Tensor


class EnCodecEncoder(nn.Module):
    """Simplified EnCodec encoder.

    Waveform → Latent features via strided convolutions.
    """

    def __init__(
        self,
        in_channels: int = 1,
        latent_dim: int = 128,
        channels: int = 64,
        strides: tuple[int, ...] = (2, 4, 5, 8),  # total = 320
    ) -> None:
        super().__init__()
        self.conv_in = nn.Conv1d(
```

```
            in_channels, channels, kernel_size=7, padding=3,
        )

        blocks: list[nn.Module] = []
        ch: int = channels
        for stride in strides:
            ch_out: int = ch * 2
            blocks.extend([
                nn.ELU(),
                nn.Conv1d(ch, ch, kernel_size=3, padding=1),
                nn.ELU(),
                nn.Conv1d(
                    ch, ch_out,
                    kernel_size=2 * stride,
                    stride=stride,
                    padding=stride // 2,
                ),
            ])
            ch = ch_out

        self.encoder = nn.Sequential(*blocks)
        self.conv_out = nn.Sequential(
            nn.ELU(),
            nn.Conv1d(ch, latent_dim, kernel_size=3, padding=1),
        )

    def forward(self, x: Tensor) -> Tensor:
        """Encode waveform to latent features.

        Args:
            x: Waveform [batch, 1, T_samples] - float32

        Returns:
            z: Latent features [batch, latent_dim, T'] - float32
               where T' = T_samples / prod(strides)
        """
        h: Tensor = self.conv_in(x)      # [B, 64, T] - float32
        h = self.encoder(h)               # [B, 1024, T'] - float32
        z: Tensor = self.conv_out(h)      # [B, 128, T'] - float32
        return z
```

9.9. 8.9 Tóm tắt

Table 9.7.: Audio codec key concepts

Concept	Equation	Role
VQ	$q(\mathbf{z}) = \arg\min_k \|\mathbf{z} - \mathbf{e}_k\|$	Discretize latent
STE	$\hat{\mathbf{z}} = \mathbf{z} + \text{sg}(\mathbf{e} - \mathbf{z})$	Gradient through argmin
RVQ	$\hat{\mathbf{z}} = \sum_q q_q(\mathbf{r}_{q-1})$	Multi-layer quantization
Bitrate	$Q \times \log_2(K) \times f$	Quality control

Neural audio codecs là **nền tảng** cho Speech LLMs. Chương tiếp theo sẽ khám phá cách AudioLM, Qwen2-Audio, và Moshi xây dựng trên codec tokens để tạo ra speech-native language models.

Part IV.

Phần IV: Speech LLMs & Tiếng Việt

10. Speech Language Models

10.1. 9.1 Từ Text LLM đến Speech LLM

Speech Language Models mở rộng paradigm LLM sang audio modality. Có 3 cấp độ tích hợp:

```
Level 1: Cascaded Pipeline
    Audio → [ASR] → Text → [LLM] → Text → [TTS] → Audio
    Ưu: Simple, modular       Nhược: High latency, error propagation

Level 2: Speech-Aware LLM
    Audio → [Audio Encoder] → Embeddings → [LLM] → Text
    Ưu: Direct audio understanding   Nhược: Text-only output

Level 3: Full-Duplex Speech LLM
    Audio   [Unified Model]   Audio + Text
    Ưu: Native speech I/O, low latency   Nhược: Complex training
```

10.2. 9.2 AudioLM — Hierarchical Audio Language Model

10.2.1. 9.2.1 Key Insight

AudioLM [9] là model đầu tiên chứng minh rằng **audio generation = language modeling on discrete tokens**. Sử dụng hierarchical token structure:

```
Level 1: Semantic Tokens (from w2v-BERT)
       Capture linguistic content, prosody
       Low frame rate (~25 Hz)
       Coarse representation

Level 2: Coarse Acoustic Tokens (SoundStream codebook 1-4)
       Capture speaker identity, environment
       Bridge semantic → acoustic
       Medium detail
```

```
Level 3: Fine Acoustic Tokens (SoundStream codebook 5-12)
       Capture timbre, fine details
       High frame rate
       Full reconstruction quality
```

10.2.2. 9.2.2 Three-Stage Generation

$$\begin{aligned} &\text{Stage 1:} \quad P(\mathbf{s}_{t+1:T} \mid \mathbf{s}_{1:t}) \quad \text{// Semantic token continuation} \\ &\text{Stage 2:} \quad P(\mathbf{a}^{1:4} \mid \mathbf{s}) \quad \text{// Semantic} \rightarrow \text{coarse acoustic} \\ &\text{Stage 3:} \quad P(\mathbf{a}^{5:12} \mid \mathbf{a}^{1:4}) \quad \text{// Coarse} \rightarrow \text{fine acoustic} \end{aligned} \tag{10.1}$$

Mỗi stage là một **autoregressive Transformer** — cùng architecture, khác tokenization level.

> **Tại sao Hierarchical?**
>
> Semantic tokens không đủ để reconstruct audio (missing speaker info). Acoustic tokens quá detailed cho LM (sequence quá dài). Hierarchy cho phép:
>
> 1. Plan high-level (semantic) trước
> 2. Fill in details (acoustic) sau
> 3. Giữ sequence lengths manageable cho Transformer

10.2.3. 9.2.3 AudioLM Results

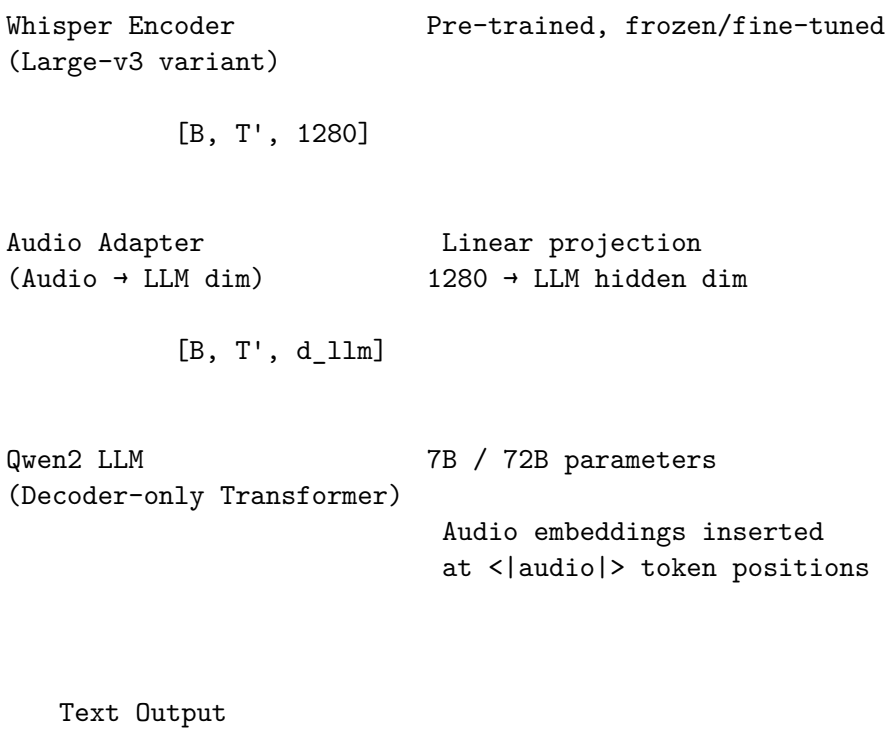
- **Speech continuation**: Nghe tự nhiên, maintain speaker identity
- **Music generation**: Coherent melodies
- **No text conditioning needed**: Purely audio-to-audio

10.3. 9.3 Qwen2-Audio — Universal Audio Understanding

10.3.1. 9.3.1 Architecture

Qwen2-Audio [7] là Speech-Aware LLM — hiểu audio nhưng output text:

```
Audio Input (any type)
```



10.3.2. 9.3.2 Capabilities

Qwen2-Audio hỗ trợ **đa dạng audio tasks** qua text prompting:

Table 10.1.: Qwen2-Audio capabilities

Task	Prompt Example	Output
ASR	"Transcribe this audio"	Text transcription
Translation	"Translate to English"	Translated text
Audio QA	"What instrument is playing?"	"Piano"
Emotion	"What emotion?"	"Happy, excited"
Sound event	"What sounds do you hear?"	"Dog barking, rain"
Speaker ID	"How many speakers?"	"2 speakers"

10.3.3. 9.3.3 Training

2-stage training:

$$\begin{aligned} &\text{Stage 1: Audio-text alignment} && \text{(freeze LLM, train adapter)} \\ &\text{Stage 2: Instruction tuning} && \text{(unfreeze all, multitask)} \end{aligned} \tag{10.2}$$

10.4. 9.4 Moshi — Full-Duplex Speech Dialogue

10.4.1. 9.4.1 What is Full-Duplex?

```
Half-Duplex (traditional):          Full-Duplex (Moshi):
User:   "Hello..."                  User:   "Hello, can you..."
AI:     [waiting...]                AI:     "Hi! Sure, what..."
User:   [waiting...]                User:   "...help me with..."
AI:     "Hi! How can I..."          AI:     "...do you need?"

Turn-by-turn                        Simultaneous, overlapping
High latency (wait for turn)        Low latency (~200ms)
```

10.4.2. 9.4.2 Architecture

Moshi [6] xử lý **2 audio streams song song** (user + AI):

```
                Moshi Architecture

  User Audio  → [Mimi Encoder]  → User Tokens


          Depth Transformer
     (processes both streams jointly)

     Input:  [user_tokens; ai_tokens;
              text_tokens]

     Output: next ai_tokens + text_tokens


  AI Tokens  → [Mimi Decoder]  → AI Audio
  Text Tokens  → Inner monologue (optional)
```

10.4.3. 9.4.3 Dual-Stream Processing

Moshi processes **2 parallel token streams** tại mỗi time step:

$$P(a_t^{\text{AI}}, w_t \mid a_{<t}^{\text{user}}, a_{<t}^{\text{AI}}, w_{<t}) \tag{10.3}$$

trong đó:

- a_t^{user}: Mimi tokens từ user audio tại time t
- a_t^{AI}: Mimi tokens cho AI audio tại time t
- w_t: Text token (inner monologue) tại time t

10.4.4. 9.4.4 Depth Transformer

Moshi sử dụng **Depth Transformer** để handle multi-codebook prediction:

```
Time step t:

Temporal Transformer (shared across codebooks):
    Process: [all_tokens_at_t-1] → hidden state h_t

Depth Transformer (per time step):
    Input: h_t
    For q = 1, 2, ..., Q:
        Predict codebook q token for both user and AI streams
        Condition on previously predicted codebooks
```

$$\begin{aligned} \mathbf{h}_t &= \text{TemporalTransformer}(\text{tokens}_{1:t-1}) \\ c_t^{(q)} &= \text{DepthTransformer}(\mathbf{h}_t, c_t^{(1)}, \ldots, c_t^{(q-1)}) \end{aligned} \tag{10.4}$$

10.4.5. 9.4.5 Inner Monologue

Moshi có khả năng **text reasoning** đồng thời với speech:

$$\text{Audio tokens (speech)} \parallel \text{Text tokens (reasoning)} \tag{10.5}$$

Text stream hoạt động như "inner thought" — giúp model plan response trước khi nói.

10.4.6. 9.4.6 Moshi Specifications

Table 10.2.: Moshi specifications

Parameter	Value
Audio codec	Mimi (12.5 Hz, 8 codebooks)
Total parameters	~7B
Temporal Transformer	32 layers, 4096 dim
Depth Transformer	6 layers, 1024 dim
Latency	~**200ms** (end-to-end)
Training data	7M hours audio
Full-duplex	Yes (simultaneous listen + speak)

⚠ Latency Warning

Latency breakdown cho Moshi:

Component	Latency
Mimi encoder (1 frame)	80ms (= 1/12.5 Hz)
Temporal Transformer	~50ms
Depth Transformer (8 codebooks)	~30ms
Mimi decoder	~40ms
Total	~**200ms**

200ms là dưới ngưỡng cảm nhận delay trong conversation (~300ms). Đây là breakthrough so với cascaded pipeline (ASR ~1s + LLM ~1s + TTS ~0.5s = ~**2.5s**).

```
import torch
import torch.nn as nn
from torch import Tensor


class DualStreamGenerator(nn.Module):
    """Simplified dual-stream speech generation (Moshi-style).

    Processes user and AI audio streams jointly.
    """

    def __init__(
        self,
        vocab_size: int = 1024,       # codec vocabulary
```

```
        n_codebooks: int = 8,          # RVQ layers
        d_model: int = 1024,
        n_heads: int = 16,
        n_temporal_layers: int = 12,
        n_depth_layers: int = 4,
        text_vocab_size: int = 32000,
    ) -> None:
        super().__init__()
        self.n_codebooks: int = n_codebooks

        # Embeddings for user/AI codec tokens
        self.user_embed = nn.ModuleList([
            nn.Embedding(vocab_size, d_model // n_codebooks)
            for _ in range(n_codebooks)
        ])
        self.ai_embed = nn.ModuleList([
            nn.Embedding(vocab_size, d_model // n_codebooks)
            for _ in range(n_codebooks)
        ])
        self.text_embed = nn.Embedding(text_vocab_size, d_model)

        # Temporal transformer (processes time steps)
        temporal_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=d_model * 4,
            batch_first=True,
        )
        self.temporal = nn.TransformerEncoder(
            temporal_layer, num_layers=n_temporal_layers,
        )

        # Depth transformer (processes codebooks at each time step)
        depth_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=n_heads // 2,
            dim_feedforward=d_model * 2,
            batch_first=True,
        )
        self.depth = nn.TransformerEncoder(
            depth_layer, num_layers=n_depth_layers,
        )
```

```
        # Output heads
        self.ai_heads = nn.ModuleList([
            nn.Linear(d_model, vocab_size)
            for _ in range(n_codebooks)
        ])
        self.text_head = nn.Linear(d_model, text_vocab_size)

    def forward_temporal(
        self,
        user_tokens: Tensor,    # [B, T, n_codebooks] - int64
        ai_tokens: Tensor,      # [B, T, n_codebooks] - int64
        text_tokens: Tensor,    # [B, T] - int64
    ) -> Tensor:
        """Temporal transformer: process sequence of time steps.

        Args:
            user_tokens: User codec tokens [B, T, Q] - int64
            ai_tokens: AI codec tokens [B, T, Q] - int64
            text_tokens: Text tokens [B, T] - int64

        Returns:
            h: Hidden states [B, T, d_model] - float32
        """
        B, T, Q = user_tokens.shape

        # Embed and sum codebook embeddings
        user_emb: Tensor = torch.cat([
            self.user_embed[q](user_tokens[:, :, q])
            for q in range(Q)
        ], dim=-1)  # [B, T, d_model] - float32

        ai_emb: Tensor = torch.cat([
            self.ai_embed[q](ai_tokens[:, :, q])
            for q in range(Q)
        ], dim=-1)  # [B, T, d_model] - float32

        text_emb: Tensor = self.text_embed(text_tokens)
        # [B, T, d_model] - float32

        # Combine streams
        combined: Tensor = user_emb + ai_emb + text_emb
        # [B, T, d_model] - float32
```

```
        # Causal mask
        mask: Tensor = nn.Transformer.generate_square_subsequent_mask(
            T, device=combined.device,
        )  # [T, T]

        h: Tensor = self.temporal(combined, mask=mask)
        # [B, T, d_model] - float32
        return h

    @torch.no_grad()
    def generate_step(
        self,
        h_t: Tensor,  # [B, 1, d_model] - float32 (temporal output at time t)
    ) -> tuple[Tensor, Tensor]:
        """Generate one time step: all codebooks + text token.

        Args:
            h_t: Temporal hidden state [B, 1, d_model] - float32

        Returns:
            ai_codes: Predicted AI codec tokens [B, n_codebooks] - int64
            text_tok: Predicted text token [B] - int64
        """
        # Depth transformer processes codebooks sequentially
        depth_input: Tensor = h_t  # [B, 1, d_model] - float32
        ai_codes: list[Tensor] = []

        for q in range(self.n_codebooks):
            depth_out: Tensor = self.depth(depth_input)
            # [B, q+1, d_model] - float32

            logits_q: Tensor = self.ai_heads[q](
                depth_out[:, -1, :]
            )  # [B, vocab] - float32
            code_q: Tensor = logits_q.argmax(dim=-1)  # [B] - int64
            ai_codes.append(code_q)

            # Add predicted code embedding for next depth step
            code_emb: Tensor = self.ai_embed[q](code_q).unsqueeze(1)
            # [B, 1, d_model//Q] - float32
            padded: Tensor = torch.zeros_like(h_t)
            padded[:, :, :code_emb.size(-1)] = code_emb
            depth_input = torch.cat([depth_input, padded], dim=1)
```

```
        # Text token prediction
        text_logits: Tensor = self.text_head(h_t[:, 0, :])
        # [B, text_vocab] - float32
        text_tok: Tensor = text_logits.argmax(dim=-1)  # [B] - int64

        return torch.stack(ai_codes, dim=1), text_tok
        # [B, Q] - int64, [B] - int64
```

10.5. 9.5 GPT-4o Voice Mode

10.5.1. 9.5.1 Capabilities (2024)

GPT-4o voice mode từ OpenAI là first commercial full-duplex Speech LLM:

- **End-to-end**: Audio in → Audio out (không qua ASR/TTS cascade)
- **Emotion/style control**: Whisper, sing, laugh
- **Multilingual**: Realtime translation
- **Low latency**: ~300ms response time
- **Reasoning**: Full GPT-4 reasoning capabilities

10.5.2. 9.5.2 Kiến trúc (suy đoán)

Dựa trên published information, GPT-4o voice likely uses:

```
Audio Input


[Audio Tokenizer]  → Audio Tokens



 Unified Multimodal
 Transformer
 (GPT-4 class)

 Input: [text; audio;
         image tokens]
 Output: [text; audio
          tokens]
```

```
[Audio Detokenizer]  → Audio Output
```

10.6. 9.6 Comparison of Speech LLMs

Table 10.4.: Speech LLM comparison

Model	Type	Audio In	Audio Out	Full-Duplex	Latency	Open?
Cascaded (ASR+LLM+TTS)	Pipeline				~2.5s	
Qwen2-Audio	Speech-aware				~500ms	
AudioLM	Audio LM				~1s	
VALL-E	TTS LM				~2s	
Moshi	Full-duplex				**~200ms**	
GPT-4o Voice	Full-duplex				**~300ms**	

10.7. 9.7 Tóm tắt

Table 10.5.: Speech LLM evolution

Evolution	Model	Key Innovation
Audio → Tokens → LM	AudioLM	Hierarchical token LM
Audio → LLM → Text	Qwen2-Audio	Audio adapter for LLM
Full-duplex dialogue	Moshi	Dual-stream + Depth Transformer
Commercial	GPT-4o	End-to-end multimodal

Chương tiếp theo sẽ khám phá **Vietnamese Speech Processing** — thách thức đặc thù của tiếng Việt (6 tones, dialects) và các solutions hiện có.

11. Vietnamese Speech Processing

11.1. 10.1 Đặc điểm Ngôn ngữ học của Tiếng Việt

11.1.1. 10.1.1 Hệ thống Thanh điệu

Tiếng Việt là **tonal language** với 6 thanh điệu (lexical tones) — thay đổi thanh điệu thay đổi nghĩa hoàn toàn:

Table 11.1.: 6 thanh điệu tiếng Việt

Thanh	Tên	Ký hiệu	Ví dụ	F0 Pattern
1	Ngang (level)	không dấu	ma (ghost)	Mid-level, flat
2	Sắc (rising)	◌́	má (cheek)	Rising
3	Huyền (falling)	◌̀	mà (but)	Low falling
4	Hỏi (dipping-rising)	◌̉	mả (tomb)	Falling then rising
5	Ngã (rising-glottalized)	◌̃	mã (code)	Rising with glottal break
6	Nặng (falling-glottalized)	◌̣	mạ (rice seedling)	Low falling, short

Tone = Fundamental Frequency (F0) Contour

Mỗi thanh điệu tương ứng với một **F0 contour** (đường biến thiên tần số cơ bản) đặc trưng. F0 là đặc trưng quan trọng nhất cần capture trong cả ASR và TTS cho tiếng Việt:

$$\text{Tone} \leftrightarrow F0(t), \quad t \in [t_{\text{onset}}, t_{\text{offset}}] \tag{11.1}$$

F0 range điển hình:

- Nam: 85–180 Hz
- Nữ: 165–300 Hz

11.1.2. 10.1.2 Thách thức cho ASR

Hỏi-Ngã Merger (thanh Hỏi và thanh Ngã):

Trong phương ngữ Nam Bộ, thanh Hỏi (falling-rising) và thanh Ngã (rising-glottalized) **hợp nhất** thành một thanh:

$$\text{Bắc Bộ: } \underbrace{\text{mả}}_{\text{Hỏi}} \neq \underbrace{\text{mã}}_{\text{Ngã}} \qquad \text{Nam Bộ: } \underbrace{\text{mả} \approx \text{mã}}_{\text{merged}} \tag{11.2}$$

Điều này nghĩa là ASR model phải:

1. Nhận diện thanh điệu từ F0 contour
2. Xử lý variation theo phương ngữ
3. Dùng context để disambiguate khi cần

11.1.3. 10.1.3 Ba Phương ngữ Chính

Table 11.2.: Ba phương ngữ chính tiếng Việt

Đặc điểm	Bắc Bộ (Hà Nội)	Trung Bộ (Huế)	Nam Bộ (Sài Gòn)
Số thanh phân biệt	6	5	5
Hỏi-Ngã	Phân biệt rõ	Có xu hướng merger	**Merged**
Phụ âm đầu	/z/ trong “da”	/j/ trong “da”	/j/ trong “da”
Phụ âm cuối	/-k/, /-t/, /-p/ rõ	Biến thể	Có thể nuốt
Ngữ điệu	Flat, formal	Melodic	Expressive

⚠ Latency Warning

Dialect-specific models cho kết quả tốt hơn, nhưng cần **3 models riêng biệt** → 3× VRAM và complexity. Production approach: train **1 unified model** với dialect labels, fine-tune per-dialect nếu cần.

11.2. 10.2 Vietnamese ASR

11.2.1. 10.2.1 Datasets

Table 11.3.: Vietnamese ASR datasets

Dataset	Giờ	Loại	Dialect	Open?
VIVOS	15h	Read speech	Bắc	
VLSP (2018–2020)	100h+	Broadcast, conversation	Mixed	Partial
FPT Open Speech	27h	Read speech	Bắc, Nam	
CommonVoice (vi)	~50h	Crowdsourced	Mixed	
Internal datasets	1000h+	Call center, meetings	Mixed	

11.2.2. 10.2.2 Approaches

Approach 1: Fine-tune Whisper

$$\text{Whisper Large-v3} \xrightarrow{\text{Fine-tune trên Vietnamese data}} \text{Whisper-Vi} \tag{11.3}$$

```python
import torch
from torch import Tensor


def prepare_whisper_vietnamese_training(
    audio_paths: list[str],
    transcriptions: list[str],
    max_duration_sec: float = 30.0,
    sample_rate: int = 16000,
) -> dict[str, list]:
    """Prepare training data for Whisper Vietnamese fine-tuning.

    Args:
        audio_paths: Paths to audio files
        transcriptions: Corresponding Vietnamese transcriptions
        max_duration_sec: Maximum audio duration
        sample_rate: Target sample rate

    Returns:
        dataset: Dictionary with processed examples
    """
    # Vietnamese special considerations:
```

```
    # 1. Normalize Unicode (NFC normalization for diacritics)
    # 2. Handle tone marks consistently
    # 3. Lowercase (Vietnamese is case-sensitive for proper nouns)

    import unicodedata

    processed_texts: list[str] = []
    for text in transcriptions:
        # NFC normalization - critical for Vietnamese diacritics
        normalized: str = unicodedata.normalize("NFC", text)
        processed_texts.append(normalized)

    # Whisper uses special tokens
    # <|startoftranscript|><|vi|><|transcribe|><|notimestamps|>...text...<|endoftex
    prefix_tokens: str = "<|startoftranscript|><|vi|><|transcribe|><|notimestamps|>

    return {
        "audio_paths": audio_paths,
        "texts": processed_texts,
        "language": "vi",
        "task": "transcribe",
    }
```

Approach 2: Wav2Vec 2.0 + CTC

$$\text{XLSR-53 (multilingual Wav2Vec 2.0)} \xrightarrow{\text{CTC fine-tune}} \text{Vietnamese ASR} \tag{11.4}$$

- XLSR-53: Pre-trained on 56K hours, 53 languages
- Fine-tune với CTC loss trên Vietnamese data
- Character-level vocabulary (chữ cái + dấu thanh)

Approach 3: Conformer + RNN-T (Production)

$$\text{Conformer Encoder} + \text{RNN-T Decoder} \xrightarrow{\text{Train from scratch}} \text{Streaming Vietnamese ASR} \tag{11.5}$$

11.2.3. 10.2.3 Vietnamese WER Results

Table 11.4.: Vietnamese ASR results

Model	Dataset	WER
Kaldi GMM-HMM [23]	VIVOS	28.1%
Wav2Vec 2.0 (XLSR fine-tune)	VIVOS	9.8%
Whisper Large-v3	VIVOS	8.2%
Whisper Large-v3 (fine-tuned)	VIVOS	**5.1%**
Conformer-RNN-T (from scratch)	Internal 1000h	**4.5%**

11.3. 10.3 Vietnamese TTS

11.3.1. 10.3.1 Thách thức cho TTS

1. **Tone generation**: Mỗi âm tiết phải có đúng F0 contour
2. **Coarticulation**: Ảnh hưởng giữa các âm liên tiếp
3. **Prosody**: Nhấn mạnh, nghi vấn, cảm thán
4. **Polyphone**: Cùng chữ viết, phát âm khác theo context

11.3.2. 10.3.2 Text Processing Pipeline cho Tiếng Việt

```
Raw Text: "Tôi có 3 con mèo, giá 50.000đ/con."


[Text Normalization]
    "tôi có ba con mèo giá năm mươi nghìn đồng một con"


[Word Segmentation] (VnCoreNLP / underthesea)
    "tôi / có / ba / con_mèo / giá / năm_mươi / nghìn / đồng / một / con"


[G2P - Grapheme to Phoneme]
    "t oj1 / k  5 / b a1 / k  n1 m  w2 / ..."


[Phoneme + Tone Sequence]
    Input to TTS model
```

11.3.3. 10.3.3 Vietnamese G2P

Vietnamese grapheme-to-phoneme tương đối **regular** (gần 1:1 mapping) so với English, nhưng có edge cases:

Table 11.5.: Vietnamese G2P challenges

Challenge	Example	Issue
Polyphone "gi"	"gì" vs "giá"	/z/ vs /z/ (dialect-dependent)
Regional variants	"v" in "vào"	/v/ (Bắc) vs /j/ (Nam)
Tone sandhi	"không" in questions	Tone may shift in context
Foreign words	"email", "marketing"	Need English pronunciation rules

11.3.4. 10.3.4 TTS Models cho Tiếng Việt

Table 11.6.: Vietnamese TTS models

Model	Approach	Quality	Speed	Open?
VITS (Vietnamese fine-tune)	End-to-end	Good	Fast	
FastSpeech 2 + HiFi-GAN	Two-stage	Good	Fast	
F5-TTS (Vietnamese)	Flow matching	**Excellent**	Medium	
Zalo TTS	Proprietary	Excellent	Fast	
FPT.AI TTS	Proprietary	Excellent	Fast	

```
def normalize_vietnamese_text(text: str) -> str:
    """Normalize Vietnamese text for TTS.

    Handles: numbers, abbreviations, special characters.

    Args:
        text: Raw Vietnamese text

    Returns:
        normalized: Normalized text ready for G2P
    """
    import re
    import unicodedata
```

```
    # Step 1: Unicode NFC normalization
    text = unicodedata.normalize("NFC", text)

    # Step 2: Number to words (simplified)
    number_words: dict[str, str] = {
        "0": "không", "1": "một", "2": "hai", "3": "ba",
        "4": "bốn", "5": "năm", "6": "sáu", "7": "bảy",
        "8": "tám", "9": "chín", "10": "mười",
    }

    # Step 3: Common abbreviations
    abbreviations: dict[str, str] = {
        "TP.HCM": "thành phố hồ chí minh",
        "TPHCM": "thành phố hồ chí minh",
        "VN": "việt nam",
        "VND": "việt nam đồng",
        "AI": "ây ai",
        "ML": "em eo",
        "NLP": "en eo pi",
        "km": "ki lô mét",
        "kg": "ki lô gam",
    }

    for abbr, expansion in abbreviations.items():
        text = text.replace(abbr, expansion)

    # Step 4: Remove special characters, keep Vietnamese diacritics
    text = re.sub(r"[^\w\sàáạảãâầấậẩẫăằắặẳẵèéẹẻẽêềếệểễ"
                  r"ìíịỉĩòóọỏõôồốộổỗơờớợởỡùúụủũưừứựửữ"
                  r"ỳýỵỷỹđ]", " ", text.lower())

    # Step 5: Collapse whitespace
    text = re.sub(r"\s+", " ", text).strip()

    return text


# Example
raw: str = "Tôi có 3 con mèo, giá 50.000đ/con ở TP.HCM"
normalized: str = normalize_vietnamese_text(raw)
print(f"Input:  {raw}")
print(f"Output: {normalized}")
```

11.4. 10.4 Vietnamese Voice Cloning

11.4.1. 10.4.1 Zero-Shot Cloning

VALL-E và F5-TTS có thể clone giọng Việt chỉ với **3 giây** audio prompt:

$$\text{[Text Vietnamese]} + \text{[3s Audio Prompt]} \xrightarrow{\text{F5-TTS}} \text{[Cloned Voice Audio]} \tag{11.6}$$

Thách thức đặc thù:

- Tone accuracy: Model phải generate đúng F0 contour cho 6 thanh
- Speaker preservation: Giữ timbre trong khi thay đổi content
- Code-switching: Vietnamese + English trong cùng utterance

11.4.2. 10.4.2 Speaker Embedding for Vietnamese

$$\mathbf{s} = \text{SpeakerEncoder}(\text{reference\_audio}), \quad \mathbf{s} \in \mathbb{R}^{256} \tag{11.7}$$

Speaker encoder cần capture:

- Vocal tract characteristics (speaker identity)
- **Tonal range** (mỗi người có range F0 khác nhau)
- **Dialect markers** (Bắc/Trung/Nam)
- Speaking rate

11.5. 10.5 Vietnamese Speech Datasets & Resources

11.5.1. 10.5.1 Open-Source Resources

Table 11.7.: Vietnamese speech resources

Resource	Type	Description
VIVOS	ASR corpus	15h, read speech, Northern Vietnamese
VinBigData VLSP	ASR corpus	Competition data, mixed dialects
CommonVoice vi	ASR corpus	Crowdsourced, ~50h
underthesea	NLP toolkit	Word segmentation, POS tagging
vPhon	G2P tool	Vietnamese phonetic transcription
Vinorm	Text normalizer	Vietnamese text normalization

11.5.2. 10.5.2 Evaluation Metrics for Vietnamese

ASR:

- **WER**: Standard, nhưng cần careful word segmentation (tiếng Việt viết cách âm tiết)
- **SER** (Syllable Error Rate): Phù hợp hơn vì tiếng Việt syllable-timed
- **Tone Accuracy**: % thanh điệu đúng (metric riêng cho tonal languages)

TTS:

- **MOS** (Mean Opinion Score): 1–5 scale, human evaluation
- **Tone MOS**: Đánh giá riêng cho tone naturalness
- **Speaker Similarity**: Cosine similarity of speaker embeddings

11.6. 10.6 Tóm tắt

Table 11.8.: Vietnamese speech processing summary

Aspect	Challenge	Solution
Tones (6)	F0 contour critical	Pitch predictor, tone embedding
Dialects (3+)	Phoneme/tone variation	Unified model + dialect labels
Hỏi-Ngã merger	Ambiguity in Southern	Context-based disambiguation
Text normalization	Numbers, abbreviations	Rule-based + dictionary
G2P	Mostly regular, some polyphones	Rule-based + exceptions
Data scarcity	Limited labeled data	Transfer learning (Whisper, XLSR)

Chương tiếp theo sẽ hoàn thiện cuốn sách với **Production Speech Systems** — cách xây dựng voice AI pipeline cho real-world deployment.

Part V.

Phần V: Production

12. Production Speech Systems

12.1. 11.1 End-to-End Voice AI Pipeline

Production voice AI system kết nối nhiều components thành pipeline hoàn chỉnh:

```
                    Voice AI Pipeline

Microphone

[VAD]  → Speech segments only (filter silence/noise)

[Streaming ASR]  → Partial transcripts → Final transcript

[NLU / LLM]  → Understanding + Response generation

[Streaming TTS]  → Audio chunks → Speaker

Speaker

Target: End-to-end latency < 1 second
```

12.1.1. 11.1.1 Latency Budget

$$L_{\text{total}} = L_{\text{VAD}} + L_{\text{ASR}} + L_{\text{LLM}} + L_{\text{TTS}} + L_{\text{network}} \tag{12.1}$$

Table 12.1.: Latency budget breakdown

Component	Target Latency	Typical Range
VAD	30–50ms	20–100ms
Streaming ASR	100–300ms	50–500ms
LLM (first token)	100–300ms	50ms–2s
Streaming TTS	50–200ms	30–500ms
Network (2× RTT)	50–100ms	20–500ms
Total	**<1s**	**300ms–3s**

⚠ Latency Warning

1 second rule: Nghiên cứu UX cho thấy users bắt đầu cảm thấy delay sau **300ms** và **frustrated** sau **1 second**. Conversation flow bị phá vỡ sau **2 seconds**. Target < 1s cho acceptable experience.

12.2. 11.2 Voice Activity Detection (VAD)

12.2.1. 11.2.1 Bài toán

VAD phân loại mỗi audio frame là **speech** hoặc **non-speech**:

$$\hat{y}_t = \begin{cases} 1 & \text{if frame } t \text{ contains speech} \\ 0 & \text{otherwise} \end{cases} \tag{12.2}$$

12.2.2. 11.2.2 Silero VAD

Silero VAD — lightweight model phổ biến nhất cho production:

Table 12.2.: Silero VAD specifications

Feature	Value
Model size	~2MB
Latency	<1ms per frame
Frame size	30ms / 60ms / 90ms
Accuracy	>99% trên clean speech
Languages	Language-agnostic

```python
import torch
from torch import Tensor


class VADSegmenter:
    """Voice Activity Detection based audio segmenter.

    Segments continuous audio stream into speech chunks.
    """

    def __init__(
        self,
        threshold: float = 0.5,
        min_speech_ms: int = 250,
        min_silence_ms: int = 300,
        sample_rate: int = 16000,
        window_ms: int = 30,
    ) -> None:
        self.threshold: float = threshold
        self.min_speech_frames: int = min_speech_ms // window_ms
        self.min_silence_frames: int = min_silence_ms // window_ms
        self.sample_rate: int = sample_rate
        self.window_size: int = sample_rate * window_ms // 1000

        # State for streaming
        self.speech_count: int = 0
        self.silence_count: int = 0
        self.is_speaking: bool = False
        self.buffer: list[Tensor] = []

    def process_frame(
        self,
        frame: Tensor,          # [window_size] - float32
        speech_prob: float,     # VAD probability for this frame
    ) -> tuple[bool, Tensor | None]:
        """Process single audio frame.

        Args:
            frame: Audio frame [window_size] - float32
            speech_prob: Speech probability (0-1)

        Returns:
            segment_complete: Whether a complete segment is ready
```

```
            segment: Complete speech segment or None
        """
        self.buffer.append(frame)

        if speech_prob >= self.threshold:
            self.speech_count += 1
            self.silence_count = 0

            if not self.is_speaking and self.speech_count >= self.min_speech_frames
                self.is_speaking = True
        else:
            self.silence_count += 1

            if self.is_speaking and self.silence_count >= self.min_silence_frames:
                # End of speech segment
                self.is_speaking = False
                segment: Tensor = torch.cat(self.buffer)
                # [total_samples] - float32
                self.buffer = []
                self.speech_count = 0
                return True, segment

        return False, None
```

12.3. 11.3 Streaming ASR

12.3.1. 11.3.1 Streaming vs Offline

Table 12.3.: Streaming vs offline ASR

Aspect	Offline	Streaming
Input	Complete utterance	Audio chunks
Latency	High (wait for end)	Low (process in real-time)
Accuracy	Higher (full context)	Lower (partial context)
Architecture	Bidirectional encoder	**Causal/chunked** encoder
Use case	Transcription	Voice assistant, live caption

12.3.2. 11.3.2 Chunked Streaming

Chia audio thành chunks và process incrementally:

$$\begin{aligned} \text{Chunk}_1 &: [0, C] && \to \text{Partial}_1 \\ \text{Chunk}_2 &: [C, 2C] && \to \text{Partial}_2 \\ &\vdots \\ \text{Chunk}_N &: [(N-1)C, NC] && \to \text{Final} \end{aligned} \tag{12.3}$$

với chunk size C thường là 160ms–640ms.

12.3.3. 11.3.3 Latency-Accuracy Trade-off

$$\text{Latency} \propto C + L_{\text{lookahead}} \tag{12.4}$$

Table 12.4.: Streaming ASR latency-accuracy trade-off

Chunk Size	Lookahead	Latency	WER Increase
160ms	0	160ms	+15% relative
320ms	160ms	480ms	+8% relative
640ms	320ms	960ms	+3% relative
Full utterance	Full	>2s	Baseline

12.3.4. 11.3.4 Endpointing

Endpointing quyết định **khi nào user đã nói xong**:

```
Strategy 1: VAD-based
    Silence > threshold → end of utterance
    Problem: Pauses within utterance cause premature cutoff

Strategy 2: Semantic endpointing
    LLM/classifier decides if utterance is complete
    More accurate but adds latency

Strategy 3: Hybrid
    VAD for quick detection + semantic for confirmation
    Best balance for production
```

12.4. 11.4 Streaming TTS

12.4.1. 11.4.1 Chunk-based Generation

Thay vì generate toàn bộ audio rồi play, streaming TTS tạo và play **từng chunk**:

$$\text{Text} \xrightarrow{\text{chunk}_1} \text{Audio}_1 \xrightarrow{\text{play}} \text{chunk}_2 \xrightarrow{\text{play}} \cdots \tag{12.5}$$

12.4.2. 11.4.2 First-Chunk Latency

$$L_{\text{TTS, first}} = L_{\text{text\_process}} + L_{\text{mel\_first\_chunk}} + L_{\text{vocoder\_first\_chunk}} \tag{12.6}$$

Table 12.5.: TTS first-chunk latency

TTS Model	First Chunk Latency	Total RTF
FastSpeech 2 + HiFi-GAN	~**30ms**	0.02
VITS	~50ms	0.01
F5-TTS	~200ms	0.15
VALL-E	~1000ms	0.8

12.5. 11.5 Inference Optimization

12.5.1. 11.5.1 Quantization

$$\text{FP32 (4B)} \xrightarrow{\text{FP16}} \text{FP16 (2B)} \xrightarrow{\text{INT8}} \text{INT8 (1B)} \xrightarrow{\text{INT4}} \text{INT4 (0.5B)} \tag{12.7}$$

Table 12.6.: Quantization trade-offs

Precision	Memory	Speed vs FP32	Quality Loss
FP32	1×	1×	None
FP16	0.5×	1.5–2×	Negligible
INT8	0.25×	2–3×	Minor (<1% WER)
INT4	0.125×	3–4×	Moderate (1–3% WER)

⚠ Latency Warning

Quantization impact varies by model:

- **Whisper**: INT8 ảnh hưởng rất ít (<0.5% WER increase)
- **TTS models**: FP16 safe, INT8 có thể ảnh hưởng quality (audible artifacts)
- **Vocoders (HiFi-GAN)**: Rất nhạy cảm — recommend FP16 minimum

Rule of thumb: ASR INT8 ok, TTS keep FP16.

12.5.2. 11.5.2 KV Cache Optimization

Cho autoregressive models (Whisper decoder, VALL-E):

$$\text{Memory}_{\text{KV}} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times L_{\text{seq}} \times \text{dtype\_size} \tag{12.8}$$

Optimization strategies:

- **Paged Attention** (vLLM): Quản lý KV cache theo pages, tránh fragmentation
- **Multi-Query Attention (MQA)**: Chia sẻ K,V heads → giảm KV cache size
- **Sliding Window**: Chỉ cache W recent tokens thay vì toàn bộ

12.5.3. 11.5.3 Batching Strategies

Table 12.7.: Batching strategies

Strategy	Throughput	Latency	Use Case
No batching	Low	**Lowest**	Single user, real-time
Static batching	Medium	Medium	Offline transcription
Dynamic batching	**High**	Low	Production server
Continuous batching	**Highest**	Low	High-traffic API

```
import torch
import time
from torch import Tensor
from dataclasses import dataclass, field


@dataclass
class ASRRequest:
```

```
    """Single ASR request."""

    audio: Tensor            # [1, T_samples] - float32
    request_id: str
    timestamp: float = field(default_factory=time.time)
    result: str | None = None


class DynamicBatcher:
    """Dynamic batching for ASR inference.

    Collects requests and batches them for efficient GPU utilization.
    """

    def __init__(
        self,
        max_batch_size: int = 16,
        max_wait_ms: float = 50.0,
        max_audio_length: int = 480000,  # 30s at 16kHz
    ) -> None:
        self.max_batch_size: int = max_batch_size
        self.max_wait_ms: float = max_wait_ms
        self.max_audio_length: int = max_audio_length
        self.queue: list[ASRRequest] = []

    def add_request(self, request: ASRRequest) -> None:
        """Add request to queue."""
        self.queue.append(request)

    def should_process(self) -> bool:
        """Check if batch should be processed.

        Returns:
            True if batch is ready (full or timeout)
        """
        if len(self.queue) == 0:
            return False
        if len(self.queue) >= self.max_batch_size:
            return True
        # Check timeout
        oldest: float = self.queue[0].timestamp
        wait_ms: float = (time.time() - oldest) * 1000
        return wait_ms >= self.max_wait_ms
```

```python
    def get_batch(self) -> tuple[Tensor, list[ASRRequest]]:
        """Get padded batch of audio.

        Returns:
            batch: Padded audio [B, max_T] - float32
            requests: Corresponding request objects
        """
        batch_requests: list[ASRRequest] = self.queue[:self.max_batch_size]
        self.queue = self.queue[self.max_batch_size:]

        # Pad to max length in batch
        max_len: int = max(r.audio.size(-1) for r in batch_requests)
        max_len = min(max_len, self.max_audio_length)

        batch_size: int = len(batch_requests)
        batch: Tensor = torch.zeros(
            batch_size, max_len,
        )  # [B, max_T] - float32

        for i, req in enumerate(batch_requests):
            length: int = min(req.audio.size(-1), max_len)
            batch[i, :length] = req.audio[0, :length]

        return batch, batch_requests
```

12.6. 11.6 TensorRT-LLM & Triton Inference Server

12.6.1. 11.6.1 TensorRT-LLM cho Whisper

```
Whisper Model (PyTorch)


[TensorRT-LLM Build]
  - FP16/INT8 quantization
  - Kernel fusion
  - KV cache management


TensorRT Engine (.engine)
```

```
[Triton Inference Server]
  - Dynamic batching
  - Model versioning
  - gRPC/HTTP API
  - Monitoring (Prometheus)
```

12.6.2. 11.6.2 Performance with TensorRT-LLM

Table 12.8.: TensorRT-LLM Whisper performance

Configuration	RTF	Throughput	VRAM
Whisper Large-v3 (PyTorch FP16)	0.03	33× RT	6 GB
Whisper Large-v3 (TRT-LLM FP16)	0.01	**100× RT**	4 GB
Whisper Large-v3 (TRT-LLM INT8)	0.008	**125× RT**	2.5 GB

12.7. 11.7 Production Architecture

12.7.1. 11.7.1 Microservice Architecture

```
                Load Balancer


    VAD          ASR          TTS
    Service      Service      Service
    (CPU)        (GPU)        (GPU)


              Message Queue
          (Redis / Kafka / gRPC)


    LLM          NLU          Dialog
    Service      Service      Manager
```

12.7.2. 11.7.2 GPU Resource Planning

Table 12.9.: GPU resource planning

Component	Model	VRAM	GPU Type
ASR	Whisper Large-v3 (INT8)	2.5 GB	T4 / L4
TTS	VITS / FastSpeech 2 (FP16)	1 GB	T4 / L4
LLM	Qwen2-7B (INT4)	4 GB	A10G / L4
Voice Cloning	F5-TTS (FP16)	3 GB	A10G
Total	-	**~10.5 GB**	**1× A10G (24GB)**

Cost Optimization

- **Spot/preemptible instances**: 60–70% cheaper, OK cho batch processing
- **Autoscaling**: Scale GPU pods dựa trên request queue length
- **Model sharing**: Nhiều models trên 1 GPU với time-sharing
- **CPU fallback**: VAD, text normalization, G2P chạy trên CPU

12.8. 11.8 Monitoring & Observability

12.8.1. 11.8.1 Key Metrics

Table 12.10.: Production monitoring metrics

Metric	Target	Alert Threshold
E2E latency (P50)	<800ms	>1.2s
E2E latency (P99)	<2s	>3s
ASR WER	<10%	>15%
TTS MOS (auto)	>3.5	<3.0
Availability	99.9%	<99.5%
GPU utilization	60–80%	<30% or >95%

12.9. 11.9 Tóm tắt

Table 12.11.: Production speech systems summary

Topic	Key Takeaway
Pipeline	VAD → ASR → LLM → TTS, target <1s E2E
Streaming	Chunked processing cho cả ASR và TTS
Optimization	INT8 cho ASR, FP16 cho TTS, dynamic batching
Infrastructure	TensorRT-LLM + Triton, microservice architecture
Monitoring	P50/P99 latency, WER, MOS, GPU utilization

$$\text{Production Speech AI} = \text{Model Quality} \times \text{Inference Speed} \times \text{System Reliability} \quad (12.9)$$

Mỗi yếu tố đều quan trọng — model tốt nhưng chậm sẽ không được users chấp nhận, model nhanh nhưng kém chất lượng sẽ bị bỏ qua.

Part VI.

Phụ lục

13. Notation Reference

13.1. A.1 General Notation

Symbol	Meaning
$\mathbf{x}$	Waveform signal (1D time-domain)
$\mathbf{X}$	Matrix (uppercase bold)
x_n	Sample at time index n
N	Number of samples / sequence length
f_s	Sampling rate (Hz)
T	Number of time frames (mel/feature level)
U	Number of text/phoneme tokens
B	Batch size
d	Model dimension / hidden size
K	Codebook size / number of classes
Q	Number of RVQ codebook layers
$\mathcal{L}$	Loss function

13.2. A.2 Signal Processing

Symbol	Meaning	Reference
f_s	Sampling frequency (Hz)	Ch. 2
f_{Nyquist}	Nyquist frequency $= f_s/2$	Ch. 2
$X[k]$	DFT coefficient at frequency bin k	Ch. 2
$w[n]$	Window function (Hann, Hamming)	Ch. 2
$\text{STFT}(m, k)$	Short-Time Fourier Transform	Ch. 2
S_{mel}	Mel spectrogram	Ch. 2
MFCC	Mel-Frequency Cepstral Coefficients	Ch. 2
F_0 / f_0	Fundamental frequency (pitch)	Ch. 2, 10
$\mathcal{M}(f)$	Mel scale: $2595 \log_{10}(1 + f/700)$	Ch. 2

13.3. A.3 ASR Notation

Symbol	Meaning	Reference
$P(\mathbf{y} \mid \mathbf{x})$	ASR posterior probability	Ch. 3
π	CTC alignment path (with blanks)	Ch. 3
$\alpha_t(s)$	CTC forward variable	Ch. 3
$\beta_t(s)$	CTC backward variable	Ch. 3
blank / $\emptyset$	CTC blank token	Ch. 3
$\alpha_{i,j}$	Attention weights	Ch. 3, 5
WER	Word Error Rate	Ch. 3
CER	Character Error Rate	Ch. 3

13.4. A.4 Self-Supervised Learning

Symbol	Meaning	Reference
$\mathbf{z}$	Latent representation	Ch. 4
$\mathbf{q}$	Quantized representation	Ch. 4
$\mathcal{C}$	Codebook (set of entries)	Ch. 4, 8
$\mathbf{e}_k$	Codebook entry k	Ch. 4, 8
G / V	Number of codebook groups / entries	Ch. 4
τ	Gumbel-Softmax temperature	Ch. 4
$\mathcal{L}_{\text{contrastive}}$	Contrastive loss	Ch. 4
$\mathcal{L}_{\text{diversity}}$	Diversity loss	Ch. 4

13.5. A.5 TTS Notation

Symbol	Meaning	Reference
$\hat{\mathbf{m}}_i$	Predicted mel frame at step i	Ch. 6
d_u	Duration of phoneme u (in frames)	Ch. 6
$\hat{f}_0(t)$	Predicted pitch at frame t	Ch. 6
$\hat{e}(t)$	Predicted energy at frame t	Ch. 6
p_{stop}	Stop token probability	Ch. 6
v_θ	Flow matching velocity field	Ch. 7
$\mathbf{x}_t$	Interpolated sample at flow time t	Ch. 7

13.6. A.6 Neural Codec Notation

Symbol	Meaning	Reference
$q(\mathbf{z})$	Vector quantization operator	Ch. 8
$\text{sg}(\cdot)$	Stop-gradient operator	Ch. 8
$\mathbf{r}_q$	Residual after q-th quantization	Ch. 8
$c_t^{(q)}$	Codec token at time t, codebook q	Ch. 8, 9
$\hat{\mathbf{z}}$	Reconstructed latent (sum of RVQ)	Ch. 8

13.7. A.7 Speech LLM Notation

Symbol	Meaning	Reference
$\mathbf{s}$	Semantic tokens	Ch. 9
$\mathbf{a}^{(q)}$	Acoustic tokens, codebook q	Ch. 9
a_t^{user}	User audio token at time t	Ch. 9
a_t^{AI}	AI audio token at time t	Ch. 9
w_t	Text (inner monologue) token	Ch. 9

13.8. A.8 Common Abbreviations

Abbreviation	Full Name
ASR	Automatic Speech Recognition
TTS	Text-to-Speech
VAD	Voice Activity Detection
CTC	Connectionist Temporal Classification
RNN-T	Recurrent Neural Network Transducer
SSL	Self-Supervised Learning
RVQ	Residual Vector Quantization
VQ	Vector Quantization
STE	Straight-Through Estimator
MOS	Mean Opinion Score
WER	Word Error Rate
RTF	Real-Time Factor
G2P	Grapheme-to-Phoneme
F0	Fundamental Frequency
MAS	Monotonic Alignment Search
CFM	Conditional Flow Matching

Abbreviation	Full Name
DiT	Diffusion Transformer
MQA	Multi-Query Attention

14. Mathematical Proofs

14.1. B.1 Discrete Fourier Transform (DFT) Completeness

14.1.1. B.1.1 Statement

DFT basis vectors form an orthogonal set — any discrete signal of length N can be perfectly reconstructed from its N DFT coefficients.

14.1.2. B.1.2 Proof

Orthogonality of DFT basis vectors:

Define DFT basis vector $\mathbf{w}_k = [1, e^{-j2\pi k/N}, e^{-j2\pi k \cdot 2/N}, \dots, e^{-j2\pi k(N-1)/N}]$.

$$\langle \mathbf{w}_k, \mathbf{w}_l \rangle = \sum_{n=0}^{N-1} e^{-j2\pi kn/N} \cdot e^{j2\pi ln/N} = \sum_{n=0}^{N-1} e^{j2\pi(l-k)n/N} \tag{14.1}$$

Case 1: $k = l$

$$\sum_{n=0}^{N-1} e^{j2\pi \cdot 0 \cdot n/N} = \sum_{n=0}^{N-1} 1 = N$$

Case 2: $k \neq l$, let $r = l - k \neq 0$

$$\sum_{n=0}^{N-1} e^{j2\pi rn/N} = \frac{1 - e^{j2\pi r}}{1 - e^{j2\pi r/N}} = \frac{1-1}{1 - e^{j2\pi r/N}} = 0$$

since $e^{j2\pi r} = 1$ for integer r.

Therefore: $\langle \mathbf{w}_k, \mathbf{w}_l \rangle = N \cdot \delta_{kl}$ — orthogonal basis. ■

Inverse DFT follows directly:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{j2\pi kn/N} \tag{14.2}$$

14.2. B.2 CTC Forward-Backward Algorithm

14.2.1. B.2.1 Statement

The CTC loss can be efficiently computed in $O(T \times U)$ time using dynamic programming, where T is the number of frames and U is the label length.

14.2.2. B.2.2 Forward Variable

Define $\alpha_t(s) =$ probability of outputting the first s symbols of the modified label sequence ℓ' (with blanks) in t time steps:

$$\alpha_t(s) = P(\pi_{1:t} \text{ maps to } \ell'_{1:s} \mid \mathbf{x}) \tag{14.3}$$

Initialization $(t = 1)$:

$$\alpha_1(1) = P(\text{blank} \mid \mathbf{x}_1), \quad \alpha_1(2) = P(\ell_1 \mid \mathbf{x}_1), \quad \alpha_1(s) = 0 \text{ for } s > 2 \tag{14.4}$$

Recursion $(t > 1)$:

$$\alpha_t(s) = P(\ell'_s \mid \mathbf{x}_t) \cdot \begin{cases} \alpha_{t-1}(s) + \alpha_{t-1}(s-1) & \text{if } \ell'_s = \text{blank or } \ell'_s = \ell'_{s-2} \\ \alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \alpha_{t-1}(s-2) & \text{otherwise} \end{cases} \tag{14.5}$$

Proof of correctness:

At time t, the token at position s in ℓ' can be reached from:

1. **Same position** s at $t-1$: Repeating the same token
2. **Previous position** $s-1$ at $t-1$: Moving to next token
3. **Two positions back** $s-2$ at $t-1$: Skipping a blank (only if $\ell'_s \neq \ell'_{s-2}$, i.e., no repeated characters)

Case 3 is excluded when $\ell'_s =$ blank or when two consecutive non-blank tokens are the same, because skipping the blank between them would cause a merge. ■

14.2.3. B.2.3 Total CTC Loss

$$P(\ell \mid \mathbf{x}) = \alpha_T(|\ell'|) + \alpha_T(|\ell'| - 1) \tag{14.6}$$

The two terms account for paths ending with the last label or with a trailing blank.

$$\mathcal{L}_{\text{CTC}} = -\log P(\ell \mid \mathbf{x}) \tag{14.7}$$

14.3. B.3 VQ-VAE ELBO Derivation

14.3.1. B.3.1 Statement

The VQ-VAE training objective is a lower bound on the log-likelihood $\log p_\theta(\mathbf{x})$.

14.3.2. B.3.2 Derivation

Starting from the marginal log-likelihood:

$$\log p_\theta(\mathbf{x}) = \log \sum_{\mathbf{z}} p_\theta(\mathbf{x}, \mathbf{z}) \tag{14.8}$$

Introduce approximate posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$:

$$\log p_\theta(\mathbf{x}) = \log \sum_{\mathbf{z}} q_\phi(\mathbf{z} \mid \mathbf{x}) \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})} \tag{14.9}$$

By Jensen's inequality (log is concave):

$$\log p_\theta(\mathbf{x}) \geq \sum_{\mathbf{z}} q_\phi(\mathbf{z} \mid \mathbf{x}) \log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})} = \text{ELBO} \tag{14.10}$$

Expanding:

$$\text{ELBO} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]}_{\text{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z}))}_{\text{regularization}} \tag{14.11}$$

For VQ-VAE specifically:

The posterior is **deterministic** (argmin quantization): $q_\phi(\mathbf{z} = \mathbf{e}_k \mid \mathbf{x}) = \mathbf{1}[k = \arg\min_j \|\text{enc}(\mathbf{x}) - \mathbf{e}_j\|]$

With a uniform prior $p(\mathbf{z}) = 1/K$ over K codebook entries:

$$D_{\text{KL}} = \log K - H(q_\phi) = \log K - 0 = \log K \quad \text{(constant)} \tag{14.12}$$

Since the KL term is constant, maximizing ELBO reduces to minimizing reconstruction loss + keeping encoder close to codebook entries:

$$\mathcal{L}_{\text{VQ-VAE}} = \|\mathbf{x} - \text{dec}(\mathbf{e}_{k^*})\|^2 + \|\text{sg}(\text{enc}(\mathbf{x})) - \mathbf{e}_{k^*}\|^2 + \beta\|\text{enc}(\mathbf{x}) - \text{sg}(\mathbf{e}_{k^*})\|^2 \tag{14.13}$$

■

14.4. B.4 Conditional Flow Matching Derivation

14.4.1. B.4.1 Statement

The Conditional Flow Matching (CFM) objective provides a simulation-free training procedure for continuous normalizing flows.

14.4.2. B.4.2 Setup

We want to learn a time-dependent velocity field $v_\theta(\mathbf{x}, t)$ that transforms a simple distribution p_0 (e.g., Gaussian) into a target distribution p_1 (e.g., data) via the ODE:

$$\frac{d\mathbf{x}_t}{dt} = v_\theta(\mathbf{x}_t, t), \quad t \in [0, 1] \tag{14.14}$$

14.4.3. B.4.3 Optimal Transport Path

Choose the **linear interpolation path** (optimal transport):

$$\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1, \quad \mathbf{x}_0 \sim p_0, \quad \mathbf{x}_1 \sim p_1 \tag{14.15}$$

The conditional velocity field generating this path is:

$$u_t(\mathbf{x} \mid \mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0 \tag{14.16}$$

Proof: Taking the time derivative of the interpolation:

$$\frac{d\mathbf{x}_t}{dt} = \frac{d}{dt}[(1-t)\mathbf{x}_0 + t\mathbf{x}_1] = -\mathbf{x}_0 + \mathbf{x}_1 = \mathbf{x}_1 - \mathbf{x}_0 \tag{14.17}$$

14.4.4. B.4.4 Training Objective

The CFM loss matches the learned velocity to the conditional velocity:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],\mathbf{x}_0\sim p_0,\mathbf{x}_1\sim p_1}\left[\|v_\theta(\mathbf{x}_t,t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2\right] \tag{14.18}$$

Key result [20]: Minimizing this conditional objective is equivalent to minimizing the **marginal** flow matching objective:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,\mathbf{x}_t\sim p_t}\left[\|v_\theta(\mathbf{x}_t,t) - u_t(\mathbf{x}_t)\|^2\right] \tag{14.19}$$

The proof relies on the fact that $\nabla_\theta\mathcal{L}_{\text{CFM}} = \nabla_\theta\mathcal{L}_{\text{FM}}$ (gradients are equal), making CFM a valid training procedure. ■

14.5. B.5 Gumbel-Softmax Gradient Estimator

14.5.1. B.5.1 Statement

Gumbel-Softmax provides a differentiable approximation to sampling from a categorical distribution, enabling gradient-based optimization through discrete choices.

14.5.2. B.5.2 Gumbel-Max Trick

To sample from a categorical distribution with logits $\ell_1, \ldots, \ell_K$:

$$k^* = \arg\max_k [\ell_k + g_k], \quad g_k \sim \text{Gumbel}(0,1) \tag{14.20}$$

where $g_k = -\log(-\log(u_k)), \quad u_k \sim \text{Uniform}(0,1)$.

14.5.3. B.5.3 Continuous Relaxation

Replace $\arg\max$ with softmax at temperature τ:

$$y_k = \frac{\exp((\ell_k + g_k)/\tau)}{\sum_{j=1}^{K}\exp((\ell_j + g_j)/\tau)} \tag{14.21}$$

Properties:

- As $\tau \to 0$: $\mathbf{y}$ approaches one-hot (exact categorical sample)

- As $\tau \to \infty$: $\mathbf{y}$ approaches uniform distribution
- For any $\tau > 0$: $\mathbf{y}$ is differentiable w.r.t. ℓ_k

Proof of convergence:

$$\lim_{\tau \to 0} y_k = \begin{cases} 1 & \text{if } k = \arg\max_j (\ell_j + g_j) \\ 0 & \text{otherwise} \end{cases} \tag{14.22}$$

This holds because as $\tau \to 0$, softmax concentrates all mass on the maximum element. ■

14.5.4. B.5.4 Application in Wav2Vec 2.0

Wav2Vec 2.0 uses Gumbel-Softmax to make codebook selection differentiable during training:

$$\mathbf{q} = \sum_{k=1}^{K} y_k \cdot \mathbf{e}_k, \quad y_k = \text{GumbelSoftmax}(\ell_k, \tau) \tag{14.23}$$

Temperature τ is annealed from 2.0 to 0.5 during training.

14.6. B.6 ELBO for VITS

14.6.1. B.6.1 Statement

VITS maximizes a variational lower bound on $\log p_\theta(\mathbf{x} \mid c)$ where c is text conditioning.

14.6.2. B.6.2 Derivation

$$\log p_\theta(\mathbf{x} \mid c) = \log \int p_\theta(\mathbf{x} \mid \mathbf{z}) p_\theta(\mathbf{z} \mid c)\, d\mathbf{z} \tag{14.24}$$

Introducing variational posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$:

$$\log p_\theta(\mathbf{x} \mid c) \geq \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]}_{\text{reconstruction (HiFi-GAN)}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \| p_\theta(\mathbf{z} \mid c))}_{\text{prior matching}} \tag{14.25}$$

The normalizing flow in VITS transforms the posterior to make it more flexible:

$$q_\phi(\mathbf{z}_K \mid \mathbf{x}) = q_\phi(\mathbf{z}_0 \mid \mathbf{x}) \prod_{k=1}^{K} \left| \det \frac{\partial f_k^{-1}}{\partial \mathbf{z}_k} \right| \tag{14.26}$$

This allows q_ϕ to model complex, multi-modal posterior distributions while maintaining tractable density evaluation. ■

15. NLP Speech Complete Mapping

15.1. C.1 Concept Mapping Table

Bảng ánh xạ đầy đủ giữa các khái niệm NLP và Speech — giúp NLP researchers nhanh chóng tìm được counterpart trong speech domain.

15.1.1. C.1.1 Data & Representation

NLP Concept	Speech Counterpart	Notes
Raw text (string)	Raw waveform (1D signal)	Text: discrete characters. Audio: continuous samples
Character	Audio sample (16-bit, 16kHz)	Smallest unit
Word / Subword (BPE)	Phoneme / Syllable	Linguistic unit
Vocabulary (32K–128K)	Codebook (1024 × 8 RVQ layers)	Discrete symbol set
Token ID (integer)	Codec token (integer per codebook)	Index into vocabulary/codebook
Token embedding	Audio frame embedding	Dense vector per unit
Sentence	Utterance	Complete expression
Document	Audio recording / Episode	Long-form content
Corpus	Speech corpus (hours)	Training data
Text length (tokens)	Duration (seconds / frames)	Sequence measure

15.1.2. C.1.2 Preprocessing

NLP	Speech	Notes
Tokenization (BPE/WordPiece)	Feature extraction (Mel/MFCC)	Convert raw → model input
Lowercasing	Pre-emphasis filter	Normalization
Stopword removal	Silence removal (VAD)	Remove uninformative parts
Text normalization	Audio normalization	Standardize input
Data augmentation (back-translation)	SpecAugment (time/freq masking)	Training regularization
Positional encoding	Sinusoidal / Relative position	Encode sequence order

15.1.3. C.1.3 Model Architectures

NLP Model	Speech Model	Shared Principle
BERT	Wav2Vec 2.0 / HuBERT	Masked prediction pre-training
GPT (autoregressive LM)	AudioLM / VALL-E	Next-token prediction
T5 (encoder-decoder)	Whisper	Seq2seq with cross-attention
Transformer encoder	Conformer	Feature extraction backbone
Transformer decoder	Tacotron 2 decoder	Autoregressive generation
VAE	VQ-VAE / VITS	Latent variable model
Diffusion model	Grad-TTS / F5-TTS	Iterative denoising/flow
GAN (text)	HiFi-GAN / WaveGAN	Adversarial training

15.1.4. C.1.4 Training Paradigms

NLP Paradigm	Speech Paradigm	Details
Masked Language Model (MLM)	Masked speech prediction	Mask → predict (BERT HuBERT)
Causal LM (next token)	Codec LM (next audio token)	GPT AudioLM
Contrastive learning (SimCLR)	Contrastive learning (Wav2Vec 2.0)	Positive/negative pairs
Knowledge distillation	Distil-Whisper	Teacher → Student
RLHF (preference learning)	MOS-based optimization	Human feedback
Instruction tuning	Audio instruction tuning	Qwen2-Audio Stage 2
Few-shot prompting	Zero-shot voice cloning (3s)	In-context learning

15.1.5. C.1.5 Tasks

NLP Task	Speech Task	Mapping
Text classification	Audio classification	Classify input
Named Entity Recognition	Speaker diarization	Segment + label
Machine Translation	Speech Translation	Cross-lingual
Text Generation	Speech Synthesis (TTS)	Generate output
Summarization	Audio summarization	Compress information
Question Answering	Audio QA	Answer from input
Sentiment Analysis	Emotion Recognition	Detect affect
Language Identification	Language ID from audio	Detect language
Text-to-Text	Speech-to-Speech	Direct transformation

15.1.6. C.1.6 Losses & Metrics

NLP	Speech	Notes
Cross-entropy (token prediction)	Cross-entropy (CTC, token prediction)	Standard classification loss
Perplexity	—	LM evaluation
BLEU / ROUGE	WER / CER	Generation quality
BERTScore	—	Semantic similarity
—	MOS (Mean Opinion Score)	Human perceptual quality
—	PESQ / ViSQOL	Automated audio quality
—	Speaker similarity (cosine)	Voice cloning evaluation
KL divergence	KL divergence (VAE)	Distribution matching

15.1.7. C.1.7 Production & Deployment

NLP Production	Speech Production	Notes
Tokenizer latency	Feature extraction latency	Preprocessing time
KV cache (autoregressive)	KV cache (Whisper decoder)	Memory optimization
Quantization (INT8/INT4)	Quantization (INT8/FP16)	Model compression
Batched inference	Dynamic batching	Throughput optimization
Streaming (token-by-token)	Streaming ASR/TTS	Real-time processing
vLLM / TGI	TensorRT-LLM + Triton	Inference servers

NLP Production	Speech Production	Notes
Context window limit	Max audio duration (30s)	Input length constraint
Token budget	Latency budget (<1s)	Resource constraint

15.2. C.2 Architecture Mapping Diagram

```
NLP Pipeline:                    Speech Pipeline:

Text Input                       Audio Input (waveform)

Tokenizer (BPE)                  Feature Extraction (Mel)

Token Embeddings                 Frame Embeddings

  Transformer                      Conformer/
  Encoder                          Transformer Enc
  (BERT/RoBERTa)                   (Wav2Vec/Whisper)

  Contextual                       Contextual Audio
  Embeddings                       Representations

  [Task Head]                      [Task Head]
  - Classification                 - CTC/Transducer (ASR)
  - Generation                     - Mel prediction (TTS)
  - QA                             - Token prediction (LM)
```

15.3. C.3 Scale Comparison

Dimension	NLP	Speech	Ratio
Input dim per step	1 (token ID)	80 (mel) or 16000 (raw)	80–16000×
Typical sequence length	512–4096 tokens	1500 frames (30s) or 480K samples	~1–100×
Vocabulary size	32K–128K	1024 (per codebook) × 8	4–16×
Pre-training data	~1–15T tokens	~680K hours	Different units
Model size (SOTA)	70B–405B	1.5B–7B	10–100× smaller
Inference speed	~50 tokens/s	Real-time (RTF 1)	Different metrics
Human eval	Preference ranking	MOS (1–5 scale)	Different scales

16. Code Listings Index

16.1. D.1 Overview

Tất cả code trong cuốn sách sử dụng:

- **Python 3.10** với 100% type hints
- **PyTorch 2.0** và **torchaudio**
- Inline tensor shape comments: `# [batch, seq, dim] - dtype`
- Code blocks có thể fold/unfold trong HTML output

16.2. D.2 Part I: Foundations

16.2.1. Chapter 1 — From NLP to Speech

Listing	Description	Key Types
1.1	Feature comparison (text vs audio)	`Tensor [1, T]`, `Tensor [80, T']`

16.2.2. Chapter 2 — Audio Signal Fundamentals

Listing	Description	Key Types
2.1	DFT implementation	`Tensor [N] → Tensor [N] (complex64)`
2.2	STFT with windowing	`Tensor [T] → Tensor [n_freq, n_frames] (complex64)`
2.3	Mel filterbank	`Tensor [n_mels, n_fft//2+1] - float32`
2.4	MFCC extraction	`Tensor [n_mfcc, T'] - float32`
2.5	SpecAugment	`Tensor [n_mels, T'] → Tensor [n_mels, T']`

Listing	Description	Key Types
2.6	Complete feature pipeline	`AudioFeatureExtractor` class

16.3. D.3 Part II: ASR

16.3.1. Chapter 3 — ASR Foundations

Listing	Description	Key Types
3.1	CTC Model	`CTCModel(nn.Module)` — encoder + CTC head
3.2	CTC greedy decode	`Tensor [B, T, V] → list[list[int]]`
3.3	RNN-Transducer	`RNNTransducer(nn.Module)` — encoder + predictor + joint

16.3.2. Chapter 4 — Self-Supervised Speech

Listing	Description	Key Types
4.1	Gumbel Vector Quantizer	`GumbelVectorQuantizer(nn.Module)`
4.2	Wav2Vec 2.0 Conv Encoder	`Wav2Vec2ConvEncoder(nn.Module)`
4.3	Conformer Conv Module	`ConformerConvModule(nn.Module)`
4.4	Conformer Block	`ConformerBlock(nn.Module)`
4.5	SSL for ASR (fine-tuning)	`SSLForASR(nn.Module)`

16.3.3. Chapter 5 — Whisper & Modern ASR

Listing	Description	Key Types
5.1	Whisper Encoder	`WhisperEncoder(nn.Module)` — Conv + Transformer
5.2	Whisper Decoder	`WhisperDecoder(nn.Module)` — Causal Transformer

16.4. D.4 Part III: TTS & Audio Codecs

16.4.1. Chapter 6 — TTS Foundations & Vocoders

Listing	Description	Key Types
6.1	Variance Predictor	`VariancePredictor(nn.Module)` — duration/pitch/energy
6.2	Length Regulator	`LengthRegulator(nn.Module)` — phoneme → mel expansion
6.3	HiFi-GAN ResBlock	`ResBlock(nn.Module)` — dilated convolutions
6.4	HiFi-GAN Generator	`HiFiGANGenerator(nn.Module)` — mel → waveform

16.4.2. Chapter 7 — End-to-End TTS

Listing	Description	Key Types
7.1	VITS Posterior Encoder	`PosteriorEncoder(nn.Module)` — audio → latent
7.2	Flow Matching Sampling	`flow_matching_sample()` — ODE Euler solver

16.4.3. Chapter 8 — Audio Codecs

Listing	Description	Key Types
8.1	Vector Quantize	`VectorQuantize(nn.Module)` — single codebook VQ
8.2	Residual VQ	`ResidualVQ(nn.Module)` — Q-layer RVQ
8.3	EnCodec Encoder	`EnCodecEncoder(nn.Module)` — strided conv encoder

16.5. D.5 Part IV: Speech LLMs & Vietnamese

16.5.1. Chapter 9 — Speech Language Models

Listing	Description	Key Types
9.1	Dual-Stream Generator	`DualStreamGenerator(nn.Module)` — Moshi-style

16.5.2. Chapter 10 — Vietnamese Speech Processing

Listing	Description	Key Types
10.1	Whisper Vietnamese prep	`prepare_whisper_vietnamese_traini`
10.2	Vietnamese text normalizer	`normalize_vietnamese_text()`

16.6. D.6 Part V: Production

16.6.1. Chapter 11 — Production Speech Systems

Listing	Description	Key Types
11.1	VAD Segmenter	`VADSegmenter` — streaming voice detection
11.2	Dynamic Batcher	`DynamicBatcher` — batch ASR requests

16.7. D.7 Dependencies

```
# Core
torch>=2.0.0
torchaudio>=2.0.0

# Audio processing
numpy>=1.24.0
scipy>=1.10.0
librosa>=0.10.0

# Vietnamese NLP (Chapter 10)
underthesea>=6.0.0

# Inference optimization (Chapter 11)
# tensorrt-llm (optional, NVIDIA GPU required)
# triton-client (optional, for Triton server)
```

16.8. D.8 Running the Code

Tất cả code listings được set `eval: false` — chúng là **reference implementations** minh họa architecture, không phải production-ready code. Để chạy:

```
# Setup environment
python -m venv speech-ai-env
source speech-ai-env/bin/activate
pip install torch torchaudio numpy scipy

# Copy code from any listing and run
python listing_example.py
```

> **Code Convention**
>
> Mọi tensor operation đều có comment theo format:
>
> ```
> result: Tensor = operation(input)  # [dim1, dim2, dim3] - dtype
> ```
>
> Ví dụ: `# [batch, seq_len, d_model] - float32` nghĩa là tensor shape `[B, L, D]` với dtype `torch.float32`.

Tài liệu Tham khảo

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *International Conference on Machine Learning*, pp. 28492–28518, 2023.

[2] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," *Advances in Neural Information Processing Systems*, vol. 33, pp. 12449–12460, 2020.

[3] J. Kim, J. Kong, and J. Son, "Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," *International Conference on Machine Learning*, pp. 5530–5540, 2021.

[4] C. Wang *et al.*, "Neural codec language models are zero-shot text to speech synthesizers," *arXiv preprint arXiv:2301.02111*, 2023.

[5] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *Transactions on Machine Learning Research*, 2023.

[6] A. Défossez *et al.*, "Moshi: A speech-text foundation model for real-time dialogue," *arXiv preprint arXiv:2410.00037*, 2024.

[7] Y. Chu *et al.*, "Qwen-audio: Advancing universal audio understanding via unified large-scale audio-language models," *arXiv preprint arXiv:2311.07919*, 2023.

[8] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.

[9] Z. Borsos *et al.*, "AudioLM: A language modeling approach to audio generation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.

[10] A. Gulati *et al.*, "Conformer: Convolution-augmented transformer for speech recognition," *Interspeech*, pp. 5036–5040, 2020.

[11] D. S. Park *et al.*, "SpecAugment: A simple data augmentation method for automatic speech recognition," *Interspeech*, pp. 2613–2617, 2019.

[12] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," *International Conference on Machine Learning*, pp. 369–376, 2006.

[13] A. Graves, "Sequence transduction with recurrent neural networks," *arXiv preprint arXiv:1211.3711*, 2012.

[14] S. Gandhi, P. von Platen, and A. M. Rush, "Distil-whisper: Robust knowledge distillation via large-scale pseudo labelling," *arXiv preprint arXiv:2311.00430*, 2023.

[15] J. Shen *et al.*, "Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions," *IEEE International Conference on Acoustics, Speech and Signal Processing*, pp. 4779–4783, 2018.

[16] Y. Ren *et al.*, "FastSpeech 2: Fast and high-quality end-to-end text to speech," *International Conference on Learning Representations*, 2021.

[17] A. van den Oord *et al.*, "WaveNet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.

[18] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," *Advances in Neural Information Processing Systems*, vol. 33, pp. 17022–17033, 2020.

[19] Y. Chen *et al.*, "F5-TTS: A fairytaler that fakes fluent and faithful speech with flow matching," *arXiv preprint arXiv:2410.06885*, 2024.

[20] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," *International Conference on Learning Representations*, 2023.

[21] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[22] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, "High-fidelity audio compression with improved RVQGAN," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[23] H.-T. Luong and H.-Q. Vu, "A non-expert kaldi recipe for vietnamese speech recognition system," *Workshop on Spoken Language Technologies for Under-resourced Languages*, 2016.